

KI-unterstützte Erstellung von ETL-Prozessen - Eine experimentelle Evaluierung

Bachelorarbeit

zur Erlangung des Grades

Bachelor of Science

der Fachrichtung Informatik

an der Dualen Hochschule Baden-Württemberg (DHBW) Stuttgart

von

Luis Geiger

07.09.2026

Matrikelnummer:

9144849

Kurs:

TINF23CE

Gutachter an der DHBW Stuttgart:

Pawel Wójcik

Erklärung

Ich erkläre mich hiermit ehrenwörtlich, dass ich die vorliegende Arbeit Bachelorarbeit mit dem Thema „*KI-unterstützte Erstellung von ETL-Prozessen - Eine experimentelle Evaluierung*“ selbständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Aus den benutzten Quellen, direkt oder indirekt, übernommene Gedanken habe ich als solche kenntlich gemacht. Diese Arbeit wurde bisher in gleicher oder ähnlicher Form oder auszugsweise noch keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Stuttgart, 07.09.2026

Ort, Datum

L.Geiger

Luis Geiger

Abstract

Klassische Extract, Transform, Load (ETL)-Prozesse sind regelbasiert und setzen voraus, dass jede Verarbeitungsvorschrift zuvor vollständig formuliert wird. Die vorliegende Arbeit untersucht experimentell, inwieweit Large Language Models (LLMs) diese Aufgabe übernehmen können. Auf einem synthetischen Datensatz mit kontrolliert eingebauten Qualitätsproblemen werden vier Modelle, drei Prompting-Strategien und neun ETL-Aufgaben gegen eine regelbasiert implementierte Vergleichspipeline gemessen, ergänzt um eine verkettete Ausführung, einen isolierten Durchlauf zur Trennung von eigener Schrittleistung und geerbtem Fehler, einen Vergleich zweier Ausführungsformen sowie zwei Ebenen zum Zuschnitt der Aufgabenstellung. Insgesamt liegen 711 Modellaufrufe zugrunde.

Die Ergebnisqualität hängt weit stärker vom Aufgabentyp ab als vom Modell: Zwischen dem besten und dem schwächsten kommerziellen Modell liegen rund acht Prozentpunkte, zwischen der am besten und der am schlechtesten gelösten Aufgabe nahezu hundert. Als zweite Einflussgröße erweist sich die Ausführungsform. Der erwartete Vorteil bei semantisch geprägten Aufgaben tritt allein in der direkten Verarbeitung ein, während die Code-Generierung zuverlässiger rechnet und je Zeile rund ein Fünfzehntel kostet. Am stärksten wirkt der Zuschnitt der Aufgabe: Dieselbe Verarbeitung bleibt als eine zusammengesetzte Aufgabe ungelöst und wird in Teilschritte zerlegt fehlerfrei bewältigt, wobei der messbare Unterschied zwischen den Prompting-Strategien dabei vollständig verschwindet. Die Arbeit leitet daraus eine hybride Architektur ab, die regelbasierte und modellgestützte Schritte je nach Aufgabentyp kombiniert und nach jedem Schritt inhaltlich prüft.

Classical ETL processes are rule-based and require every transformation rule to be specified in advance. This thesis experimentally evaluates to what extent LLMs can take over that task. Using a synthetic dataset with deliberately injected quality defects, four models, three prompting strategies and nine ETL tasks are measured against a rule-based reference pipeline, complemented by a chained execution, an isolated run separating a step's own performance from inherited error, a comparison of two execution modes, and two levels addressing how the task itself is framed. The evaluation rests on 711 model calls.

Result quality depends far more on the type of task than on the model: roughly eight

percentage points separate the best from the weakest commercial model, whereas nearly a hundred separate the best-solved from the worst-solved task. The execution mode emerges as a second decisive factor. The expected advantage on semantically demanding tasks materialises only under direct processing, while code generation computes more reliably at about one fifteenth of the cost per row. The strongest lever, however, is task framing: the same transformation remains unsolved when posed as one composite task and is completed without error once decomposed into individual steps, at which point the measurable difference between prompting strategies disappears entirely. On this basis the thesis derives a hybrid architecture that combines rule-based and model-based steps according to task type and validates the result after every step.

Inhaltsverzeichnis

Abstract	II
Abbildungsverzeichnis	VII
Tabellenverzeichnis	VII
Skriptverzeichnis	VII
Abkürzungsverzeichnis	VIII
1. Einleitung	1
1.1. Motivation und Problemstellung	1
1.2. Zielsetzung und Forschungsfragen	2
1.3. Methodisches Vorgehen	3
1.4. Aufbau der Arbeit	3
2. Grundlagen	4
2.1. ETL-Prozesse	5
2.1.1. Definition und Phasen	5
2.1.2. Typische Herausforderungen	6
2.1.3. Klassische Werkzeuge und Verfahren	6
2.2. Datenqualität und Datenbereinigung	7
2.2.1. Dimensionen der Datenqualität	7
2.2.2. Häufige Datenprobleme	8
2.3. Künstliche Intelligenz und maschinelles Lernen	8
2.4. Large Language Models	9
2.4.1. Funktionsweise und Architektur	9
2.4.2. Überblick aktueller Modelle	10
2.4.3. Stärken und Limitationen bei strukturierten Daten	10
2.5. Prompt Engineering	11
2.5.1. Grundlegende Prompting-Strategien	11
2.5.2. Einflussfaktoren auf die Ergebnisqualität	12
3. Stand der Forschung	12
3.1. KI-Einsatz in der Datenverarbeitung	13
3.2. LLMs für strukturierte Daten und synthetische Datensätze	13
3.3. Verwandte Arbeiten zur Automatisierung von ETL	14
3.4. Forschungslücke und Abgrenzung	15

4. Konzeption der experimentellen Evaluierung	16
4.1. Forschungsdesign und Hypothesen	17
4.2. Auswahl der Large Language Models	20
4.3. Definition der ETL-Aufgaben	22
4.3.1. Datenbereinigung	22
4.3.2. Datentransformation	23
4.3.3. Duplikaterkennung	23
4.4. Prompting-Strategien	24
4.5. Evaluationskriterien	24
4.5.1. Genauigkeit	25
4.5.2. Vollständigkeit und Konsistenz	26
4.5.3. Effizienz	27
4.6. Vergleich gegen eine manuelle ETL-Pipeline	27
5. Erstellung des synthetischen Datensatzes	28
5.1. Anforderungen an den Datensatz	28
5.2. Datenmodell und -struktur	29
5.3. Eingebaute Datenqualitätsprobleme	30
5.4. Generierungsverfahren und verwendete Werkzeuge	33
5.5. Validierung des Datensatzes	34
6. Implementierung	35
6.1. Systemarchitektur und Versuchsumgebung	35
6.2. Eingesetzte Technologien und Bibliotheken	36
6.3. Umsetzung der klassischen ETL-Pipeline	37
6.4. Integration der LLMs in die Pipeline	38
6.5. Umsetzung der Prompting-Strategien	40
6.6. Mess- und Auswertungsverfahren	40
7. Durchführung der Experimente	42
7.1. Versuchsablauf	42
7.2. Erhobene Messdaten	44
7.3. Reproduzierbarkeit und Versuchsbedingungen	45
8. Ergebnisse und Auswertung	46
8.1. Ergebnisse je ETL-Aufgabe	47
8.1.1. Datenbereinigung	49
8.1.2. Duplikaterkennung	51
8.1.3. Datentransformation	52
8.2. Vergleich der Modelle	53

8.3. Vergleich der Prompting-Strategien	56
8.4. Ergebnisse der verketteten Pipeline	58
8.4.1. Qualitätsverlauf über die Kette	58
8.4.2. Trennung von eigener Leistung und Fehlerfortpflanzung	60
8.5. Reproduzierbarkeit der Ergebnisse	61
8.6. Der Zuschnitt der Aufgabenstellung	63
8.6.1. Gedrittelte Strecke	64
8.6.2. Explizite statt impliziter Anforderungen	65
8.7. Vergleich der Ausführungsmodi	68
8.8. Vergleich KI-basiert vs. klassisch	70
8.9. Diskussion der Ergebnisse	71
8.9.1. Stärken, Schwächen und Risiken	71
8.9.2. Einordnung in den Stand der Forschung	72
9. Handlungsempfehlungen für die Praxis	75
9.1. Eignung von LLMs für unterschiedliche ETL-Szenarien	75
9.2. Empfehlungen zur Modell- und Prompt-Auswahl	77
9.3. Hybride Architekturen	79
10. Fazit und Ausblick	81
10.1. Zusammenfassung der Ergebnisse	81
10.2. Beantwortung der Forschungsfragen	83
10.3. Limitationen der Arbeit	85
10.4. Ausblick auf weitere Forschung	87
Literatur	89
A. Anhang	94
A.1. Prompt-Vorlagen	94
A.2. Aufgabentexte der neun ETL-Aufgaben	96
A.3. Aufgabentext der explizit gegliederten Variante	97
A.4. Repositorium	98

Abbildungsverzeichnis

8.1. Bestes Ergebnis je Aufgabe und Ansatz	48
8.2. Modellvergleich über alle Läufe	54
8.3. Prompting-Strategien je Aufgabe	57
8.4. Qualitätsverlauf der Pipeline	59
8.5. Streuung des Endschritts über fünf Wiederholungen	62
8.6. Wirkung des Aufgabenzuschnitts	64
8.7. Wirkung einer expliziten Aufgabenstellung	66
8.8. Aufwand der beiden Ausführungsmodi	69

Tabellenverzeichnis

4.1. Untersuchungsebenen des Versuchsdesigns	20
4.2. Untersuchte Modelle	21
5.1. Beispiele verfälschter Werte im Datensatz	32
5.2. Beispiele eingefügter Duplikate	33
8.1. Ergebnisse je ETL-Aufgabe	49
9.1. Eignungsmatrix nach Aufgabentyp	77

Skriptverzeichnis

8.1. Zwei lauffähige Lösungen derselben Bereinigungsaufgabe: Die obere Variante bleibt in der eingesetzten Pandas-Version wirkungslos, weil Textspalten nicht mehr als Objektspalten geführt werden. Die untere benennt die Spalten explizit.	50
---	----

Abkürzungsverzeichnis

ELT Extract, Load, Transform

ETL Extract, Transform, Load

KI Künstliche Intelligenz

LLM Large Language Model

ML Maschinelles Lernen

VRAM Video Random Access Memory

1. Einleitung

1.1. Motivation und Problemstellung

Die Verfügbarkeit, Qualität und zeitnahe Bereitstellung von Daten sind zu einem zentralen Erfolgsfaktor für datengetriebene Organisationen geworden. Im Zentrum der dafür notwendigen Dateninfrastruktur stehen seit Jahrzehnten ETL-Prozesse, die Daten aus heterogenen Quellsystemen extrahieren, in eine einheitliche Struktur überführen und in Zielsysteme wie Data Warehouses oder Analyseplattformen laden [1, 2]. Trotz ihrer langen Etablierung gelten klassische ETL-Pipelines zunehmend als Engpass: Sie sind in der Regel regelbasiert, statisch modelliert und erfordern erheblichen manuellen Aufwand bei Entwurf, Anpassung und Wartung [2, 3]. Mit dem Zuwachs von Datenvolumen, der Heterogenität der Quellen und steigenden Anforderungen an Aktualität geraten herkömmliche ETL-Architekturen an ihre Grenzen [2, 4]. Parallel dazu haben sich in den letzten Jahren LLMs als leistungsfähige Werkzeuge etabliert, die neben natürlicher Sprache auch strukturierte und semi-strukturierte Daten verarbeiten können [5]. Arbeiten zeigen, dass LLMs im Kontext der Datenverarbeitung Aufgaben wie Datenbereinigung, Schema-Mapping, Transformationslogik oder Anomalieerkennung übernehmen oder zumindest unterstützen können [6, 7]. Damit eröffnet sich die Perspektive, klassische ETL-Prozesse durch Künstliche Intelligenz (KI)-gestützte Komponenten zu ergänzen oder partiell zu ersetzen, um manuellen Aufwand zu reduzieren und die Anpassungsfähigkeit der Pipelines zu erhöhen. Der praktische Anreiz liegt dabei weniger in der technischen Machbarkeit an sich als vielmehr in der Aussicht auf Effizienz- und Kostenvorteile: Übernehmen Modelle Routineschritte der Transformationslogik ganz oder unterstützend, lässt sich der zeitintensive und damit kostenträchtige manuelle Entwicklungs- und Wartungsaufwand spürbar verringern, was den Einsatz gerade im wirtschaftlichen Kontext motiviert. Gleichzeitig ist die praktische Eignung von LLMs für ETL-Aufgaben bislang nur unzureichend systematisch untersucht. Während konzeptionelle Beiträge die potenziellen Vorteile beschreiben, fehlt es an empirischen Vergleichen, die unterschiedliche Modelle und Prompting-Strategien unter kontrollierten Bedingungen gegen klassische Verfahren stellen. Offen ist insbesondere, wie zuverlässig LLMs typische ETL-Teilaufgaben wie Datenbereinigung, Datentransformation und Duplikaterkennung bearbeiten, wie stabil und reproduzierbar ihre Ergebnisse aufgrund der probabilistischen Modellarchitektur ausfallen, wie sich verschiedene Prompting-Ansätze auf die Ergebnisqualität auswirken und unter welchen Bedingungen ein Einsatz gegenüber etablierten regelbasierten Methoden sinnvoll ist. Genau

an diesem Forschungsdefizit setzt die vorliegende Arbeit an. Sie betrachtet dabei den autonomen Einsatz der Modelle, also die Verarbeitung ohne menschliche Nachbearbeitung des erzeugten Ergebnisses. Damit bleibt eine weitere offene Frage bewusst außerhalb des Untersuchungsrahmens: wie sich ein solcher Einsatz zum heutigen De-facto-Standard der Softwareentwicklung verhält, bei dem KI nicht autonom, sondern als Assistenz einer menschlichen Fachkraft wirkt. Die hier erhobenen Werte beschreiben folglich, was ohne Nachbearbeitung entsteht, und markieren damit die Untergrenze dessen, was im assistierten Einsatz zu erwarten wäre. Auf diese Abgrenzung wird in Kapitel 10 zurückzukommen sein.

1.2. Zielsetzung und Forschungsfragen

Ziel dieser Arbeit ist eine experimentelle Evaluierung, inwieweit sich aktuelle LLMs für ausgewählte ETL-Aufgaben eignen, sei es als eigenständige Automatisierung oder als unterstützendes Werkzeug einer menschlichen Fachkraft. Im Zentrum steht der systematische Vergleich mehrerer aktueller LLMs und unterschiedlicher Prompting-Strategien, bewertet anhand von Genauigkeit, Vollständigkeit, Konsistenz und Effizienz bei der Lösung der definierten ETL-Teilaufgaben und ausdrücklich nicht als allgemeiner Leistungsvergleich der Modelle. Grundlage ist ein kontrolliert erstellter synthetischer Datensatz, der typische Datenqualitätsprobleme abbildet. Die Ergebnisse werden mit einer klassisch implementierten ETL-Pipeline verglichen, um Stärken, Schwächen und praktische Einsatzgrenzen KI-gestützter Ansätze fundiert einordnen zu können. Auf dieser Grundlage sollen Handlungsempfehlungen für den Einsatz von LLMs in realen ETL-Szenarien abgeleitet werden. Aus dieser Zielsetzung ergeben sich zwei leitende Forschungsfragen:

- FF1:** Inwieweit eignen sich aktuelle LLMs für die Bearbeitung typischer ETL-Teilaufgaben – Datenbereinigung, Datentransformation und Duplikaterkennung – im Vergleich zu einer klassischen, regelbasierten Pipeline, wie hängt diese Eignung neben dem Aufgabentyp von der Ausführungsform ab – also davon, ob das Modell Verarbeitungscode erzeugt oder die Daten unmittelbar verarbeitet und wie die Aufgabe zugeschnitten ist –, und welches Vorgehen legen die Ergebnisse je Aufgabentyp nahe?
- FF2:** Wie stark beeinflusst die Wahl der Prompting-Strategie die Ergebnisqualität der untersuchten LLMs, und welche Strategien sind für die betrachteten ETL-Aufgaben zu empfehlen?

Da LLMs probabilistische Ausgaben erzeugen, wird die Reproduzierbarkeit ihrer Ergebnisse nicht als eigenständige Forschungsfrage, sondern als querschnittliche Untersuchungsdimension über beide Fragen hinweg behandelt: Jede Konfiguration wird mehrfach ausgeführt, sodass die Stabilität der Ergebnisse durchgängig mitbewertet werden kann.

1.3. Methodisches Vorgehen

Die Arbeit folgt einem experimentellen Vorgehen: Nach einer literaturbasierten Fundierung wird ein kontrolliertes Versuchsdesign entwickelt, dessen Ergebnisse gegen eine regelbasiert implementierte ETL-Pipeline als Maßstab gestellt werden. Grundlage aller Messungen ist ein synthetischer Datensatz mit gezielt eingebauten Qualitätsproblemen, dessen Soll-Lösung damit exakt bekannt ist. Gemessen wird auf mehreren Untersuchungsebenen, die nacheinander je einen Einflussfaktor freilegen: eine vollständig gekreuzte Faktormatrix aus Modell, Prompting-Strategie und Aufgabe; eine verkettete Pipeline aus neun aufeinander aufbauenden Schritten sowie ein sonst identischer isolierter Durchlauf, der eigene Schrittleistung von geerbtem Fehler trennt; ein Vergleich zweier Ausführungsformen, in denen das Modell entweder Verarbeitungscode erzeugt oder die Daten unmittelbar verarbeitet; und schließlich zwei Ebenen, die den Zuschnitt der Aufgabenstellung selbst zur Untersuchungsgröße machen. Da LLMs probabilistische Ausgaben erzeugen, wird jede Konfiguration mehrfach ausgeführt und die Streuung durchgängig mitberichtet. Die Auswertung verbindet die quantitativen Kennzahlen mit einer qualitativen Einordnung beobachteter Fehlermuster, aus der die abschließenden Handlungsempfehlungen abgeleitet werden.

1.4. Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in zehn Kapitel. Nach der vorliegenden Einleitung, in der Motivation, Zielsetzung, Forschungsfragen und methodisches Vorgehen dargelegt werden, behandelt Kapitel 2 die theoretischen Grundlagen. Hier werden zentrale Konzepte zu ETL-Prozessen, Datenqualität, LLMs sowie Prompt Engineering eingeführt, die als Bezugsrahmen für die weitere Untersuchung dienen. Kapitel 3 widmet sich dem aktuellen Stand der Forschung. Es werden Arbeiten zum Einsatz von KI in der Datenverarbeitung, zur Verarbeitung strukturierter Daten durch LLMs sowie zur Automatisierung von ETL-Prozessen aufgearbeitet und gegeneinander abgegrenzt. Auf dieser Basis wird die für diese Arbeit relevante Forschungslücke präzisiert. In Kapi-

tel 4 wird die Konzeption der experimentellen Evaluierung beschrieben. Dies umfasst das Forschungsdesign, die Auswahl der untersuchten Modelle, die Definition der ETL-Aufgaben, die eingesetzten Prompting-Strategien sowie die Evaluationskriterien einschließlich der manuellen Pipeline. Kapitel 5 dokumentiert anschließend die Erstellung des synthetischen Datensatzes, einschließlich Anforderungen, Datenmodell, eingebauter Qualitätsprobleme und der durchgeführten Validierung. Die technische Umsetzung wird in Kapitel 6 dargestellt. Es beschreibt die Systemarchitektur, die eingesetzten Technologien sowie die konkrete Implementierung der klassischen ETL-Pipeline und der LLM-basierten Komponenten. Kapitel 7 behandelt die Durchführung der Experimente und legt Versuchsablauf, erhobene Messdaten sowie die Bedingungen zur Sicherstellung der Reproduzierbarkeit offen. Kapitel 8 präsentiert und analysiert die Ergebnisse. Neben einer aufgabenspezifischen Auswertung erfolgen ein Vergleich der Modelle, der Prompting-Strategien sowie eine Gegenüberstellung von KI-basiertem und klassischem Ansatz. Die Ergebnisse werden anschließend diskutiert und in den Stand der Forschung eingeordnet. Aus den gewonnenen Erkenntnissen leitet Kapitel 9 Handlungsempfehlungen für die Praxis ab, insbesondere zur Eignung von LLMs in unterschiedlichen ETL-Szenarien sowie zur Gestaltung hybrider Architekturen. Kapitel 10 fasst die zentralen Ergebnisse zusammen, beantwortet die in Abschnitt 1.2 formulierten Forschungsfragen, reflektiert die Limitationen der Arbeit und gibt einen Ausblick auf weiterführende Forschungsfragen.

2. Grundlagen

Dieses Kapitel legt die theoretischen Grundlagen, auf denen die experimentelle Evaluierung der folgenden Kapitel aufbaut. Zunächst werden ETL-Prozesse als etabliertes Verfahren der Datenintegration beschrieben (Abschnitt 2.1). Anschließend wird das Konzept der Datenqualität als zentraler Bezugspunkt für die untersuchten ETL-Teilaufgaben eingeführt (Abschnitt 2.2). Darauf folgt eine Einordnung von Künstlicher Intelligenz und maschinellem Lernen als übergeordnetem Rahmen (Abschnitt 2.3), bevor Large Language Models als die in dieser Arbeit eingesetzte verarbeitende Komponente betrachtet werden (Abschnitt 2.4). Den Abschluss bildet eine Darstellung des Prompt Engineering als zentrale Steuerungsmethode für diese Modelle (Abschnitt 2.5).

2.1. ETL-Prozesse

2.1.1. Definition und Phasen

Der Begriff ETL steht für die drei aufeinanderfolgenden Phasen *Extract*, *Transform* und *Load* und bezeichnet einen der zentralen Prozesse der Datenintegration, mit dem Daten aus unterschiedlichen Quellsystemen in ein einheitliches Zielsystem überführt werden [1, 2]. Historisch ist der ETL-Prozess eng mit dem Aufbau von Data Warehouses verbunden, in denen konsolidierte und bereinigte Daten als Grundlage für Auswertungen und Entscheidungsunterstützung bereitgestellt werden [1, 8]. Der Prozess bildet damit das Bindeglied zwischen unverarbeiteten Rohdaten und den daraus abgeleiteten, nutzbaren Informationen [1, 2].

In der ersten Phase, der Extraktion, werden die relevanten Daten aus einer oder mehreren Quellen ausgelesen. Diese Quellen sind in der Praxis heterogen und reichen von relationalen Datenbanken über Dateien und Tabellen bis hin zu Schnittstellen externer Dienste, sodass bereits hier unterschiedliche Formate und Strukturen zusammengeführt werden müssen [1, 2]. Die Transformationsphase bildet den inhaltlichen Kern des Prozesses: Die extrahierten Daten werden bereinigt, vereinheitlicht und in die vom Zielsystem geforderte Struktur überführt. Typische Operationen sind die Korrektur fehlerhafter oder fehlender Werte, die Vereinheitlichung von Formaten, die Zusammenführung und Aggregation von Datensätzen, das Auflösen von Duplikaten sowie das Abbilden der Quellstrukturen auf das Zielschema [1, 2, 4]. In der abschließenden Ladephase werden die transformierten Daten schließlich in das Zielsystem geschrieben, etwa in ein Data Warehouse oder eine Analyseplattform, wobei zwischen vollständigem Laden und inkrementellem Aktualisieren unterschieden wird [1, 8].

Die strikte Abfolge der drei Phasen ist dabei nicht das einzig mögliche Gestaltungsmuster. Insbesondere im Kontext von Big Data hat sich mit Extract, Load, Transform (ELT) eine Variante etabliert, bei der die Rohdaten zunächst geladen und erst innerhalb des Zielsystems transformiert werden, um von dessen Verarbeitungskapazitäten zu profitieren [2, 4]. Für die vorliegende Arbeit bleibt jedoch die klassische ETL-Reihenfolge maßgeblich, da der Schwerpunkt auf den Transformationsschritten der Datenbereinigung, Datentransformation und Duplikaterkennung liegt.

2.1.2. Typische Herausforderungen

Obwohl ETL-Prozesse seit Jahrzehnten etabliert sind, gelten sie in modernen Datenlandschaften zunehmend als anspruchsvoll und wartungsintensiv. Diese Einschätzung gründet auf mehreren miteinander verbundenen Herausforderungen. Eine zentrale davon ist die Sicherung der Datenqualität: Da die Eingangsdaten aus unterschiedlichen Quellen stammen, weisen sie häufig Fehler, Lücken und Inkonsistenzen auf, die im Transformationsschritt erkannt und behoben werden müssen, um verlässliche Analyseergebnisse zu gewährleisten [1, 9]. Eng damit verbunden ist die Heterogenität der Quellen, die sich in unterschiedlichen Formaten, Schemata und Bedeutungen gleichartiger Felder äußert und einen erheblichen Aufwand bei der Vereinheitlichung verursacht [1, 2].

Hinzu kommt die Frage der Skalierbarkeit. Mit dem stetigen Wachstum von Datenvolumen und -vielfalt geraten klassische, batch-orientierte ETL-Architekturen an ihre Grenzen und können den Anforderungen an Verarbeitungsgeschwindigkeit und Aktualität nur noch eingeschränkt gerecht werden [2, 4]. Ein weiterer kritischer Aspekt ist der hohe manuelle Aufwand bei Entwurf, Anpassung und Wartung: Klassische Pipelines sind in der Regel regelbasiert und statisch modelliert, sodass jede Änderung an den Quellstrukturen oder den fachlichen Anforderungen einen manuellen Eingriff erfordert [2, 3]. Diese geringe Anpassungsfähigkeit führt dazu, dass ein Großteil der Arbeitszeit von Datenfachleuten in die Erstellung und Pflege solcher Pipelines fließt, was sie zu einem häufig benannten Engpass datengetriebener Organisationen macht [4, 10]. Genau diese Schwachstellen motivieren die in dieser Arbeit untersuchte Frage, inwieweit lernende und sprachbasierte Verfahren den manuellen Aufwand reduzieren und die Flexibilität klassischer ETL-Prozesse erhöhen können.

2.1.3. Klassische Werkzeuge und Verfahren

Zur Umsetzung von ETL-Prozessen hat sich über die Jahre ein breites Spektrum an Werkzeugen und Verfahren herausgebildet. Auf der einen Seite stehen dedizierte ETL-Plattformen mit grafischer Modellierung wie Talend, Pentaho oder Informatica, die den Prozess über vorkonfigurierte Komponenten abbilden und sich vor allem in betrieblichen Data-Warehouse-Umgebungen etabliert haben [1, 8]. Auf der anderen Seite werden ETL-Strecken zunehmend programmatisch umgesetzt, etwa mit der Python-Bibliothek Pandas für die Verarbeitung tabellarischer Daten oder mit verteilten Frameworks wie Apache Spark für große Datenmengen [1, 2].

Allen klassischen Ansätzen ist gemeinsam, dass die Verarbeitungslogik regelbasiert und explizit vorgegeben wird: Entwicklerinnen und Entwickler definieren die Transformations- und Bereinigungsregeln manuell, etwa in Form von Bedingungen, Zuordnungstabellen oder Skripten [8, 10]. Dieser Ansatz bietet den Vorteil hoher Nachvollziehbarkeit und deterministischer, reproduzierbarer Ergebnisse, stößt jedoch dort an Grenzen, wo Datenprobleme vielfältig, kontextabhängig oder schwer im Voraus zu formalisieren sind [2, 3]. In der vorliegenden Arbeit dient eine solche regelbasiert implementierte Pipeline als Vergleichsmaßstab, gegen den die untersuchten LLM-gestützten Verfahren gemessen werden.

2.2. Datenqualität und Datenbereinigung

2.2.1. Dimensionen der Datenqualität

Datenqualität ist ein mehrdimensionales Konstrukt, das beschreibt, inwieweit ein Datenbestand für seinen vorgesehenen Verwendungszweck geeignet ist. In der Literatur hat sich ein Kern wiederkehrender Qualitätsdimensionen herausgebildet, zu denen insbesondere Genauigkeit, Vollständigkeit, Konsistenz, Aktualität und Eindeutigkeit zählen [9, 11]. Die Genauigkeit beschreibt, inwieweit die gespeicherten Werte die reale Sachlage korrekt abbilden, während die Vollständigkeit den Grad angibt, in dem alle erforderlichen Werte tatsächlich vorhanden sind [9, 11]. Die Konsistenz bezieht sich auf die Widerspruchsfreiheit der Daten innerhalb eines Bestands und über Systemgrenzen hinweg, etwa bei einheitlichen Formaten und Bezeichnungen [9, 11]. Die Aktualität erfasst, ob die Daten den jeweils gültigen Zeitstand widerspiegeln, und die Eindeutigkeit verlangt, dass jeder reale Sachverhalt nur einmal und nicht in mehreren widersprüchlichen oder duplizierten Datensätzen abgelegt ist [9, 11].

Mit dem Aufkommen von Big Data ist dieser klassische Kanon erweitert worden, da Volumen, Geschwindigkeit und Vielfalt der Daten zusätzliche Anforderungen stellen und feinere Differenzierungen der Qualitätsdimensionen erforderlich machen [9, 11]. Für die vorliegende Arbeit sind insbesondere die Dimensionen Genauigkeit, Vollständigkeit, Konsistenz und Eindeutigkeit von Bedeutung, da sie unmittelbar mit den untersuchten ETL-Teilaufgaben der Datenbereinigung, Datentransformation und Duplikaterkennung korrespondieren.

2.2.2. Häufige Datenprobleme

Aus den Qualitätsdimensionen lassen sich die in der Praxis am häufigsten auftretenden Datenprobleme ableiten, die im Rahmen der Datenbereinigung erkannt und behoben werden müssen. Fehlende Werte gehören zu den verbreitetsten Mängeln und beeinträchtigen unmittelbar die Vollständigkeit. Sie erfordern Entscheidungen über Imputation, Entfernung oder Kennzeichnung der betroffenen Datensätze [9, 12]. Inkonsistente Formate – etwa unterschiedliche Datums-, Zahlen- oder Einheitendarstellungen sowie abweichende Schreibweisen gleicher Inhalte – beeinträchtigen die Konsistenz und müssen im Transformationsschritt vereinheitlicht werden [9, 11].

Ein weiteres zentrales Problem sind Duplikate, also mehrfach vorhandene Datensätze, die denselben realen Sachverhalt beschreiben, sich aber in Schreibweise oder Vollständigkeit unterscheiden können. Ihre Erkennung ist anspruchsvoll, da sie über den exakten Abgleich hinaus häufig eine unscharfe Ähnlichkeitsbetrachtung erfordert [12, 13]. Hinzu kommen Ausreißer und fehlerhafte Werte, die durch Eingabefehler, Messfehler oder Systembrüche entstehen und die Genauigkeit des Datenbestands gefährden [9, 12]. Die Bedeutung einer gründlichen Bereinigung ergibt sich daraus, dass mangelhafte Eingangsdaten unmittelbar auf nachgelagerte Analysen und Modelle durchschlagen, was häufig mit dem Grundsatz „garbage in, garbage out“ umschrieben wird [9, 11]. Die gezielte Konstruktion solcher Datenprobleme bildet später die Grundlage des in Kapitel 5 beschriebenen synthetischen Datensatzes.

2.3. Künstliche Intelligenz und maschinelles Lernen

Die in dieser Arbeit untersuchten Sprachmodelle sind Teil eines übergeordneten Forschungs- und Anwendungsfeldes, das dieser Abschnitt einordnet, bevor die Modelle selbst betrachtet werden. Unter dem Begriff der KI wird das Bestreben verstanden, Maschinen mit Fähigkeiten auszustatten, die üblicherweise menschliche Intelligenz voraussetzen, etwa das Erkennen von Mustern, das Ziehen von Schlüssen oder das Treffen von Entscheidungen [14, 15]. KI ist damit ein weit gefasster Oberbegriff, der von regelbasierten Expertensystemen bis zu lernenden Verfahren reicht.

Den heute dominierenden Zweig der KI bildet Maschinelles Lernen (ML). Anders als regelbasierte Systeme, deren Verhalten explizit vorgegeben wird, leiten ML-Verfahren die zugrunde liegenden Zusammenhänge selbstständig aus Beispieldaten ab und übertragen sie auf neue Fälle [7, 15]. Genau dieser Unterschied, gelernte statt fest pro-

grammierter Verarbeitungslogik, begründet den Kontrast zwischen den untersuchten LLMs und der regelbasierten Vergleichspipeline. In der Datenverarbeitung sind solche Verfahren bereits etabliert, etwa zur Bereinigung, zur Imputation fehlender Werte, zum Abgleich von Entitäten und zur Anomalieerkennung [7, 12]. Ihre leistungsfähigste Ausprägung ist das auf tiefen neuronalen Netzen beruhende *Deep Learning*, dessen Fortschritte, insbesondere die Transformer-Architektur, die Grundlage der im folgenden Abschnitt behandelten LLMs bilden [5, 16].

2.4. Large Language Models

2.4.1. Funktionsweise und Architektur

LLMs sind Sprachmodelle mit einer sehr großen Zahl an Parametern, die auf großen Mengen an Textdaten vortrainiert werden und in der Lage sind, natürliche Sprache zu verstehen und zu erzeugen [5, 16]. Technische Grundlage nahezu aller aktuellen Modelle ist die Transformer-Architektur, deren Selbstaufmerksamkeitsmechanismus es erlaubt, Abhängigkeiten zwischen weit auseinanderliegenden Textbestandteilen effizient zu erfassen [5, 16]. Eingaben werden dabei zunächst in kleinere Einheiten, sogenannte Token, zerlegt, die das Modell intern als numerische Vektoren verarbeitet. Die Ausgabe entsteht, indem das Modell schrittweise das jeweils wahrscheinlichste nachfolgende Token vorhersagt [5].

Charakteristisch für LLMs ist der zweistufige Entstehungsprozess aus einem aufwendigen Vortraining auf großen, allgemeinen Datenmengen und einer anschließenden Anpassung, etwa durch Feinabstimmung oder durch ein Training mit menschlichem Feedback, das die Modelle an die Erwartungen der Nutzenden ausrichtet [5]. Ein zentraler Befund der Forschung ist, dass mit zunehmender Modell- und Datengröße neue Fähigkeiten auftreten, die kleinere Modelle nicht aufweisen. Dazu zählt insbesondere das In-Context-Lernen, also die Fähigkeit, eine Aufgabe allein anhand von Anweisungen und Beispielen im Eingabetext zu lösen, ohne dass die Modellparameter verändert werden [5, 17]. Genau diese Eigenschaft bildet die Grundlage dafür, LLMs ohne aufgabenspezifisches Nachtraining für ETL-Teilaufgaben einzusetzen. Zugleich ist die Ausgabe der Modelle probabilistischer Natur: Da das nächste Token aus einer Wahrscheinlichkeitsverteilung gezogen wird, können identische Eingaben zu unterschiedlichen Ausgaben führen, was unmittelbare Folgen für die Reproduzierbarkeit der Ergebnisse hat [5].

2.4.2. Überblick aktueller Modelle

Seit der breiten öffentlichen Aufmerksamkeit für dialogfähige Modelle hat sich das Feld der LLMs rasant entwickelt und ausdifferenziert [5, 18]. Auf dem Markt konkurriert heute eine Reihe leistungsfähiger Modellfamilien, darunter die GPT-Modelle von OpenAI, die Claude-Modelle von Anthropic, die Gemini-Modelle von Google sowie offen verfügbare Modelle wie die Llama-Reihe [5, 18]. Diese Modelle unterscheiden sich nicht nur in ihrer Architektur und Parameterzahl, sondern auch in praxisrelevanten Eigenschaften wie der Größe des Kontextfensters, der Verarbeitungsgeschwindigkeit, den Nutzungskosten und der Verfügbarkeit als offenes oder geschlossenes Modell [5, 18].

Eine wesentliche Entwicklung der jüngeren Vergangenheit ist die Erweiterung hin zu multimodalen Modellen, die neben Text auch weitere Modalitäten wie Bilder verarbeiten können [5, 16]. Angesichts der hohen Innovationsgeschwindigkeit verschieben sich die Leistungsgrenzen zwischen den Modellen kontinuierlich, weshalb vergleichende Bewertungen anhand standardisierter Kennzahlen wie Qualität, Geschwindigkeit und Preis an Bedeutung gewonnen haben [18, 19]. Für die vorliegende Arbeit ist diese Heterogenität von unmittelbarer Relevanz, da sie die Auswahl mehrerer aktueller Modelle für den experimentellen Vergleich begründet und nahelegt, dass sich die Eignung für ETL-Aufgaben zwischen den Modellen unterscheiden kann.

2.4.3. Stärken und Limitationen bei strukturierten Daten

Im Kontext der Datenverarbeitung liegt die Stärke von LLMs in ihrer Flexibilität: Sie können semi- und unstrukturierte Inhalte interpretieren, kontextabhängige Zusammenhänge erfassen und über reine Mustererkennung hinaus auch implizites Wissen einbringen, ohne dass die zugrunde liegenden Regeln explizit vorgegeben werden müssen [5, 6]. Damit eignen sie sich potenziell für Aufgaben wie die Bereinigung uneinheitlicher Werte, die Vereinheitlichung von Formaten oder das Erkennen ähnlicher Datensätze, die mit starren Regeln nur schwer abzudecken sind [6].

Diesen Stärken stehen jedoch erhebliche Limitationen gegenüber. Eine zentrale Einschränkung ist die Neigung zu Halluzinationen, also zur Erzeugung plausibel wirkender, aber sachlich falscher oder frei erfundener Ausgaben, was bei der Verarbeitung strukturierter Daten zu schwer erkennbaren Fehlern führen kann [5, 6]. Hinzu kommt die Begrenzung durch das Kontextfenster, die der Menge an Daten, die in einem einzelnen Verarbeitungsschritt übergeben werden kann, eine technische Obergrenze setzt

[5]. Ferner fallen die Nutzungskosten und die Verarbeitungszeit ins Gewicht, die insbesondere bei großen Datenmengen zu einem begrenzenden Faktor werden [5, 18]. Schließlich erschwert die bereits beschriebene probabilistische Natur der Modelle die Reproduzierbarkeit, da wiederholte Ausführungen identischer Aufgaben voneinander abweichende Ergebnisse liefern können [5, 6]. Diese Spannung zwischen Flexibilität und Verlässlichkeit bildet einen der zentralen Ausgangspunkte und zugleich einen der Untersuchungsgegenstände der vorliegenden Arbeit.

2.5. Prompt Engineering

2.5.1. Grundlegende Prompting-Strategien

Da LLMs ihre Aufgaben primär über das In-Context-Lernen lösen, kommt der Gestaltung der Eingabe – dem sogenannten Prompt – eine entscheidende Bedeutung zu. Unter Prompt Engineering versteht man das gezielte Entwerfen und Optimieren solcher Eingaben, um das Modellverhalten zu steuern, ohne die Modellparameter selbst zu verändern [20, 21]. Die einfachste Strategie ist das Zero-Shot-Prompting, bei dem das Modell eine Aufgabe allein anhand einer Anweisung und ohne weitere Beispiele bearbeitet [17, 20]. Beim Few-Shot-Prompting werden der Anweisung hingegen einige beispielhafte Ein- und Ausgabepaare beigelegt, anhand derer das Modell das gewünschte Verhalten ableitet. Dieser Ansatz verbessert die Ergebnisqualität bei vielen Aufgaben spürbar [17, 20].

Darüber hinaus haben sich anspruchsvollere Strategien etabliert. Beim Chain-of-Thought-Prompting wird das Modell angeleitet, seine Lösung in nachvollziehbaren Zwischenschritten zu entwickeln, was sich insbesondere bei mehrstufigen oder schlussfolgernden Aufgaben als wirksam erwiesen hat [20, 21]. Eine weitere verbreitete Technik ist das Role-Prompting, bei dem dem Modell eine bestimmte Rolle oder Perspektive zugewiesen wird, um Stil und Inhalt der Ausgabe zu lenken [20, 21]. Es wird in dieser Arbeit nicht als eigener Untersuchungsfaktor geführt, da eine Rollenzuweisung, die eines erfahrenen Data Engineers, in allen eingesetzten Systemanweisungen einheitlich enthalten ist und damit für alle verglichenen Strategien gleichermaßen gilt. Sie wäre als Faktor nicht von den übrigen Strategien trennbar. Angesichts der Vielzahl mittlerweile beschriebener Techniken weisen systematische Übersichten allerdings auf eine uneinheitliche Begriffsbildung und eine teils fragmentierte Forschungslage hin, was eine sorgfältige Auswahl und Definition der eingesetzten Strategien erforderlich macht [20, 21]. Für die vorliegende Arbeit werden daher klar abgegrenzte Prompting-Strategien

definiert und systematisch gegeneinander verglichen.

2.5.2. Einflussfaktoren auf die Ergebnisqualität

Die Qualität der von einem LLM erzeugten Ausgabe hängt von einer Reihe von Einflussfaktoren ab, die über die reine Wahl der Prompting-Strategie hinausgehen. Beim Few-Shot-Prompting wirken sich Auswahl, Anzahl und Reihenfolge der mitgegebenen Beispiele unmittelbar auf das Ergebnis aus, sodass schon kleine Änderungen an den Beispielen die Modellantwort verschieben können [17, 21]. Auch die genaue Formulierung und Strukturierung der Anweisung beeinflusst die Ergebnisqualität erheblich, weshalb das Auffinden geeigneter Prompts häufig einen iterativen Prozess darstellt [20, 22].

Ein technisch bedeutsamer Parameter ist die Temperatur, die steuert, wie stark die Ausgabe von der wahrscheinlichsten Vorhersage abweicht: Niedrige Werte führen zu deterministischeren und stabileren, hohe Werte zu vielfältigeren, aber weniger vorher-sagbaren Antworten, was die Reproduzierbarkeit der Ergebnisse unmittelbar berührt [5, 21]. Zudem skaliert die Leistungsfähigkeit beim Prompting mit der Größe und Leistungsfähigkeit des zugrunde liegenden Modells, sodass dieselbe Strategie je nach Modell unterschiedlich gut wirkt [17, 21]. Schließlich ist mit längeren und beispielreicheren Prompts ein höherer Verbrauch an Token und damit ein höherer Aufwand verbunden, sodass zwischen Ergebnisqualität und Effizienz abzuwägen ist [21, 22]. Diese Faktoren begründen, warum in der vorliegenden Arbeit sowohl unterschiedliche Modelle als auch unterschiedliche Prompting-Strategien unter kontrollierten Bedingungen verglichen und ihre Ergebnisse mehrfach erhoben werden.

3. Stand der Forschung

Aufbauend auf den theoretischen Grundlagen ordnet dieses Kapitel die vorliegende Arbeit in den aktuellen Forschungsstand ein. Zunächst wird der breitere Einsatz von Künstlicher Intelligenz in der Datenverarbeitung betrachtet (Abschnitt 3.1). Anschließend wird die Forschung zur Verarbeitung strukturierter und tabellarischer Daten durch LLMs sowie zur Erzeugung synthetischer Datensätze aufgearbeitet, die die Wahl der Datengrundlage dieser Arbeit begründet (Abschnitt 3.2), bevor verwandte Arbeiten zur Automatisierung von ETL-Prozessen diskutiert werden (Abschnitt 3.3). Auf dieser Grundlage wird abschließend die Forschungslücke präzisiert, an der die vorliegende

Arbeit ansetzt (Abschnitt 3.4).

3.1. KI-Einsatz in der Datenverarbeitung

Der Einsatz von Künstlicher Intelligenz und maschinellem Lernen hat die Datenverarbeitung in den vergangenen Jahren grundlegend verändert. Schon vor dem Aufkommen großer Sprachmodelle wurden Verfahren des maschinellen Lernens eingesetzt, um Schritte der Datenaufbereitung, der Mustererkennung und der Entscheidungsunterstützung zu automatisieren und so aus großen Datenbeständen Erkenntnisse zu gewinnen [14, 15]. Insbesondere im Bereich der Datenaufbereitung haben sich lernende Verfahren als nützlich erwiesen, etwa bei der Datenbereinigung, der Imputation fehlender Werte, dem Abgleich von Entitäten und der Erkennung von Ausreißern [7, 12]. Eine systematische Übersicht zeigt dabei ein wechselseitiges Verhältnis: Maschinelles Lernen wird einerseits zur Datenbereinigung genutzt, andererseits ist eine hohe Datenqualität Voraussetzung für die Leistungsfähigkeit der Modelle selbst [7, 12].

Mit der Verfügbarkeit dialogfähiger Sprachmodelle hat sich dieser Trend noch einmal verstärkt. Frühe Arbeiten beschreiben, wie Modelle wie ChatGPT Datenfachleute bei zahlreichen Teilschritten ihres Arbeitsablaufs unterstützen können, von der Datenbereinigung und Vorverarbeitung über die Modellbildung bis hin zur Interpretation der Ergebnisse [6, 15]. Gleichzeitig betonen diese Beiträge, dass der Einsatz solcher Modelle mit Herausforderungen wie Verzerrungen, mangelnder Nachvollziehbarkeit und Fragen der Verlässlichkeit verbunden ist und daher einer kritischen Begleitung bedarf [6, 14]. Insgesamt zeichnet die Literatur das Bild einer zunehmend KI-gestützten Datenverarbeitung, in der die Automatisierung von Routineaufgaben im Vordergrund steht, eine systematische empirische Bewertung der erzielten Qualität jedoch häufig noch aussteht [7, 12].

3.2. LLMs für strukturierte Daten und synthetische Datensätze

Während LLMs ursprünglich für die Verarbeitung natürlicher Sprache entwickelt wurden, hat sich die Forschung zunehmend ihrer Anwendung auf strukturierte und tabellarische Daten zugewandt. Hierbei zeigt sich, dass die Modelle prinzipiell in der Lage sind, tabellarische Inhalte zu interpretieren, Zusammenhänge zwischen Feldern zu erfassen und auf dieser Basis Transformations- und Bereinigungsaufgaben zu unterstüt-

zen [6, 23]. Ein wichtiger Anwendungsstrang ist die semantische Interpretation komplexer Datenbeziehungen, bei der LLMs über den rein syntaktischen Abgleich hinaus den inhaltlichen Bezug zwischen Datensätzen erschließen und so etwa beim Abbilden unterschiedlicher Schemata helfen können [7, 23].

Ein zweiter, intensiv untersuchter Bereich ist die Erzeugung synthetischer Daten durch LLMs. Mehrere Übersichtsarbeiten dokumentieren, dass sich große Sprachmodelle eignen, um realistische tabellarische und textuelle Datensätze zu generieren, die etwa zur Datenanreicherung oder zum Ausgleich seltener Fälle dienen [24, 25]. Zugleich verweisen empirische Studien auf deutliche Grenzen: Die Eignung der erzeugten Daten variiert stark je nach Aufgabe, und insbesondere bei subjektiven oder komplexen Aufgabenstellungen bleibt die Qualität der synthetischen Daten hinter realen Daten zurück [24, 26]. Für tabellarische Daten im Speziellen weist die Literatur darauf hin, dass dieser Datentyp trotz seiner praktischen Bedeutung lange vernachlässigt wurde und eigene methodische Herausforderungen mit sich bringt [27, 28]. Diese Befunde sind für die vorliegende Arbeit in zweifacher Hinsicht relevant: Sie begründen einerseits die Verwendung eines synthetischen Datensatzes und mahnen andererseits zur empirischen Prüfung der tatsächlichen Verarbeitungsqualität, statt sich auf die grundsätzliche Fähigkeit der Modelle zu verlassen [26, 28]. Da eine solche empirische Prüfung im produktiven Einsatz jedoch nicht bei jeder Ausführung wiederholt werden kann, rückt zugleich die Stabilität der Ergebnisse über wiederholte Ausführungen in den Blick.

3.3. Verwandte Arbeiten zur Automatisierung von ETL

Die Automatisierung von ETL-Prozessen mit Methoden der Künstlichen Intelligenz ist Gegenstand einer wachsenden Zahl von Arbeiten. Ein erster, schon länger verfolgter Ansatz nutzt klassische Verfahren des maschinellen Lernens, um einzelne Schritte der Pipeline zu automatisieren, etwa das Abbilden von Schemata, die Erkennung von Anomalien und die Ableitung von Transformationsregeln aus den Daten [10, 29]. Darauf aufbauend beschreibt eine Reihe neuerer Beiträge umfassendere Architekturen, in denen KI-Komponenten auf allen Stufen des ETL-Prozesses eingebettet werden, um Aufgaben wie automatisierte Schemaerkennung, intelligente Qualitätssicherung und Leistungsoptimierung zu übernehmen [2, 30].

Ein wiederkehrendes Leitbild dieser Arbeiten ist die Vorstellung selbstheilender und adaptiver Datenpipelines, die Probleme proaktiv erkennen, eigenständig beheben und sich an veränderte Datenstrukturen anpassen, statt rein reaktiv auf Fehler zu reagie-

ren [29, 31]. In diesem Zusammenhang werden zunehmend autonome KI-Agenten diskutiert, die Datenaufnahme, Bereinigung und Integration weitgehend selbstständig ausführen sollen [31, 32]. Mehrere Beiträge widmen sich dabei spezifischen Teilaufgaben, etwa der intelligenten Anomalieerkennung in ETL-Strecken, der automatisierten Qualitätssicherung oder der automatisierten Erzeugung von Merkmalen [13, 33]. Insbesondere wird betont, dass nicht ein vollständiger Ersatz, sondern eine hybride Verknüpfung von regelbasierten Systemen und KI-Modellen erstrebenswert sei, um Nachvollziehbarkeit und Kontrolle bei gleichzeitig höherer Flexibilität zu wahren [2, 30].

Arbeiten, die explizit generative Sprachmodelle für die ETL-Automatisierung einsetzen, sind hingegen noch vergleichsweise jung. Eine Untersuchung zur generativen KI in der ETL-Automatisierung zeigt, dass sich mit Hilfe von Einbettungs- und generativen Verfahren aus natürlichsprachlichen Anfragen passende ETL-Lösungen ableiten lassen, der manuelle Aufwand dadurch deutlich reduziert, jedoch nicht vollständig beseitigt wird [3, 34]. Insgesamt überwiegen in diesem Feld konzeptionelle und architektonische Beiträge, die das Potenzial KI-gestützter ETL-Prozesse beschreiben, während belastbare, vergleichende Evaluierungen einzelner Modelle und Verarbeitungsstrategien noch selten sind [35, 36].

3.4. Forschungslücke und Abgrenzung

Aus der Aufarbeitung des Forschungsstands lassen sich mehrere Lücken ableiten, an denen die vorliegende Arbeit ansetzt. Erstens beschreiben viele Beiträge das Potenzial KI-gestützter ETL-Prozesse auf konzeptioneller oder architektonischer Ebene, ohne dieses Potenzial unter kontrollierten Bedingungen empirisch zu überprüfen [7, 35]. Zweitens stützt sich ein Großteil der bestehenden Arbeiten auf klassische Verfahren des maschinellen Lernens, während der gezielte Einsatz aktueller LLMs für typische ETL-Teilaufgaben und insbesondere dessen systematischer Vergleich mit etablierten Verfahren bislang nur unzureichend untersucht ist [2, 34]. Drittens fehlt es an Untersuchungen, die mehrere aktuelle Modelle und unterschiedliche Prompting-Strategien gemeinsam und unter einheitlichen Bedingungen gegen eine klassische, regelbasierte Pipeline stellen, obwohl die Literatur sowohl die Heterogenität der Modelle als auch den starken Einfluss der Prompt-Gestaltung deutlich herausstellt [18, 21]. Viertens wird die Reproduzierbarkeit der Ergebnisse, die sich unmittelbar aus der probabilistischen Natur der Modelle ergibt, in den vorhandenen Arbeiten kaum systematisch erhoben, obwohl sie für den praktischen Einsatz von zentraler Bedeutung ist [5, 6]. Sie wird in dieser Arbeit bewusst nicht als eigene Forschungsfrage geführt, da sie keine für sich

stehende Eigenschaft ist, sondern die Belastbarkeit jeder anderen Aussage bedingt: Ein Eignungsurteil über ein Verfahren und ein Urteil über eine Prompting-Strategie sind nur so tragfähig wie die Wiederholbarkeit der zugrunde liegenden Messung. Sie wird deshalb als querschnittliche Dimension über beide Forschungsfragen erhoben.

An genau diesen Lücken setzt die vorliegende Arbeit an. Sie verbindet die genannten Aspekte zu einer experimentellen Evaluierung, die mehrere aktuelle LLMs und unterschiedliche Prompting-Strategien anhand eines kontrolliert erstellten synthetischen Datensatzes für die Teilaufgaben Datenbereinigung, Datentransformation und Duplikaterkennung untersucht und die Ergebnisse mit einer klassisch implementierten ETL-Pipeline vergleicht. Durch die mehrfache Ausführung identischer Aufgaben wird die Reproduzierbarkeit als querschnittliche Dimension einbezogen. Damit grenzt sich die Arbeit von rein konzeptionellen Beiträgen ab und liefert eine empirisch fundierte Grundlage, um die in Kapitel 1 formulierten Forschungsfragen zu beantworten und Handlungsempfehlungen für den praktischen Einsatz abzuleiten [2, 34].

Die Zuordnung zu den in Abschnitt 1.2 formulierten Forschungsfragen ist dabei die folgende: Die erste und die zweite Lücke – das Ausbleiben empirischer Prüfung konzeptioneller Entwürfe und die Fokussierung bestehender Arbeiten auf klassische Verfahren des maschinellen Lernens – werden gemeinsam von FF1 aufgenommen, die den Vergleich aktueller LLMs mit dem regelbasierten Vorgehen empirisch führt und dabei neben dem Aufgabentyp auch die Ausführungsform einbezieht. Die dritte Lücke ist zweigeteilt: Ihr Teil zum Vergleich mehrerer aktueller Modelle fällt ebenfalls an FF1, ihr Teil zur Prompt-Gestaltung an FF2. Die vierte Lücke, die kaum untersuchte Reproduzierbarkeit, wird aus den in Abschnitt 4.1 genannten Gründen nicht in eine eigene Forschungsfrage überführt, sondern als querschnittliche Dimension zu beiden Fragen erhoben.

4. Konzeption der experimentellen Evaluierung

Aufbauend auf den theoretischen Grundlagen aus Kapitel 2 und der in Kapitel 3 präzisierten Forschungslücke beschreibt dieses Kapitel die Konzeption der experimentellen Evaluierung. Es überführt die in Abschnitt 1.2 formulierten Forschungsfragen in ein konkretes Untersuchungsdesign und legt damit die methodische Grundlage für die fol-

gende Umsetzung. Zunächst werden das übergeordnete Forschungsdesign und die daraus abgeleiteten Hypothesen dargelegt (Abschnitt 4.1). Anschließend werden die Auswahl der untersuchten LLMs (Abschnitt 4.2), die Definition der betrachteten ETL-Aufgaben (Abschnitt 4.3) sowie die verglichenen Prompting-Strategien (Abschnitt 4.4) beschrieben. Darauf folgt die Operationalisierung der Evaluationskriterien (Abschnitt 4.5), bevor abschließend die klassische ETL-Pipeline als Vergleichsbasis definiert wird (Abschnitt 4.6). Die konkrete technische Umsetzung des hier entworfenen Designs ist Gegenstand der Kapitel 5 bis 7.

4.1. Forschungsdesign und Hypothesen

Zur Beantwortung der Forschungsfragen wird ein experimentelles Untersuchungsdesign gewählt, das den Einsatz von LLMs für ausgewählte ETL-Teilaufgaben unter kontrollierten Bedingungen mit einer klassischen, regelbasierten Pipeline vergleicht. Ein experimenteller Zugang ist hier angemessen, da die Forschungslücke gerade im Fehlen belastbarer, vergleichender Evaluierungen unter einheitlichen Bedingungen besteht, während das grundsätzliche Potenzial KI-gestützter ETL-Prozesse in der Literatur bereits vielfach konzeptionell beschrieben wurde [2, 35]. Den Kern des Designs bildet ein vollständig gekreuztes Versuchsschema über die drei Faktoren Modell, Prompting-Strategie und ETL-Aufgabe: Für jede Kombination dieser Faktoren wird die Verarbeitung durchgeführt und anhand einheitlicher, in Abschnitt 4.5 definierter Kriterien bewertet, sodass sich die Einflüsse der einzelnen Faktoren systematisch isolieren lassen [5, 8].

Eine zentrale Gestaltungsentscheidung betrifft die Einbindung der Modelle in den Verarbeitungsprozess. Als primärer Ausführungsmodus werden sie angewiesen, einen ETL-Verarbeitungsschritt als Python-Code zu erzeugen, der anschließend deterministisch auf den Daten ausgeführt und dessen Ergebnis gegen eine bekannte Soll-Lösung (Ground Truth) geprüft wird. Diese Code-Generierung entkoppelt die Verarbeitung von der Datenmenge und damit von der Begrenzung durch das Kontextfenster, erzeugt ein nachvollziehbares und wiederausführbares Artefakt und stellt die unmittelbare Vergleichbarkeit mit der regelbasierten Pipeline her, die ebenfalls programmatisch auf Basis der Bibliothek Pandas umgesetzt wird [1, 3].

Ergänzend wird ein zweiter Ausführungsmodus untersucht, die direkte In-Context-Verarbeitung, bei der dem Modell die Eingabedaten im Prompt bereitgestellt werden und es die Ergebnistabelle ohne den Umweg über generierten Code zurückgibt. Dieser Modus zielt

auf die semantisch geprägten Aufgaben, bei denen sich das erforderliche Wissen nicht vollständig in feste Regeln überführen lässt, etwa bei uneinheitlich geschriebenen Länderangaben oder unscharfen Duplikaten: Das Modell wendet sein Weltwissen unmittelbar auf jeden einzelnen Wert an, statt eine verallgemeinernde Vorschrift zu formulieren [6, 23]. Da die Daten dabei vollständig im Prompt mitgeführt werden, begrenzt das Kontext- und Ausgabefenster diesen Modus auf überschaubare Datenmengen. Er wird deshalb nicht als vollständig gekreuzter Faktor, sondern als ergänzender Vergleich auf denselben Aufgaben und Modellen geführt [5, 22].

Die bis hierhin beschriebene Faktormatrix setzt für jede Aufgabe unmittelbar auf den unveränderten Rohdaten auf. Eine reale ETL-Strecke besteht jedoch aus einer Kette, in der jeder Schritt auf dem Ergebnis seines Vorgängers aufsetzt [1, 2]. Das Design wird daher um eine verkettete Ausführung ergänzt, in der die Ausgabe jedes Schritts die Eingabe des nächsten bildet und ein Abbruch die abhängigen Schritte blockiert, sodass sichtbar wird, wie sich lokal geringe Abweichungen über die Kette hinweg auswirken. Damit ein so gemessener Verlust nicht fälschlich dem betreffenden Schritt zugeschrieben wird, obwohl er aus fehlerhaften Eingabedaten resultiert, tritt ein zweiter, sonst identischer Durchlauf hinzu, in dem jeder Schritt statt der Ausgabe seines Vorgängers die entsprechende Soll-Zwischentabelle erhält. Die Differenz beider Durchläufe beziffert unmittelbar den Anteil des Ergebnisverlusts, der auf vorgelagerte Schritte zurückgeht, eine für FF1 wesentliche Trennung, da eine Empfehlung je Aufgabentyp voraussetzt, dass ein Fehler dem verursachenden Schritt und nicht seinem Nachfolger zugerechnet wird [3, 8].

Da LLMs auch bei identischer Eingabe voneinander abweichende Ausgaben erzeugen können, wird die Reproduzierbarkeit als querschnittliche Dimension in das Design aufgenommen: Jede Konfiguration wird mehrfach ausgeführt, sodass sich die Streuung über die Wiederholungen quantifizieren lässt [5, 6]. Die Wiederholung bezieht sich dabei auf den Generierungsschritt. Das Modell wird jedes Mal erneut zur Code-Erzeugung aufgefordert, während die Ausführung eines einzelnen erzeugten Skripts deterministisch bleibt. Steuerbare Einflussgrößen wie die in Abschnitt 2.5.2 eingeführte Temperatur bleiben über alle Durchläufe konstant und niedrig, um die nicht modellbedingte Variabilität zu minimieren [5, 21]. Als unveränderliche Grundlage aller Experimente dient der in Kapitel 5 beschriebene synthetische Datensatz, dessen Eigenschaften damit exakt bekannt sind.

Aus den Forschungsfragen und dem Forschungsstand lassen sich die folgenden Hy-

pothesen ableiten, die im Rahmen der Auswertung in Kapitel 8 geprüft werden. Zu FF1 werden dabei zwei Hypothesen formuliert, die ihre beiden Teilfragen, den Aufgabentyp und die Ausführungsform, getrennt adressieren, zu FF2 eine. Jede bleibt damit unmittelbar auf eine der beiden Fragen bezogen und bereitet deren Beantwortung vor:

H1 (zu FF1): Die Eignung von LLMs hängt stark vom Aufgabentyp ab. Bei klar formalisierbaren Aufgaben erreichen sie bestenfalls eine mit der regelbasierten Pipeline vergleichbare Ergebnisqualität, während sie bei Aufgaben mit hoher semantischer Anforderung Vorteile zeigen und mit zunehmender struktureller Komplexität an Zuverlässigkeit verlieren. Ein über alle Aufgaben hinweg überlegener Ansatz existiert daher nicht [6, 12].

H1a (zu Ausführungsform und Aufgabenzuschnitt): Die Eignung wird nicht allein vom Aufgabentyp bestimmt, sondern zusätzlich von der Ausführungsform. Erwartet wird, dass der Ausführungsmodus darüber entscheidet, welche Fähigkeit eines Modells überhaupt zur Wirkung kommt – Einzelfallverständnis in der Direktverarbeitung, verlässliche Wiederholbarkeit in der Code-Generierung –, und dass der Zuschnitt der Aufgabe die Ergebnisqualität stärker beeinflusst als die Wahl des Modells, da mit der Zahl der in einem Arbeitsschritt gleichzeitig zu treffenden Entscheidungen die Fehleranfälligkeit überproportional wächst [1, 23].

H2 (zu FF2): Aufwändigere Prompting-Strategien, die zusätzlichen Kontext, Beispiele oder Zwischenschritte bereitstellen, verbessern die Ergebnisqualität gegenüber einem einfachen Zero-Shot-Prompt, wobei der Nutzenzuwachs mit steigendem Aufwand abnimmt und der zusätzliche Kontext bei zu enger Führung auch nachteilig wirken kann [20, 21].

Tabelle 4.1 fasst die daraus entstehenden Untersuchungsebenen zusammen. Sie unterscheiden sich darin, wie eine Aufgabe gestellt wird – unabhängig auf den Rohdaten, als Glied einer Kette oder als Gruppe zusammengefasster Schritte – und welche Eingabe sie dabei erhält. Erst ihr Zusammenspiel erlaubt die Zuordnung eines beobachteten Fehlers zu seiner Ursache: Die Faktormatrix misst die Leistung bei einer isoliert gestellten Aufgabe, die verkettete Ausführung die Leistung im Realbetrieb, und die Differenz zum isolierten Durchlauf beziffert den Anteil, der aus vorgelagerten Schritten stammt.

Die Reproduzierbarkeit wird, wie in Abschnitt 1.2 festgelegt, nicht als eigenständige Hy-

Tabelle 4.1.: Die Ebenen des Untersuchungsdesigns mit ihrer jeweiligen Fragestellung und Eingabeform. Die konkreten Laufzahlen je Ebene nennt Kapitel 7.

Ebene	Eingabe je Aufgabe	Beantwortet
Faktormatrix	Rohdaten, kein Vorgängerergebnis	Einfluss von Modell, Strategie und Aufgabentyp
Direkt-Modus	Rohdaten, im Prompt eingebettet	Einfluss des Ausführungsmodus
Verkettete Pipeline	Ausgabe des Vorgängerschritts	Leistung im Realbetrieb einer Strecke
Isolierter Durchlauf	Soll-Ausgabe des Vorgängers	Trennung eigener Leistung von geerbtem Fehler
Gruppierte Strecke	Soll-Ausgabe der Vorgängerguppe	Einfluss des Aufgabenschnitts
Explizit gegliederte Aufgabe	Rohdaten, benannte Teilschritte	Einfluss der Ausdrücklichkeit der Anforderung
Regelbasierte Pipeline	wie die jeweilige Ebene	Vergleichsmaßstab für alle Ebenen

pothese geführt, sondern als querschnittliche Dimension zu beiden: Für H1 ist die Stabilität Bestandteil der Eignungsbeurteilung, da ein im Mittel gutes, aber stark schwankendes Ergebnis produktiv einen anderen Wert besitzt als ein gleich gutes, verlässlich wiederholbares. Für H2 ist sie zusätzliches Unterscheidungsmerkmal zwischen den Strategien. Erwartet wird, dass identische Konfigurationen trotz niedriger Temperatur messbar voneinander abweichen und die Stabilität mit zunehmender Aufgabenkomplexität abnimmt [5, 6].

4.2. Auswahl der Large Language Models

Die Auswahl der untersuchten Modelle folgt dem Ziel, ein für den praktischen Einsatz repräsentatives und zugleich aussagekräftiges Spektrum aktueller LLMs abzudecken. Wie in Abschnitt 2.4.2 dargestellt, unterscheiden sich die verfügbaren Modellfamilien nicht nur in Architektur und Parameterzahl, sondern auch in praxisrelevanten Eigenschaften wie Kontextfenster, Verarbeitungsgeschwindigkeit, Nutzungskosten und Verfügbarkeit als offenes oder geschlossenes Modell [5, 18]. Gerade diese Heterogenität begründet den Vergleich mehrerer Modelle, da sich die Eignung für ETL-Aufgaben zwischen ihnen unterscheiden kann und ein Einzelmodell keine verallgemeinerbaren Aussagen zuließe [18, 19].

Als Auswahlkriterien werden die Aktualität und Verbreitung der Modelle, die Abdeckung unterschiedlicher Anbieter sowie die Berücksichtigung sowohl geschlossener als auch offen verfügbarer Modelle herangezogen. Auf dieser Grundlage werden je ein Vertreter der etablierten geschlossenen Modellfamilien von OpenAI, Anthropic und Google sowie ein offen verfügbares, lokal ausführbares Modell aus der Llama-Reihe untersucht. Damit deckt die Auswahl unterschiedliche Kosten-, Leistungs- und Betriebsmodelle ab und erlaubt insbesondere den praktisch bedeutsamen Vergleich zwischen kommerziell betriebenen und lokal betreibbaren Modellen [5, 18]. Tabelle 4.2 weist die konkret eingesetzten Modelle mit ihrer Versionskennung und den zugrunde gelegten Preisen aus.

Tabelle 4.2.: Die vier untersuchten Modelle mit Versionskennung und den für die Kostenschätzung zugrunde gelegten Listenpreisen in US-Dollar je einer Million Token (Stand: 27. Juli 2026).

Anbieter	Versionskennung	Betrieb	Eingabe	Ausgabe
Anthropic	claude-sonnet-5	Cloud	3,00	15,00
OpenAI	gpt-5.6-terra	Cloud	2,50	15,00
Google	gemini-3.6-flash	Cloud	1,50	7,50
Meta	llama3.1 (8 B, über Ollama)	lokal	keine Nutzungskosten	

Bei den drei kommerziellen Modellen handelt es sich jeweils um das mittlere Leistungsniveau der aktuellen Generation des Anbieters, nicht um dessen Spitzenmodell. Diese Wahl ist bewusst getroffen: Sie bildet ab, was in einem Vorhaben mit begrenztem Budget realistisch eingesetzt würde, und vermeidet zugleich, dass Preisunterschiede zwischen den Anbietern das Ergebnis dominieren. Alle drei führen intern einen Denkprozess aus, dessen Token zum Ausgabetarif abgerechnet werden und in den ausgewiesenen Kosten enthalten sind. Die angegebenen Preise sind die zum genannten Stichtag gültigen Listenpreise. Zeitlich befristete Einführungspreise wurden bewusst nicht verwendet, da sie die Kostenangaben nicht dauerhaft nachvollziehbar hielten.

Um die Modelle technisch austauschbar ansprechen zu können, werden sie hinter einer einheitlichen Schnittstelle gekapselt, sodass der Versuchsablauf unabhängig vom konkreten Anbieter erfolgt. Die Details dieser Abstraktion werden in Kapitel 6 beschrieben. Da sich das Feld der Modelle mit hoher Geschwindigkeit weiterentwickelt, wird die exakte Versionskennung jedes Modells bei jeder Ausführung protokolliert, um die spätere Nachvollziehbarkeit und Reproduzierbarkeit der Ergebnisse sicherzustellen [5, 18].

4.3. Definition der ETL-Aufgaben

Die untersuchten ETL-Aufgaben orientieren sich an den in Abschnitt 2.2.2 beschriebenen, in der Praxis am häufigsten auftretenden Datenproblemen und decken die drei in dieser Arbeit fokussierten Teilbereiche der Transformationsphase ab: die Datenbereinigung, die Datentransformation und die Duplikaterkennung. Um nicht nur die grundsätzliche Eignung, sondern auch deren Grenzen in Abhängigkeit von der Aufgabenkomplexität zu erfassen, wird jeder Teilbereich in drei aufeinander aufbauenden Schwierigkeitsstufen operationalisiert. Diese reichen von einer einfachen, klar formalisierbaren Aufgabe über eine mittlere Stufe bis hin zu einer anspruchsvollen Variante, die semantisches Verständnis oder mehrstufige Verarbeitung erfordert. Daraus ergibt sich eine Matrix aus neun Aufgaben, die zugleich die Schwierigkeitsachse als systematische Vergleichsdimension einführt und so eine differenzierte Beurteilung der Modelle erlaubt [11, 12]. Alle Aufgaben beziehen sich auf den synthetischen Datensatz aus Kapitel 5, der ein kleines E-Commerce-Szenario mit den Tabellen Kunden, Produkte und Bestellungen abbildet, und sind so gestaltet, dass für jede von ihnen eine eindeutige Soll-Lösung existiert.

4.3.1. Datenbereinigung

Die Aufgaben zur Datenbereinigung adressieren die Beseitigung von Mängeln, die unmittelbar die Genauigkeit, Vollständigkeit und Konsistenz des Datenbestands beeinträchtigen [9, 12]. Die einfache Stufe umfasst grundlegende Bereinigungsoperationen wie das Entfernen überflüssiger Leerzeichen in Textspalten und das einheitliche Kennzeichnen fehlender Werte. Diese Aufgabe ist mit wenigen Regeln vollständig beschreibbar und dient als Referenzpunkt für die grundsätzliche Verarbeitungsfähigkeit der Modelle. Die mittlere Stufe verlangt die Vereinheitlichung uneinheitlich formatierter Datumsangaben, die in mehreren unterschiedlichen Darstellungen vorliegen, auf ein einheitliches Zielformat und prüft damit die Fähigkeit zur formatübergreifenden Normalisierung. Die anspruchsvolle Stufe schließlich erfordert die semantische Vereinheitlichung von Länderangaben, die in verschiedenen Sprachen, Abkürzungen und Schreibweisen einschließlich Tippfehlern vorliegen, auf einen standardisierten Code. Gerade diese Aufgabe stellt eine für LLMs zentrale Achse dar, da sie über den rein syntaktischen Abgleich hinausgeht und ein inhaltliches Verständnis der Werte voraussetzt, das mit starren Regeln nur schwer abzubilden ist [6, 23].

4.3.2. Datentransformation

Die Aufgaben zur Datentransformation betreffen die Überführung der Daten in die geforderte Struktur und Typisierung sowie die Ableitung neuer Informationen aus vorhandenen Werten. Die einfache Stufe umfasst Typkonvertierungen, etwa die Umwandlung uneinheitlich formatierter Preisangaben in einen numerischen Wert und textueller Wahrheitswerte in einen Boolean-Datentyp, und prüft damit die korrekte Behandlung von Datentypen. Die mittlere Stufe verlangt die Berechnung abgeleiteter Spalten aus bestehenden Feldern, beispielsweise die Ermittlung eines Gesamtbetrags sowie die Extraktion einzelner Datumsbestandteile, und adressiert damit zeilenweise Berechnungslogik. Die anspruchsvolle Stufe bildet die komplexeste Aufgabe der gesamten Untersuchung und verbindet mehrere Verarbeitungsschritte: Sie erfordert die Verknüpfung der drei Tabellen über ihre Schlüsselbeziehungen, eine begleitende Bereinigung der beteiligten Felder sowie eine anschließende Gruppierung und Aggregation der Ergebnisse. Damit prüft sie, ob die Modelle eine mehrstufige, aus Join, Bereinigung und Aggregation zusammengesetzte Verarbeitungslogik korrekt umsetzen können, wie sie für reale ETL-Strecken charakteristisch ist [1, 2].

4.3.3. Duplikaterkennung

Die Aufgaben zur Duplikaterkennung zielen auf die Eindeutigkeit des Datenbestands und damit auf die Identifikation und Auflösung mehrfach vorhandener Datensätze [12, 13]. Die einfache Stufe beschränkt sich auf exakte Duplikate, bei denen vollständig identische Datensätze zu entfernen sind, und ist mit einem deterministischen Abgleich vollständig lösbar. Die mittlere Stufe verlangt die schlüsselbasierte Entdoppelung anhand eines eindeutigen Merkmals, wobei bei mehreren übereinstimmenden Datensätzen anhand einer definierten Regel der zu behaltende Datensatz ausgewählt werden muss. Die anspruchsvolle Stufe adressiert schließlich unscharfe Duplikate (Fuzzy-Duplikate), bei denen derselbe reale Sachverhalt in abweichenden Schreibweisen vorliegt, etwa durch Tippfehler, unterschiedliche Groß- und Kleinschreibung oder geringfügig abweichende Formatierungen. Da hier ein exakter Abgleich nicht ausreicht und stattdessen eine Ähnlichkeitsbetrachtung erforderlich ist, bildet diese Aufgabe die zweite zentrale Achse, an der sich die semantischen Fähigkeiten der Modelle gegenüber regelbasierten Verfahren zeigen [6, 12].

4.4. Prompting-Strategien

Da LLMs ihre Aufgaben primär über das In-Context-Lernen lösen, kommt der Gestaltung der Eingabe eine entscheidende Bedeutung für die Ergebnisqualität zu, weshalb die Prompting-Strategie als eigenständiger Untersuchungsfaktor in das Design aufgenommen wird [20, 21]. Angesichts der in Abschnitt 2.5.1 angesprochenen uneinheitlichen Begriffsbildung im Feld werden für diese Arbeit klar abgegrenzte Strategien definiert, die sich in ihrem Aufwand und dem Umfang der bereitgestellten Hilfestellung systematisch unterscheiden und gegeneinander verglichen werden.

Den Ausgangspunkt bildet das Zero-Shot-Prompting, bei dem das Modell die Aufgabe allein anhand einer präzisen Anweisung und ohne weitere Beispiele bearbeitet. Diese Strategie dient als Referenzbasis, die misst, was ein Modell ohne zusätzliche Hilfestellung aus einer reinen Aufgabenbeschreibung erzeugt [17, 20]. Darauf aufbauend wird das Few-Shot-Prompting untersucht, das der Anweisung einige beispielhafte Ein- und Ausgabepaare beifügt, anhand derer das Modell das gewünschte Verhalten ableitet und das die Ergebnisqualität bei vielen Aufgaben spürbar verbessert [17, 21]. Als dritte Strategie wird das Chain-of-Thought-Prompting einbezogen, bei dem das Modell zur Entwicklung seiner Lösung in nachvollziehbaren Zwischenschritten angeleitet wird, was sich insbesondere bei den mehrstufigen Aufgaben dieser Untersuchung als wirksam erweisen könnte [20, 21]. Die Untersuchung beschränkt sich bewusst auf diese drei etablierten Grundformen, da sie die in der Literatur übliche Steigerung von reiner Anweisung über beispielgestützte Anleitung bis zur angeleiteten Zwischenschrittbildung vollständig abbilden und sich damit als abgestufte Reihe interpretieren lassen. Um faire Vergleichsbedingungen zu gewährleisten, werden die Strategien für alle Aufgaben nach einem einheitlichen Aufbau formuliert und versioniert vorgehalten, sodass jede eingesetzte Prompt-Variante eindeutig dokumentiert und wiederverwendbar bleibt [21, 22].

4.5. Evaluationskriterien

Damit die Ergebnisse aller Verfahren über Modelle, Prompting-Strategien und Aufgaben hinweg vergleichbar gemessen werden können, werden die Bewertungskriterien aus den in Abschnitt 2.2.1 eingeführten Dimensionen der Datenqualität abgeleitet und operationalisiert [8, 11]. Grundlage jeder Bewertung ist der Abgleich der erzeugten Ausgabedatei mit der für jede Aufgabe bekannten Soll-Lösung. Die folgenden Unterabschnitte beschreiben die vier herangezogenen Kriterien Genauigkeit, Vollständigkeit,

Konsistenz und Effizienz. Die Stabilität der Ergebnisse, als querschnittliche Reproduzierbarkeitsdimension über beide Forschungsfragen hinweg, wird nicht als separates Einzelkriterium erhoben, sondern daraus abgeleitet, wie stark die genannten Qualitätskennzahlen über die wiederholten Ausführungen einer Konfiguration streuen: Eine geringe Streuung entspricht dabei einer hohen, eine große Streuung einer geringen Stabilität [5, 6].

4.5.1. Genauigkeit

Die Genauigkeit erfasst, inwieweit die erzeugten Werte mit der Soll-Lösung übereinstimmen, und bildet das zentrale Korrektheitskriterium [9, 11]. Sie wird als Anteil der korrekt verarbeiteten Werte am Gesamtbestand operationalisiert, sodass auch teilweise korrekte Ergebnisse differenziert bewertet werden können und nicht jede Abweichung gleichermaßen ins Gewicht fällt. Da die Ergebnistabellen programmatisch erzeugt werden und korrekte Werte häufig in abweichender Zeilenreihenfolge liefern, werden beide Tabellen vor dem Abgleich einheitlich sortiert und damit reihenfolgeunabhängig verglichen.

Zu klären ist, wie mit Ergebnissen zu verfahren ist, deren Datensatzanzahl von der Soll-Lösung abweicht. Der naheliegende Weg besteht darin, den zellweisen Abgleich in diesem Fall zu verweigern, da sich die Zeilen nicht mehr eindeutig einander zuordnen lassen. Dieses strenge Vorgehen hat jedoch eine unerwünschte Eigenschaft: Es bewertet ein Ergebnis, das von fünfhundert erwarteten Datensätzen 514 liefert und in allen übrigen Werten korrekt ist, genauso wie ein vollständig falsches. Gerade bei der Duplikaterkennung, bei der die Datensatzanzahl das eigentliche Ergebnis der Aufgabe ist, verwirft es damit die gesamte inhaltliche Information.

Aus diesem Grund werden zwei Ausprägungen der Genauigkeit unterschieden. Die *strenge* Genauigkeit setzt die Übereinstimmung von Spaltenmenge und Datensatzanzahl voraus und beträgt andernfalls null. Sie beantwortet die Frage, ob ein Ergebnis vollständig strukturkonform ist. Die *abgeglichene* Genauigkeit verlangt lediglich die Übereinstimmung des Zielschemas und bestimmt auch bei abweichender Datensatzanzahl den Anteil korrekt reproduzierter Werte, indem sie die Tabellen über die gemeinsamen Schlüssel beziehungsweise die sortierte Reihenfolge einander zuordnet und die überzähligen oder fehlenden Datensätze als Abweichung verbucht. Numerische Felder werden dabei mit einer Toleranz verglichen, um Rundungsunterschiede nicht als inhaltlichen Fehler zu werten.

Als Hauptkennzahl der Auswertung dient die abgegliche Genauigkeit, da sie den Grad der Aufgabenerfüllung abbildet und nicht lediglich deren vollständiges Gelingen oder Scheitern. Die strenge Ausprägung wird ergänzend herangezogen, wo die exakte Datensatzanzahl selbst Teil der Anforderung ist. Ergänzend wird als *Erfolgsquote* der Anteil derjenigen Läufe ausgewiesen, die überhaupt eine strukturell verwertbare Ausgabe erzeugt haben. Die Differenz zwischen Genauigkeit und Erfolgsquote trennt damit das Scheitern an der Ausführung von der inhaltlichen Abweichung eines lauffähigen Ergebnisses. Fehlgeschlagene Ausführungen gehen mit dem Wert null in alle Mittelwerte ein, da sie im praktischen Einsatz ebenfalls kein verwertbares Ergebnis liefern.

4.5.2. Vollständigkeit und Konsistenz

Die Vollständigkeit gibt an, inwieweit das Ergebnis alle geforderten Datensätze und Felder enthält, ohne dass erwartete Werte fehlen oder unzulässig entfernt werden. Die Konsistenz bezieht sich auf die Widerspruchsfreiheit der Ausgabe, insbesondere auf die Übereinstimmung von Datentypen und Formaten mit dem Zielschema [9, 11]. Beide werden über den Abgleich mit der Soll-Lösung erfasst, die eine über das Vorhandensein aller erwarteten Spalten und die korrekte Datensatzanzahl, die andere über Schemakonformität und Typisierung. Die Vollständigkeit ist dabei vor allem für die Duplikaterkennung von Bedeutung, bei der sowohl unzureichendes als auch übermäßiges Entfernen sie verletzt, die Konsistenz vor allem für die Transformationsaufgaben, bei denen die korrekte Typisierung ein wesentliches Ergebnismerkmal ist.

Anders als die Genauigkeit werden beide in der Auswertung nicht als eigenständige Kennzahlen berichtet, sondern erfüllen eine vorgelagerte Funktion. Die Vollständigkeit entscheidet darüber, ob ein Ergebnis überhaupt vergleichbar ist, und geht in die abgegliche Genauigkeit ein, indem fehlende oder überzählige Datensätze dort als Abweichung gewertet werden. Wo der Umfang eigenständige Aussagekraft besitzt, etwa bei der Zahl der entfernten Duplikate, wird er unmittelbar als Datensatzanzahl berichtet. Die Konsistenz wirkt als Ausschlusskriterium: Eine Ausgabe, die das Zielschema verletzt, gilt als strukturell nicht verwertbar, geht mit dem Wert null in die Genauigkeit ein und wird als *Schema-Bruch* gesondert von einem Programmabbruch ausgewiesen. Berichtet werden in Kapitel 8 folglich zwei Qualitätsgrößen, die abgegliche Genauigkeit und die Erfolgsquote, in denen Vollständigkeit und Konsistenz als strukturelle Voraussetzungen aufgehen, sowie die Effizienzgrößen des folgenden Abschnitts.

4.5.3. Effizienz

Die Effizienz erfasst den für die Verarbeitung erforderlichen Aufwand und ergänzt die qualitätsbezogenen Kriterien um die für den praktischen Einsatz entscheidende Wirtschaftlichkeitsdimension [18, 22]. Erhoben werden die Verarbeitungszeit eines Aufrufs, der Verbrauch an Token sowie die daraus resultierenden Nutzungskosten. Diese Kennzahlen sind insbesondere für die vergleichende Bewertung der Prompting-Strategien relevant, da aufwändigere Strategien mit längeren Eingaben einen höheren Token-Verbrauch verursachen und somit zwischen Ergebnisqualität und Effizienz abzuwägen ist [21, 22]. Zugleich ermöglichen sie den praxisnahen Vergleich zwischen kommerziell betriebenen und lokal ausgeführten Modellen sowie die Gegenüberstellung mit der klassischen Pipeline, deren Ausführung keine vergleichbaren Modellkosten verursacht.

4.6. Vergleich gegen eine manuelle ETL-Pipeline

Um die Ergebnisse der LLM-gestützten Verarbeitung einordnen zu können, wird als Vergleichsmaßstab eine klassische, regelbasiert implementierte ETL-Pipeline herangezogen, die dieselben neun Aufgaben auf demselben Datensatz löst. Diese Pipeline bildet den etablierten Stand der Technik ab, bei dem die Verarbeitungslogik manuell und explizit in Form von Regeln und Skripten vorgegeben wird, und sie wird ebenfalls programmatisch auf Basis der Bibliothek Pandas umgesetzt, um eine technisch vergleichbare Ausgangslage herzustellen [8, 10]. Sie dient als Referenzpunkt, gegen den sich die Stärken und Schwächen der KI-gestützten Ansätze messen lassen, und liefert damit die Grundlage zur Beantwortung von FF1.

Charakteristisch für diese Vergleichsbasis ist, dass sie deterministische und vollständig nachvollziehbare Ergebnisse erzeugt: Identische Eingaben führen stets zu identischen Ausgaben, und die zugrunde liegende Logik ist explizit einsehbar [2, 3]. Damit bildet die regelbasierte Pipeline insbesondere bei den klar formalisierbaren Aufgaben die Referenz, während gerade bei den semantisch geprägten Aufgaben der Länder-Vereinheitlichung und der Fuzzy-Duplikaterkennung erwartet wird, dass sich die Grenzen rein regelbasierter Verfahren zeigen. Diese arbeiten zwar auch bei solchen Aufgaben deterministisch, erfassen die zugrunde liegende semantische Ähnlichkeit jedoch nur eingeschränkt. Die Pipeline wird mit den identischen Evaluationskriterien aus Abschnitt 4.5 bewertet, sodass ein unmittelbarer Vergleich zwischen klassischem und KI-gestütztem Vorgehen unter gleichen Bedingungen möglich ist und sich auf dieser

Grundlage in Kapitel 9 Handlungsempfehlungen ableiten lassen. Ein hybrider Ansatz, der beide Verfahren kombiniert, ist dabei nicht selbst Gegenstand der Evaluierung. Da sich die Nachvollziehbarkeit regelbasierter und die Flexibilität KI-gestützter Verfahren in der Praxis jedoch häufig ergänzen, wird er als naheliegende Konsequenz des Vergleichs in den Handlungsempfehlungen aufgegriffen [2, 30].

5. Erstellung des synthetischen Datensatzes

Das in Kapitel 4 entworfene Untersuchungsdesign setzt einen Datensatz voraus, dessen Eigenschaften exakt bekannt sind: Nur wenn die enthaltenen Datenqualitätsprobleme gezielt eingebracht wurden und für jede Aufgabe eine eindeutige Soll-Lösung existiert, lassen sich die in Abschnitt 4.5 definierten Kriterien überhaupt objektiv messen. Dieses Kapitel dokumentiert daher die Erstellung des synthetischen Datensatzes, der in Abschnitt 4.1 bereits als unveränderliche Grundlage aller Experimente eingeführt wurde. Es schließt unmittelbar an die in Abschnitt 4.3 definierten neun ETL-Aufgaben an und zeigt, wie das kleine E-Commerce-Szenario mit den Tabellen Kunden, Produkte und Bestellungen so konstruiert wird, dass genau die dort geforderten Probleme in kontrolliertem Umfang vorliegen. Zunächst werden die aus dem Forschungsdesign abgeleiteten Anforderungen begründet (Abschnitt 5.1), anschließend das Datenmodell und die saubere Zielstruktur beschrieben (Abschnitt 5.2). Darauf folgt die detaillierte Zuordnung der gezielt eingebauten Qualitätsprobleme zu den einzelnen Aufgaben (Abschnitt 5.3), bevor das zweistufige Generierungsverfahren (Abschnitt 5.4) und die abschließende Validierung des Datensatzes (Abschnitt 5.5) dargelegt werden. Der so erzeugte Datensatz bildet die Eingabe für die technische Umsetzung in Kapitel 6 und die Durchführung der Experimente in Kapitel 7.

5.1. Anforderungen an den Datensatz

Die Anforderungen an den Datensatz ergeben sich unmittelbar aus den Forschungsfragen und dem in Kapitel 4 festgelegten Design. Eine erste Anforderung betrifft die Repräsentativität: Der Datensatz muss die in Abschnitt 2.2.2 beschriebenen, in der Praxis typischen Datenqualitätsprobleme abbilden, damit die Ergebnisse über das konkrete Szenario hinaus aussagekräftig sind und die Aufgaben nicht künstlich vereinfacht wirken [11, 12]. Zugleich soll er nicht beliebige, sondern gerade jene Probleme enthalten,

die den neun in Abschnitt 4.3 definierten Aufgaben entsprechen, sodass jede Aufgabe an einem tatsächlich vorhandenen Mangel ansetzt.

Eine zweite, für das gewählte experimentelle Vorgehen zentrale Anforderung ist die vollständige Kontrollierbarkeit. Da die Qualität der Verarbeitung durch den Abgleich mit einer bekannten Soll-Lösung gemessen wird, müssen sowohl der fehlerfreie Sollzustand als auch Art und Umfang der eingebrachten Fehler exakt bekannt sein. Bei realen Datenbeständen wäre diese eindeutige Referenz nur mit erheblichem manuellem Annotationsaufwand und verbleibender Unsicherheit herstellbar, weshalb die Literatur den Einsatz synthetischer Daten gerade dann empfiehlt, wenn eine kontrollierte und exakt bekannte Ausgangslage benötigt wird [26, 37]. Die in Abschnitt 3.2 referierten Vorbehalte gegenüber synthetischen Daten betreffen primär deren Einsatz als Trainingsmaterial. Für die hier verfolgte Rolle als kontrollierte Testgrundlage überwiegt hingegen der Vorteil der bekannten Ground Truth [24, 28].

Eine dritte Anforderung ist die Reproduzierbarkeit des Datensatzes selbst. Da die Reproduzierbarkeit der Modellergebnisse eine eigenständige, querschnittliche Untersuchungsdimension bildet, darf die Datengrundlage keine zusätzliche Variabilität einführen. Die Generierung muss deterministisch sein, sodass sie jederzeit identisch wiederholt werden kann [5, 6]. Schließlich soll der Datensatz eine vierte Anforderung erfüllen, die zwischen Vielfalt und Handhabbarkeit vermittelt: Er muss groß und vielfältig genug sein, um die drei Schwierigkeitsstufen jeder Aufgabe sinnvoll auszuprägen, zugleich aber überschaubar genug bleiben, um die in Abschnitt 4.5.3 erhobenen Aufwandskennzahlen nicht durch reine Datenmenge zu dominieren und die wiederholte Ausführung aller Konfigurationen praktikabel zu halten.

5.2. Datenmodell und -struktur

Als Anwendungsszenario dient ein bewusst einfach gehaltenes E-Commerce-Datenmodell, das aus den drei Tabellen Kunden, Produkte und Bestellungen besteht. Diese Wahl ist kein Selbstzweck, sondern folgt aus den Aufgaben: Das Szenario ist allgemein verständlich, deckt sämtliche in Abschnitt 4.3 geforderten Problemtypen ab und stellt über die Beziehung der Bestellungen zu Kunden und Produkten zugleich die Schlüsselstruktur bereit, die für die anspruchsvolle Transformationsaufgabe aus Join, Bereinigung und Aggregation benötigt wird [1, 2]. Die Tabelle der Bestellungen verweist über Fremdschlüssel auf Kunden und Produkte und verbindet die drei Tabellen so zu einem kleinen relationalen Modell.

Grundlage des Modells ist die Definition eines sauberen Zielschemas für jede Tabelle, das den fehlerfreien Sollzustand festlegt. Für die Kundentabelle umfasst dieses Schema eine eindeutige Kundennummer, den vollständigen Namen, die E-Mail-Adresse, einen zweistelligen Länder-Code nach ISO-3166-1-alpha-2 sowie das Registrierungsdatum. Die Produkttabelle besteht aus einer Produktnummer, der Bezeichnung, einer Kategorie, dem Preis als nichtnegativem Dezimalwert und einem Wahrheitswert zur Verfügbarkeit. Die Bestelltabelle schließlich verknüpft Bestell-, Kunden- und Produktnummer mit der bestellten Menge, dem Einzelpreis zum Bestellzeitpunkt und dem Bestelldatum. Der Gesamtbetrag ist dort bewusst *nicht* enthalten, da seine Berechnung Gegenstand der mittleren Transformationsaufgabe ist. Er tritt erst in deren Soll-Lösung als abgeleitete Spalte hinzu. Dieses saubere Schema erfüllt eine doppelte Funktion: Es beschreibt einerseits das von den Modellen zu erreichende Zielformat und dient andererseits, in seiner konkret erzeugten Ausprägung, als bekannte Referenz für die Bewertung. Die typisierte, maschinenlesbare Festlegung des Schemas stellt dabei sicher, dass die als Referenz erzeugten Daten strikt schemakonform sind und damit als verlässliche Ground Truth taugen [9, 11].

Der Umfang des Datensatzes ist so gewählt, dass er die genannte Abwägung zwischen Vielfalt und Handhabbarkeit einlöst. Die Kundentabelle umfasst in ihrer sauberen Fassung fünfhundert Datensätze, die Produkttabelle einhundert und die Bestelltabelle zweitausend Datensätze. Diese Größenordnung ist einerseits ausreichend, um Datenqualitätsprobleme in statistisch merklichem, aber nicht trivialem Ausmaß einzustreuen und die Aggregation über Länder und Kategorien mit belastbaren Häufigkeiten zu füllen, andererseits klein genug, um die in Abschnitt 4.6 vorgesehene wiederholte Ausführung über alle Kombinationen aus Modell, Prompting-Strategie und Aufgabe ohne unverhältnismäßigen Zeit- und Kostenaufwand zu ermöglichen.

5.3. Eingebaute Datenqualitätsprobleme

Kern der Datensatzkonstruktion ist das gezielte Einbringen von Qualitätsproblemen, das dieser Abschnitt entlang der drei in Abschnitt 4.3 definierten Aufgabenkategorien beschreibt. Jedes Problem ist dabei nicht beliebig gewählt, sondern korrespondiert mit genau einer Aufgabe und einer der in Abschnitt 2.2.1 eingeführten Qualitätsdimensionen, sodass sich die spätere Bewertung lückenlos auf einen bewusst gesetzten Mangel zurückführen lässt [11, 12]. Auf diese Weise ist die in Abschnitt 4.3 eingeführte Matrix aus neun Aufgaben und drei Schwierigkeitsstufen unmittelbar in der Struktur des Datensatzes verankert.

Die Probleme der *Datenbereinigung* sind ausschließlich in der Kundentabelle verortet und in aufsteigender Schwierigkeit gestaffelt: zusätzliche Leerzeichen in den Namensfeldern und entfernte Länderangaben für die einfache Stufe, das in vier Darstellungen abgelegte Registrierungsdatum für die mittlere und die Verfälschung der Länderangaben durch wechselnde Schreibweisen, Sprachen, Abkürzungen und gezielte Tippfehler für die anspruchsvolle [9, 12]. Gerade die letztgenannte Verfälschung bildet die in Abschnitt 4.3.1 hervorgehobene semantische Achse ab, da ihre Auflösung ein inhaltliches Verständnis der Werte und nicht nur einen syntaktischen Abgleich erfordert [6, 23].

Die Probleme der *Datentransformation* betreffen die Produkt- und die Bestelltabelle und knüpfen an die Dimension der Konsistenz an. In der Produkttabelle werden die numerischen Preisangaben in uneinheitliche textuelle Schreibweisen überführt und die Verfügbarkeit von einem Wahrheitswert in wechselnde textuelle Entsprechungen umgeschrieben, womit die einfache Aufgabe der Typkonvertierung ihre Grundlage erhält. Die mittlere Stufe benötigt keine eigene Verfälschung, sondern nutzt die vorhandene Struktur der Bestelltabelle, aus der sich Gesamtbetrag und Datumsbestandteile ableiten lassen. Die anspruchsvolle Stufe schließlich verlangt die Verknüpfung aller drei Tabellen über ihre Schlüsselbeziehungen, eine begleitende Bereinigung der zuvor beschriebenen Mängel und eine abschließende Aggregation über Land und Produktkategorie. Sie bündelt damit bewusst mehrere eingebaute Probleme in einem mehrstufigen Verarbeitungsschritt, wie er für reale ETL-Strecken charakteristisch ist [1, 2].

Die Probleme der *Duplikaterkennung* zielen auf die Eindeutigkeit der Kundentabelle. Für die einfache und die mittlere Stufe werden vollständig identische Datensätze eingefügt, die sowohl als exakte Duplikate erkennbar sind als auch, über die mehrfach vergebene E-Mail-Adresse, die schlüsselbasierte Entdoppelung ermöglichen. Für die anspruchsvolle Stufe tritt derselbe Kunde mit geringfügig abweichendem Namen und abweichender E-Mail-Schreibweise erneut auf. Da solche Fuzzy-Duplikate nur über eine Ähnlichkeitsbetrachtung auflösbar sind, bilden sie die zweite semantische Achse der Untersuchung [6, 12]. Der Anteil der eingebrachten Probleme ist durchweg so bemessen, dass er die Ergebniskennzahlen merklich beeinflusst, ohne den Charakter des Datensatzes als überwiegend sauberen Bestand zu verfälschen: Die zusätzlichen exakten und unscharfen Duplikate betreffen jeweils rund vier Prozent der Kundendatensätze.

Wie sich das konkret in den Daten niederschlägt, zeigt Tabelle 5.1 anhand tatsächlicher Zeilen des erzeugten Datensatzes. Die Gegenüberstellung von Roh- und Sollwert

macht zugleich sichtbar, weshalb die Aufgaben unterschiedlich schwer sind: Während sich Leerzeichen und Datumsformate durch eine endliche Menge syntaktischer Regeln vollständig auflösen lassen, ist bei den Länderangaben für jeden einzelnen Wert zu entscheiden, welches Land gemeint ist. Fehlende Werte werden nicht gelöscht, sondern einheitlich als UNKNOWN gekennzeichnet, sodass die Zeilenzahl erhalten bleibt und die Vollständigkeit im Sinne von Abschnitt 4.5.2 messbar bleibt.

Tabelle 5.1.: Beispiele tatsächlich erzeugter Rohwerte und ihrer Sollwerte, gegliedert nach der zugehörigen Aufgabe. Sichtbare Leerzeichen sind mit _ gekennzeichnet.

Aufgabe	Rohwert	Sollwert
Leerzeichen	_ _Timm Haering-Faust_ _	Timm Haering-Faust
fehlender Wert	(leer)	UNKNOWN
Datumsformat	2025-06-27 (ISO)	2025-06-27
Datumsformat	05.11.2023 (deutsch)	2023-11-05
Datumsformat	December 22 2025 (US)	2025-12-22
Datumsformat	1639353600 (Unix)	2021-12-13
Länderangabe	Deutschland / Germany	DE
Länderangabe	Deutshcland (Typfehler)	DE
Länderangabe	SUI / Schweiz	CH
Länderangabe	Holland / Niederlande	NL
Länderangabe	England / UK	GB
Preisangabe	134.94 EUR	134.94
Preisangabe	€498.06	498.06
Preisangabe	323,37	323.37
Verfügbarkeit	yes / true / 1	True
Verfügbarkeit	no / false / 0	False

Da der Kern eines Duplikats im Verhältnis zweier Datensätze zueinander liegt, stellt Tabelle 5.2 sie zeilenweise gegenüber. Bei den exakten Duplikaten ist der eingefügte Datensatz in allen Feldern einschließlich der Kundennummer identisch. Bei den unscharfen weichen Name und E-Mail-Adresse durch genau jene Muster ab, die auch in realen Beständen entstehen, veränderte Groß- und Kleinschreibung, ein zusätzliches Leerzeichen, ein Punkt im lokalen Teil der Adresse. Die inhaltliche Identität bleibt für einen Menschen unmittelbar erkennbar, ist über einen exakten Vergleich jedoch nicht mehr zugänglich, weshalb diese Aufgabe die in Abschnitt 4.3.3 beschriebene Ähnlichkeitsbetrachtung erzwingt.

Aus den 500 sauberen Kundendatensätzen entstehen durch das Anfügen von 20 exakten und 20 unscharfen Duplikaten insgesamt 540 Rohdatensätze, was die in Abschnitt 5.2 genannte Zielgröße von 500 eindeutigen Kunden als Sollzustand der Dupli-

Tabelle 5.2.: Eingefügte Duplikate im Vergleich zu ihrem Ausgangsdatensatz. Bei den unscharfen Duplikaten sind die abweichenden Stellen die einzige Änderung gegenüber dem Original.

Art	Nr.	Name	E-Mail
exakt (Original)	145	Angelina Siering	waehnerbirgitt@example.net
exakt (Duplikat)	145	Angelina Siering	waehnerbirgitt@example.net
unscharf (Original)	308	Uta Lange	gkabus@example.org
unscharf (Duplikat)	505	Uta_░░Lange	gka.bus@example.org
unscharf (Original)	493	Bernfried Harloff	gunter53@example.org
unscharf (Duplikat)	520	BERNFRIED HARLOFF	GUNTER53@example.org
unscharf (Original)	465	Norma Schmiedecke-Hofmann	elfriede16@example.org
unscharf (Duplikat)	510	norma schmiedecke-hofmann	ELFRIEDE16@example.org

kataufgaben festlegt. Die Häufigkeitsverteilung der Länderangaben bestätigt zugleich die beabsichtigte Streuung: Für die zehn im Datensatz vertretenen Länder liegen je Land drei bis sechs verschiedene Schreibweisen vor, von Standardcodes wie DE über ausgeschriebene deutsche und englische Namen bis zu untypischen Kürzeln wie SUI für die Schweiz oder GER für Deutschland, und 45 Angaben fehlen vollständig. Genau diese Streuung ist es, an der sich in Kapitel 8 der Unterschied zwischen regelbasierter Zuordnung und sprachlichem Verständnis bemessen lässt.

5.4. Generierungsverfahren und verwendete Werkzeuge

Die Erzeugung des Datensatzes folgt einem zweistufigen Prinzip, das die in Abschnitt 5.1 geforderte Kontrollierbarkeit unmittelbar umsetzt. Im ersten Schritt werden für jede Tabelle vollständig saubere, strikt schemakonforme Daten erzeugt, die den fehlerfreien Sollzustand verkörpern. Im zweiten Schritt werden aus dieser sauberen Fassung durch eine kontrollierte Verfälschung die tatsächlich zu verarbeitenden Rohdaten abgeleitet, während die saubere Fassung selbst als Referenz erhalten bleibt. Dieses Vorgehen kehrt den üblichen Ablauf einer ETL-Strecke bewusst um: Nicht die Bereinigung wird nachträglich auf unbekannte Rohdaten angewandt, sondern die Verunreinigung wird gezielt auf bekannte saubere Daten aufgebracht, sodass die exakte Umkehrung jeder Verfälschung und damit die eindeutige Soll-Lösung jeder Aufgabe von vornherein feststeht [26, 37].

Für die Erzeugung realistisch wirkender Basiswerte wie Namen und E-Mail-Adressen

wird eine etablierte Bibliothek zur Generierung synthetischer Testdaten eingesetzt, deren deutschsprachige Lokalisierung dem Szenario angemessene Werte liefert. Die kontrollierte Verfälschung, also das Einstreuen von Leerzeichen, fehlenden Werten, uneinheitlichen Formaten, verfälschten Länderangaben sowie exakten und unscharfen Duplikaten, erfolgt anschließend durch eigens implementierte Routinen, die jede der in Abschnitt 5.3 beschriebenen Problemklassen erzeugen. Sowohl die Basisdaten als auch die Verfälschung werden über einen zentralen Zufallszahlengenerator gesteuert, der mit einem festen Startwert initialisiert wird. Dadurch ist die Generierung vollständig deterministisch: Derselbe Startwert führt stets zu einem identischen Datensatz, während unterschiedliche Startwerte voneinander unabhängige, aber strukturell gleichartige Datensätze erzeugen. Diese Determiniertheit stellt sicher, dass die untersuchte Variabilität allein den Modellen und nicht der Datengrundlage zuzurechnen ist [5, 6]. Die erzeugten Rohdaten werden in einem gängigen tabellarischen Austauschformat abgelegt und dienen zusammen mit der separat gespeicherten sauberen Referenz als Eingabe für die in Kapitel 6 beschriebene Verarbeitungsumgebung.

5.5. Validierung des Datensatzes

Bevor der Datensatz als Grundlage der Experimente verwendet wird, ist zu prüfen, ob er die in Abschnitt 5.1 formulierten Anforderungen tatsächlich erfüllt. Die Validierung setzt zunächst an der sauberen Referenz an: Da diese als maßgebliche Ground Truth dient, wird ihre strikte Konformität mit dem definierten Zielschema überprüft, sodass sichergestellt ist, dass die als korrekt geltenden Werte die geforderten Datentypen, Wertebereiche und Formate einhalten [9, 11]. Nur unter dieser Voraussetzung ist der spätere Abgleich der Modellergebnisse mit der Referenz aussagekräftig.

In einem zweiten Schritt wird geprüft, ob die eingebauten Qualitätsprobleme in der beabsichtigten Art und im vorgesehenen Umfang vorhanden sind. Kontrolliert wird insbesondere, dass jede der neun Aufgaben aus Abschnitt 4.3 auf einen tatsächlich vorliegenden Mangel trifft, dass die verschiedenen Datums- und Preisformate wie vorgesehen auftreten und dass sich die exakten und unscharfen Duplikate eindeutig auf ihre jeweiligen Ausgangsdatsätze zurückführen lassen. Damit ist zugleich gewährleistet, dass für jede Aufgabe eine eindeutige Soll-Lösung existiert, was die in Abschnitt 4.5 definierten Kriterien überhaupt erst berechenbar macht [8, 12]. Ergänzend wird die Reproduzierbarkeit der Generierung dadurch bestätigt, dass die wiederholte Ausführung mit identischem Startwert einen bitgenau identischen Datensatz liefert.

Mit dieser Validierung steht ein Datensatz zur Verfügung, dessen Sollzustand, Fehlerbild und Erzeugung vollständig bekannt und nachvollziehbar sind und der damit die kontrollierte Vergleichsgrundlage für die weitere Untersuchung bildet. Zugleich ist die dem Vorgehen innewohnende Einschränkung zu benennen: Ein synthetischer Datensatz kann die Vielfalt und die unvorhergesehenen Unregelmäßigkeiten realer Datenbestände nur annähern, weshalb die aus ihm gewonnenen Ergebnisse unter diesem Vorbehalt zu interpretieren und im Rahmen der Limitationen in Kapitel 10 erneut aufzugreifen sind [26, 28]. Auf der so geschaffenen Grundlage kann im folgenden Kapitel 6 die technische Umsetzung der Verarbeitungsumgebung beschrieben werden, welche die hier erzeugten Daten sowohl der klassischen Pipeline als auch den LLM-gestützten Verfahren zuführt.

6. Implementierung

Das vorangegangene Kapitel 4 hat das Untersuchungsdesign entworfen und der Datensatz aus Kapitel 5 stellt die unveränderliche Datengrundlage bereit. Dieses Kapitel beschreibt, wie dieses Design in ein lauffähiges Softwaresystem überführt wurde, mit dem sich die vollständige Matrix aus Modell, Prompting-Strategie und ETL-Aufgabe automatisiert durchführen und einheitlich bewerten lässt. Zunächst wird die modulare Systemarchitektur und die Versuchsumgebung dargestellt (Abschnitt 6.1), gefolgt von den eingesetzten Technologien und Bibliotheken (Abschnitt 6.2). Anschließend werden die Umsetzung der klassischen Vergleichspipeline (Abschnitt 6.3), die Integration der LLMs über eine einheitliche Schnittstelle (Abschnitt 6.4) sowie die konkrete Realisierung der Prompting-Strategien (Abschnitt 6.5) beschrieben. Den Abschluss bildet das automatisierte Mess- und Auswertungsverfahren (Abschnitt 6.6), das die in Abschnitt 4.5 definierten Kriterien technisch operationalisiert und damit die Grundlage für die Durchführung in Kapitel 7 schafft.

6.1. Systemarchitektur und Versuchsumgebung

Das System ist als modulares Python-Projekt aufgebaut, dessen Komponenten entlang der einzelnen Verarbeitungsschritte getrennt sind und über klar umrissene Schnittstellen zusammenwirken. Diese Trennung folgt dem Ziel, die einzelnen Faktoren des Versuchsdesigns unabhängig voneinander austauschen zu können, ohne den übrigen Ablauf zu verändern, und erleichtert zugleich die Nachvollziehbarkeit der Ergebnisse [3, 8]. Ein Datenmodul kapselt die Erzeugung des synthetischen Datensatzes, die

Referenzlösungen sowie die Definition der neun Aufgaben, ein Modellmodul stellt die Anbindung der LLMs bereit, ein Modul für die klassische Pipeline enthält die regelbasierte Vergleichslösung, ein Experimentmodul steuert den Ablauf und die Ausführung des erzeugten Codes, ein Evaluationsmodul bewertet die Ergebnisse, und ein Auswertungsmodul bereitet die erhobenen Messdaten für die Analyse auf.

Der grundlegende Datenfluss folgt für jede Konfiguration demselben Muster: Ausgehend vom synthetischen Datensatz und der Aufgabenbeschreibung wird ein Prompt erzeugt, an das jeweilige Modell übergeben und dessen Antwort weiterverarbeitet. Entsprechend der in Abschnitt 4.1 getroffenen Gestaltungsentscheidung erzeugt das Modell dabei nicht das Ergebnis unmittelbar, sondern ein ausführbares Verarbeitungs-skript, das anschließend deterministisch auf den Daten ausgeführt wird. Das so entstandene Ergebnis wird als tabellarische Ausgabedatei gespeichert, gegen die bekannte Soll-Lösung geprüft und gemeinsam mit allen erhobenen Kennzahlen in einem unveränderlichen Ergebnisdatensatz protokolliert [1, 5]. Sämtliche steuerbaren Randbedingungen sind in einer zentralen Konfiguration gebündelt, die unter anderem den Zufallskern für die Datenerzeugung, die konstant niedrig gehaltene Temperatur, die Obergrenze der Ausgabe-Token sowie die Anzahl der Wiederholungen je Konfiguration festlegt. Diese zentrale und explizit hinterlegte Parametrisierung ist eine wesentliche Voraussetzung für die in Abschnitt 4.5 als querschnittliche Dimension eingeführte Reproduzierbarkeit, da sie die Bedingungen jedes Durchlaufs eindeutig dokumentiert [5, 6]. Die Ausführung erfolgt durchgängig auf demselben Rechner unter identischer Laufzeitumgebung. Die konkreten Angaben dazu finden sich in Abschnitt 7.3.

6.2. Eingesetzte Technologien und Bibliotheken

Als Implementierungssprache dient Python. Ausschlaggebend sind zwei nachprüfbare Gründe: Sämtliche untersuchten Modellanbieter stellen offizielle Python-Schnittstellen bereit, und die für die Datenverarbeitung benötigten Bibliotheken, allen voran Pandas, liegen dort in ausgereifter Form vor, wie auch die einschlägigen Übersichtsarbeiten zur automatisierten Datenverarbeitung durchgehend voraussetzen [7, 12]. Die eigentliche Datenverarbeitung stützt sich sowohl in der klassischen Pipeline als auch im vom Modell erzeugten Code auf die Bibliothek Pandas, die sich als De-facto-Standard für tabellarische Verarbeitung etabliert hat. Ihre einheitliche Verwendung stellt sicher, dass die regelbasierte und die KI-gestützte Lösung auf derselben technischen Grundlage arbeiten und damit unmittelbar vergleichbar bleiben [1, 3]. Sämtliche Zwischen- und Ergebnistabellen werden im spaltenorientierten Parquet-Format ausgetauscht, da dieses

die Datentypen der Spalten mitführt und so einen typerhaltenden Abgleich zwischen erzeugter Ausgabe und Soll-Lösung ermöglicht, wie er für das Konsistenzkriterium aus Abschnitt 4.5.2 benötigt wird.

Die Anbindung der Modelle erfolgt über die jeweils offiziellen Softwarebibliotheken der Anbieter für die geschlossenen Modelle von OpenAI, Anthropic und Google sowie über eine lokale Laufzeitumgebung für das offen verfügbare Modell der Llama-Reihe. Diese heterogenen Schnittstellen werden, wie in Abschnitt 6.4 beschrieben, hinter einer einheitlichen Abstraktion zusammengeführt, sodass der Versuchsablauf unabhängig vom konkreten Anbieter bleibt [5, 18]. Ergänzend kommen etablierte Hilfsbibliotheken zum Einsatz: eine typsichere Konfigurationsverwaltung zur Bündelung der Versuchsparmeter, eine Auszeichnungssprache zur versionierten Ablage der Prompt-Vorlagen sowie Werkzeuge zur Visualisierung und interaktiven Auswertung, mit denen die in Kapitel 8 dargestellten Tabellen und Abbildungen erzeugt werden. Als repräsentative Vertreter der jeweiligen Anbieter wird je ein aktuelles Modell eingesetzt, dessen exakte Versionskennung bei jedem Aufruf protokolliert wird, um der raschen Weiterentwicklung des Modellfeldes Rechnung zu tragen und die spätere Nachvollziehbarkeit zu sichern [5, 18].

6.3. Umsetzung der klassischen ETL-Pipeline

Die in Abschnitt 4.6 als Vergleichsmaßstab definierte klassische Pipeline ist als regelbasierte Lösung umgesetzt, die dieselben neun Aufgaben ausschließlich mit fest vorgegebener Verarbeitungslogik und ohne Zugriff auf ein Sprachmodell löst. Für die klar formalisierbaren Aufgaben, etwa das Entfernen exakter Duplikate oder die Typkonvertierung, nutzt sie dieselben deterministischen Routinen, die auch der Erzeugung der Soll-Lösung zugrunde liegen. Für diese Aufgaben ist die explizit formulierte Regel zugleich die korrekte Lösung, sodass hier definitionsgemäß eine hohe Ergebnisqualität zu erwarten ist [2, 3].

Von besonderer Bedeutung für die Aussagekraft des Vergleichs ist die bewusste Gestaltung der Pipeline bei den semantisch geprägten Aufgaben. Für die Vereinheitlichung der Länderangaben greift sie auf eine absichtlich begrenzte Zuordnungstabelle zurück, die zwar ausgeschriebene Ländernamen und Standardcodes abdeckt, jedoch keine Tippfehler, untypischen Abkürzungen oder umgangssprachlichen Bezeichnungen berücksichtigt. Nicht auflösbare Werte werden einheitlich als unbekannt gekennzeichnet. Für die Erkennung unscharfer Duplikate bildet sie einen normalisierten

Schlüssel aus Name und E-Mail-Bestandteil und entfernt darüber Mehrfachnennungen, kann damit aber nur solche Abweichungen erfassen, die sich durch einfache Normalisierung angleichen lassen. Diese Heuristiken sind realistisch für ein rein regelbasiertes Vorgehen, stoßen jedoch genau dort an ihre Grenzen, wo ein inhaltliches Verständnis der Werte erforderlich wäre. Gerade an dieser Stelle wird der in Hypothese H1 erwartete Unterschied zwischen regelbasierter und KI-gestützter Verarbeitung messbar [6, 12]. Da die Pipeline vollständig deterministisch arbeitet und identische Eingaben stets zu identischen Ausgaben führen, dient sie im weiteren Verlauf als stabiler und nachvollziehbarer Referenzpunkt, gegen den sich die Ergebnisse der Modelle einordnen lassen.

6.4. Integration der LLMs in die Pipeline

Um die in Abschnitt 4.2 ausgewählten Modelle trotz ihrer unterschiedlichen Schnittstellen einheitlich ansprechen zu können, werden sie hinter einer gemeinsamen Abstraktion gekapselt. Jeder Anbieter implementiert dieselbe abstrakte Schnittstelle, die einen Aufruf entgegennimmt und eine standardisierte Antwort zurückliefert, in der neben dem erzeugten Text auch die exakte Modellkennung, der Verbrauch an Eingabe- und Ausgabe-Token sowie die Antwortzeit enthalten sind. Eine vorgeschaltete Erzeugungsinstanz liefert anhand eines Anbieternamens die passende Implementierung, sodass der Versuchsablauf vollständig unabhängig vom konkreten Modell bleibt und die Faktorachse Modell ohne Eingriffe in die übrige Logik variiert werden kann [5, 18]. Diese standardisierte Antwort ist zugleich die Datenbasis für die spätere Effizienzbewertung nach Abschnitt 4.5.3, da sie die dafür benötigten Kennzahlen bei jedem Aufruf unmittelbar mitführt.

Der eigentliche Verarbeitungsschritt folgt dem Ansatz der Code-Generierung: Aus der Modellantwort wird der enthaltene Python-Code extrahiert und anschließend in einem eigenen Betriebssystemprozess mit fester Zeitbegrenzung ausgeführt. Ein Durchlauf gilt nur dann als erfolgreich, wenn der Prozess fehlerfrei endet und tatsächlich die erwartete Ausgabedatei erzeugt wurde. Andernfalls wird das Ergebnis als fehlgeschlagene Ausführung protokolliert. Da an dieser Stelle von einem Modell erzeugter Code ausgeführt wird, ist die Ausführung in einen separaten Prozess mit Zeitlimit ausgelagert. Dieses Vorgehen ist für die kontrollierte, lokale Durchführung der Experimente vertretbar, stellt jedoch bewusst keine vollständige Sicherheitsisolation dar und wäre für einen unbeaufsichtigten Produktivbetrieb entsprechend abzusichern [6, 23].

Ein zentrales Element der Integration ist die Behandlung von Fehlern, die aufgrund der probabilistischen Natur der Modelle und der Nutzung externer Dienste unvermeidlich auftreten. Dabei wird bewusst zwischen zwei Arten von Fehlern unterschieden: Vorübergehende Störungen des Dienstes, die sich etwa in bestimmten Statuscodes oder Verbindungsabbrüchen äußern, werden als transient eingestuft und der betreffende Aufruf mit exponentiell wachsender Wartezeit mehrfach wiederholt. Ein inhaltlich fehlerhaftes, aber technisch wohlgeformtes Ergebnis, etwa ein Skript, das mit einem Programmfehler abbricht, wird hingegen nicht wiederholt, da es einen gültigen Messwert über die Fähigkeit des Modells darstellt und eine Wiederholung das Ergebnis verfälschen würde [5, 6]. Ergänzend werden anbieterspezifische Besonderheiten aktueller Modelle abgefangen: Lehnt ein Modell einen Sampling-Parameter wie die Temperatur ab, wird dieser automatisch weggelassen und die Anpassung im Ergebnisdatensatz vermerkt, sodass solche Abweichungen bei der Interpretation der Reproduzierbarkeit sichtbar bleiben. Für jeden Aufruf werden schließlich sämtliche relevanten Angaben, von der Modellkennung über den Token-Verbrauch und die Kosten bis hin zum erzeugten Code und dem Ausführungsergebnis, vollständig protokolliert und bilden so die lückenlose Grundlage für die anschließende Auswertung.

Der in Abschnitt 4.1 eingeführte zweite Ausführungsmodus, die direkte In-Context-Verarbeitung, ist über dieselbe einheitliche Modellschnittstelle umgesetzt, unterscheidet sich jedoch im Ablauf. Hier werden die Eingabedaten unmittelbar in den Prompt eingebettet und das Modell angewiesen, die transformierte Ergebnistabelle direkt in einem klar abgegrenzten Format zurückzugeben, das anschließend eingelesen und gegen die Soll-Lösung geprüft wird. Die Auswertung der Antwort ist dabei bewusst robust gegenüber am Ausgabelimit abgeschnittenen Antworten gestaltet, sodass auch unvollständige Ergebnisse noch verarbeitet und als solche erkannt werden. Da aktuelle Modelle standardmäßig einen internen Denkprozess ausführen, der das für die Ergebnistabelle verfügbare Ausgabebudget aufzehren würde, bevor die eigentlichen Daten erzeugt werden, wird dieses Verhalten für den Direkt-Modus deaktiviert und das Ausgabelimit angehoben [5, 22]. Anders als bei der wiederholten Code-Erzeugung wird jede Konfiguration in diesem Modus nur einmal ausgeführt, um den durch die eingebetteten Daten erhöhten Token-Verbrauch zu begrenzen. Eine Wiederholung erfolgt ausschließlich bei einem technischen Fehlschlag wie einem transienten Anbieterfehler oder einer leeren beziehungsweise nicht auswertbaren Antwort, nicht hingegen bei einem wohlgeformten, aber inhaltlich abweichenden Ergebnis, das einen gültigen Messwert darstellt.

6.5. Umsetzung der Prompting-Strategien

Die in Abschnitt 4.4 konzipierten Strategien sind als versionierte Vorlagen umgesetzt, die getrennt vom Programmcode in einem eigenen, menschenlesbaren Format abgelegt sind. Jede Vorlage besteht aus einer eindeutigen Kennung, einer Systemanweisung und einer Nutzervorlage mit klar benannten Platzhaltern, in die zur Laufzeit die Aufgabenbeschreibung sowie je nach Ausführungsmodus die Ein- und Ausgabepfade oder die eingebetteten Daten eingesetzt werden. Diese versionierte und vom Code entkoppelte Ablage stellt sicher, dass jede eingesetzte Prompt-Variante eindeutig dokumentiert, unverändert wiederverwendbar und damit nachvollziehbar bleibt, was eine wesentliche Voraussetzung für die Vergleichbarkeit und Reproduzierbarkeit der Ergebnisse ist [21, 22].

Alle drei Strategien folgen einem einheitlichen Grundaufbau und unterscheiden sich ausschließlich im Umfang der zusätzlich bereitgestellten Hilfestellung, sodass beobachtete Qualitätsunterschiede eindeutig auf die jeweilige Strategie und nicht auf abweichende Formulierungen zurückführbar sind. Die Zero-Shot-Vorlage beschränkt sich auf die reine Aufgabenanweisung und bildet damit die Referenzbasis. Die Few-Shot-Vorlage ergänzt ein vollständig ausformuliertes Ein- und Ausgabepaar. Dieses Beispiel bezieht sich bewusst auf eine andersartige Aufgabe auf einer separaten Beispieltabelle und nicht auf eine der neun evaluierten Aufgaben, damit es keine Musterlösung vorwegnimmt, sondern lediglich Stil und Format der erwarteten Lösung vermittelt [17, 21]. Die Chain-of-Thought-Vorlage leitet das Modell an, seine Lösung in nachvollziehbaren Zwischenschritten zu entwickeln [20, 21]. Da alle Platzhalter für alle Strategien identisch belegt sind, bleibt der Vergleich über die Strategien hinweg fair und kontrolliert.

Für die Direktverarbeitung existiert zu jeder der drei Strategien eine eigene Vorlage, die dieselbe Hilfestellung bietet, aber statt eines Ausgabepfads die Eingabedaten selbst aufnimmt und statt eines Skripts die Ergebnistabelle anfordert. Diese Spiegelung ist Voraussetzung dafür, den Vergleich der beiden Ausführungsmodi in Kapitel 8 unter gleichen Prompt-Bedingungen zu führen, statt einen ausgewählten Prompt des einen Modus gegen einen festen Prompt des anderen zu stellen.

6.6. Mess- und Auswertungsverfahren

Die in Abschnitt 4.5 definierten Kriterien werden durch eine automatisierte Bewertungsroutine operationalisiert, die die erzeugte Ausgabedatei mit der bekannten Soll-

Lösung abgleicht. Um zu berücksichtigen, dass programmatisch erzeugte Ergebnisse korrekte Werte häufig in abweichender Zeilenreihenfolge liefern, werden beide Tabellen vor dem Abgleich einheitlich sortiert und damit reihenfolgeunabhängig verglichen. Die Genauigkeit wird als Anteil der zellweise übereinstimmenden Werte bestimmt, die Vollständigkeit aus dem Vorhandensein der erwarteten Spalten und der korrekten Datensatzanzahl abgeleitet und die Konsistenz über die Übereinstimmung der Datentypen mit dem Zielschema erfasst [8, 11]. Entsprechend der in Abschnitt 4.5.1 getroffenen Unterscheidung sind beide Ausprägungen der Genauigkeit implementiert: Die strenge Variante bricht den Abgleich ab, sobald Spaltenmenge oder Datensatzanzahl abweichen. Die abgegliche Variante, die der Auswertung zugrunde liegt, verlangt lediglich ein gültiges Zielschema, ordnet die Tabellen über ihre Schlüssel beziehungsweise die sortierte Reihenfolge einander zu und verbucht überzählige oder fehlende Datensätze als Abweichung. Numerische Spalten werden dabei mit einer Toleranz verglichen, sodass Rundungsunterschiede nicht als inhaltlicher Fehler zählen, während Textspalten exakt abgeglichen werden. Fehlt die strukturelle Grundlage vollständig, weil eine erwartete Spalte gar nicht vorhanden ist, wird das Ergebnis als Schema-Bruch gekennzeichnet und getrennt von einem Programmabbruch ausgewiesen. Die Effizienzkennzahlen aus Zeit, Token-Verbrauch und Kosten stammen unmittelbar aus der standardisierten Modellantwort und werden ergänzend zur inhaltlichen Bewertung erfasst [18, 22].

Jeder einzelne Durchlauf wird als eigenständiger, unveränderlicher Ergebnisdatensatz abgelegt, der die vollständige Konfiguration, die erhobenen Qualitäts- und Effizienzkennzahlen, die exakte Modellkennung sowie den erzeugten Code umfasst. Diese lückenlose Protokollierung erlaubt es, jede spätere Auswertung ohne erneute Modellaufrufe aus den gespeicherten Datensätzen nachzuvollziehen, und bildet zugleich die Grundlage für die Erhebung der Reproduzierbarkeit, die sich aus der Streuung der Kennzahlen über die wiederholten Ausführungen einer Konfiguration ergibt [5, 6]. Eine gesonderte Auswertungsschicht führt die einzelnen Datensätze zusammen und klassifiziert die Ergebnisse nach ihrem Zustand, um technische Fehlschläge von inhaltlichen Abweichungen zu trennen. Da jeder Lauf mit einem Zeitstempel im Dateinamen abgelegt wird und ein wiederholter Aufruf derselben Konfiguration, etwa nach einem Abbruch der Gesamtausführung, somit einen zweiten Datensatz erzeugt, statt den ersten zu überschreiben, wird je Konfiguration einschließlich ihrer Wiederholungsnummer der jeweils jüngste Datensatz herangezogen. Es handelt sich dabei ausdrücklich nicht um eine Auswahl nach Ergebnisqualität: Unterschieden wird allein nach dem Zeitpunkt der Ausführung, sodass eine unvollständig abgebrochene Gesamtausführung keine veral-

teten Werte in die Auswertung trägt. Auf dieser Grundlage werden die in Kapitel 8 dargestellten Vergleiche, Übersichten und detaillierten Abweichungsanalysen erstellt. Damit ist die technische Umsetzung des Untersuchungsdesigns abgeschlossen, und die tatsächliche Durchführung der Experimente sowie die dabei erhobenen Messdaten sind Gegenstand des folgenden Kapitels 7.

7. Durchführung der Experimente

Nachdem Kapitel 6 die technische Umsetzung des Untersuchungssystems beschrieben hat, dokumentiert dieses Kapitel die eigentliche Durchführung der Experimente. Es legt offen, in welchem Ablauf die einzelnen Läufe erfolgten (Abschnitt 7.1), welche Messdaten dabei in welcher Form erhoben wurden (Abschnitt 7.2) und unter welchen Bedingungen die Durchführung stattfand, um die Nachvollziehbarkeit und Reproduzierbarkeit der Ergebnisse sicherzustellen (Abschnitt 7.3). Damit schafft es die Grundlage für die anschließende Auswertung in Kapitel 8.

7.1. Versuchsablauf

Ausgangspunkt jeder Durchführung ist der in Kapitel 5 beschriebene synthetische Datensatz sowie die zu jeder Aufgabe vorab bestimmte Soll-Lösung. Auf dieser gemeinsamen Grundlage werden die Experimente vollständig automatisiert über den in Abschnitt 6.1 dargestellten Experiment-Runner ausgeführt, sodass jede Konfiguration unter identischen Bedingungen und ohne manuelle Eingriffe durchläuft. Der Ablauf durchmustert die vollständige Faktormatrix systematisch, indem er für jede Kombination aus Modell, Prompting-Strategie und Aufgabe nacheinander die vorgesehenen Wiederholungen ausführt und das Ergebnis unmittelbar festhält [5, 8].

Vorauszuschicken ist eine Entscheidung, die den Umfang der ausgewerteten Datenbasis bestimmt. Die Erprobung der Untersuchungsanlage erfolgte auf einem ersten Datensatz: An ihm wurden die Prompts formuliert, der Aufgabenzuschnitt festgelegt und die Auswertungslogik entwickelt. Zwei der in Kapitel 8 berichteten Prompt-Bestandteile, die Angabe der Eingabe-Datentypen und die Anweisung zum Umgang mit akademischen Graden, sind unmittelbare Reaktionen auf dort beobachtete Fehler. Die an diesem Datensatz gemessenen Werte sind damit nicht unbefangen: Sie beschreiben die Leistung auf Daten, gegen die die Konfiguration justiert wurde, und überschätzen die auf unbekannten Daten zu erwartende Leistung in unbekanntem Umfang. Diese Ge-

fahr ist in der methodischen Literatur zur Bewertung von LLMs als zentrale Fehlerquelle beschrieben [5, 8]. Nach Abschluss sämtlicher Anpassungen wurde deshalb ein zweiter, bis dahin nicht verwendeter Datensatz mit dem Startwert 2 erzeugt und ohne jede weitere Änderung an Prompts, Aufgabenstellungen oder Auswertungscode durchlaufen. Er folgt derselben Erzeugungsvorschrift, enthält jedoch andere Namen, andere Adressen und eine andere Zuordnung der eingebauten Abweichungen. Für die Direktverarbeitung kommt zusätzlich seine halbierte Fassung mit dem Startwert 102 zum Einsatz, da die vollständige Fassung das Ausgabefenster der Modelle überschreitet. Sämtliche in Kapitel 8 berichteten Ergebnisse stammen ausschließlich aus diesen beiden Fassungen. Der erste Datensatz diene allein der Entwicklung und wird nicht ausgewertet.

Entsprechend den beiden in Abschnitt 4.1 eingeführten Ausführungsmodi und den dort festgelegten Untersuchungsebenen gliedert sich die Durchführung in mehrere Teilläufe. Im primären Modus der Code-Generierung wird für jede Konfiguration ein Verarbeitungsskript erzeugt, ausgeführt und bewertet. Aus vier Modellen, drei Prompting-Strategien, neun Aufgaben und je zwei Wiederholungen ergeben sich 216 Einzelläufe der Faktormatrix. Im ergänzenden Direkt-Modus löst jedes der drei kommerziellen Modelle dieselben neun Aufgaben einmalig unmittelbar auf den eingebetteten Daten, woraus sich 27 weitere Läufe ergeben. Zusätzlich löst die regelbasierte Vergleichspipeline dieselben neun Aufgaben in je einem deterministischen Durchlauf. Jeder einzelne Lauf wird direkt nach seiner Ausführung als eigenständiger Ergebnisdatensatz gespeichert, sodass der Gesamtablauf jederzeit unterbrochen und fortgesetzt werden kann, ohne dass bereits erhobene Ergebnisse verloren gehen oder überschrieben werden. Die auf den externen Modellzugriff zurückzuführenden, vorübergehenden Störungen werden dabei, wie in Abschnitt 6.4 beschrieben, durch eine automatische Wiederholung transienter Fehlaufrufe abgefangen, während inhaltlich fehlerhafte, aber technisch vollständige Ergebnisse bewusst als gültige Messwerte erhalten bleiben [5, 6].

Auf die Faktormatrix folgt die verkettete Ausführung, die dieselben Aufgaben als durchgängige Strecke aus neun aufeinander aufbauenden Schritten durchläuft. Sie wird je Modell fünfmal wiederholt und nach jedem Schritt unmittelbar gegen die zugehörige Soll-Zwischentabelle geprüft, sodass sich sowohl der Qualitätsverlauf über die Kette als auch dessen Streuung über die Wiederholungen bestimmen lässt. Da ein Schritt seine Nachfolger mit Daten versorgt, muss er strukturell verwertbar sein: Bricht der erzeugte Code ab oder verletzt seine Ausgabe das erwartete Schema, wird der Versuch bis zu dreimal wiederholt und die Kette andernfalls an dieser Stelle abgebrochen,

wobei die abhängigen Schritte als blockiert protokolliert werden. Diese Wiederholung dient allein der Herstellung einer ausführbaren Kette und ist nicht mit der zur Reproduzierbarkeit erhobenen Mehrfachausführung zu verwechseln. Ergänzend wird die Kette in einem zweiten, sonst identischen Durchlauf so ausgeführt, dass jeder Schritt die Soll-Ausgabe seines Vorgängers erhält. Beide Durchläufe zusammen erlauben in Kapitel 8 die Trennung von eigener Schrittleistung und geerbtem Fehler. Beide Formen werden auch für die regelbasierte Pipeline ausgeführt, sodass sich die Fehlerfortpflanzung zwischen KI-gestützter und deterministischer Verarbeitung vergleichen lässt. Im Direkt-Modus wird dieselbe Strecke auf der halbierten Datensatzfassung verkettet und isoliert je zweimal durchlaufen.

Zwei weitere Teilläufe untersuchen den Zuschnitt der Aufgabenstellung als eigenständige Einflussgröße. Im ersten werden die neun Pipeline-Schritte zu drei Gruppen von je drei Schritten zusammengefasst, die jeweils die Soll-Ausgabe ihrer Vorgängergruppe erhalten. Ausgeführt werden drei Modelle, drei Strategien, drei Gruppen und zwei Wiederholungen, also 54 Läufe. Im zweiten wird die zusammengesetzte Aufgabe aus Verknüpfung, Bereinigung und Aggregation erneut in einem einzigen Arbeitsschritt gestellt, diesmal jedoch mit sechs ausdrücklich benannten Teilschritten und einer Soll-Lösung, die diesen Schritten wörtlich folgt und aus den Eingabedaten allein erreichbar ist. Sie wird in beiden Ausführungsmodi, unter allen drei Strategien und in je zwei Wiederholungen gestellt, was 36 weitere Läufe ergibt. Insgesamt umfasst die ausgewertete Datenbasis damit 711 Modellaufrufe sowie 27 deterministische Durchläufe der regelbasierten Vergleichspipeline.

7.2. Erhobene Messdaten

Für jeden Lauf wird ein umfassender Ergebnisdatensatz erhoben, der eine spätere Auswertung ohne erneute Modellaufrufe ermöglicht. Er umfasst zum einen die vollständige Konfiguration des Laufs, also das eingesetzte Modell mit seiner exakten Versionskennung, die verwendete Prompting-Strategie, die bearbeitete Aufgabe mit Kategorie und Schwierigkeitsgrad, den zugrunde liegenden Zufallskern, die Nummer der Wiederholung sowie die gewählte Temperatur. Zum anderen enthält er die Ergebnisse der in Abschnitt 4.5 definierten Bewertung, namentlich die Genauigkeit, die Vollständigkeit und die Konsistenz sowie die zugehörigen strukturellen Kennzeichen wie die Übereinstimmung von Schema und Datensatzanzahl, anhand derer die grundsätzliche Vergleichbarkeit eines Ergebnisses beurteilt wird [8, 11].

Ergänzend zu den Qualitätskennzahlen werden für jeden Lauf die Effizienzgrößen aus Abschnitt 4.5.3 festgehalten, also der Verbrauch an Eingabe- und Ausgabe-Token, die daraus abgeleiteten Nutzungskosten sowie die Antwortzeit des Modells und, im Modus der Code-Generierung, die Dauer und der Erfolg der anschließenden Ausführung [18, 22]. Um über die Kennzahlen hinaus auch die konkrete Lösung nachvollziehbar zu machen, wird das jeweils erzeugte Artefakt gesichert, der vollständige Quelltext im Code-Generierungs-Modus, die zurückgegebene Ergebnistabelle im Direkt-Modus. Alle Läufe werden als unveränderliche Datensätze in einem einheitlichen, maschinenlesbaren Format abgelegt, die Ergebnistabellen im typerhaltenden Parquet-Format. Für die verkettete Ausführung kommt je Schritt hinzu, welche Eingaben er erhalten hat, wie viele Versuche bis zu einer verwertbaren Ausgabe nötig waren und wie lange Modellaufruf und Ausführung gedauert haben, sodass sich neben dem Qualitätsverlauf auch der Aufwand der gesamten Strecke beziffern lässt.

7.3. Reproduzierbarkeit und Versuchsbedingungen

Die Reproduzierbarkeit wurde bereits im Untersuchungsdesign als querschnittliche Dimension verankert (Abschnitt 4.5) und schlägt sich unmittelbar in den Versuchsbedingungen nieder. Um vergleichbare Bedingungen zu schaffen, werden alle steuerbaren Einflussgrößen über sämtliche Läufe hinweg konstant gehalten: Der Datensatz wird über einen festen Zufallskern reproduzierbar erzeugt und unverändert verwendet, und die Temperatur wird durchgehend auf einen niedrigen Wert festgelegt, um die nicht durch das Modell selbst bedingte Variabilität zu minimieren [5, 21]. Da sich das Feld der Modelle mit hoher Geschwindigkeit weiterentwickelt und die genaue Modellfassung bei den kommerziell betriebenen Anbietern nicht selbst kontrolliert werden kann, wird die exakte Versionskennung jedes Modells bei jedem Aufruf protokolliert. Im Rahmen dieser Arbeit kamen je ein aktueller Vertreter von Anthropic, OpenAI und Google sowie ein lokal ausgeführtes Modell der Llama-Reihe zum Einsatz [5, 18].

Um die probabilistische Variabilität der Modelle nicht nur zu kontrollieren, sondern selbst messbar zu machen, wird sie auf zwei Ebenen erhoben. In der Faktormatrix wird jede Konfiguration zweifach ausgeführt, was anzeigt, ob ein beobachteter Unterschied zwischen Modellen oder Strategien überhaupt Bestand hat. Die vollständige Pipeline wird darüber hinaus in beiden Formen je fünfmal durchlaufen, sodass sich die Streuung auf Ebene einer ganzen Verarbeitungsstrecke beziffern lässt, jener Ebene, auf der sich die in Abschnitt 4.1 als querschnittliche Dimension zu H1 und H2 eingeführte Reproduzierbarkeit praktisch auswirkt [5, 6]. Der ergänzende Direkt-Modus wird

demgegenüber mit weniger Wiederholungen ausgeführt, um den durch die eingebetteten Daten deutlich erhöhten Token-Verbrauch zu begrenzen: als Einzelaufgabe einfach, als Kette zweifach. Die dort erhobenen Werte sind entsprechend zurückhaltender zu lesen, tragen aber, wie Kapitel 8 zeigt, Unterschiede von mehreren Größenordnungen und damit weit jenseits der beobachteten Streuung. Zu berücksichtigen ist ferner, dass einige aktuelle Modelle bestimmte Sampling-Parameter nicht mehr akzeptieren oder standardmäßig einen internen Denkprozess ausführen. Solche anbieterspezifischen Abweichungen werden, wie in Abschnitt 6.4 beschrieben, abgefangen und im Ergebnisdatensatz vermerkt.

Sämtliche Läufe wurden auf demselben Rechner unter Windows 11 mit Python 3.12 ausgeführt. Für die Einordnung der Ergebnisse ist davon eine einzige Angabe erheblich: Die verwendete Grafikeinheit verfügt über 8 Gigabyte Video Random Access Memory (VRAM). Für die kommerziellen Modelle ist das ohne Belang, da dort lediglich Netzwerkaufrufe abgesetzt und die erzeugten Skripte lokal ausgeführt werden. Für das lokal betriebene Modell ist es die bestimmende Randbedingung. Der verfügbare Videospeicher begrenzt sowohl die Modellgröße, eingesetzt wurde die Acht-Milliarden-Parameter-Variante, als auch das Kontextfenster, das auf 8.192 Token festgelegt werden musste, da größere Zwischenspeicher die Verarbeitung auf die deutlich langsamere CPU zurückfallen lassen. Diese Begrenzung ist bei der Einordnung der Ergebnisse in Abschnitt 8.2 zu berücksichtigen. Eine bitgenaue Reproduktion bleibt trotz aller Vorkehrungen aufgrund der probabilistischen Natur der Modelle und möglicher serverseitiger Änderungen nicht garantierbar. Die protokollierten Versionskennungen, die konstanten Bedingungen und die vollständige Ablage aller Läufe sichern jedoch eine im Rahmen des Möglichen weitgehende Nachvollziehbarkeit [5, 6].

8. Ergebnisse und Auswertung

Die in Kapitel 7 beschriebene Durchführung hat eine geschlossene Datenbasis erzeugt, deren Auswertung Gegenstand dieses Kapitels ist. Es folgt der Struktur des Untersuchungsdesigns aus Kapitel 4: zunächst die Ergebnisse je ETL-Aufgabe (Abschnitt 8.1), dann der Vergleich der Modelle (Abschnitt 8.2) und der Prompting-Strategien (Abschnitt 8.3), darauf die verkettete Pipeline mit der Trennung von eigener Schrittleistung und Fehlerfortpflanzung (Abschnitt 8.4) und die querschnittliche Reproduzierbarkeit (Abschnitt 8.5). Die Abschnitte 8.6 und 8.7 behandeln Aufgabenzuschnitt und Ausführungsmodi als eigenständige Gestaltungsgrößen, Abschnitt 8.8 den Vergleich

mit der regelbasierten Pipeline, bevor Abschnitt 8.9 die Befunde an den Hypothesen spiegelt und in den Stand der Forschung einordnet.

Sämtliche berichteten Werte stammen ausschließlich aus dem Datensatz mit dem Startwert 2 beziehungsweise, wo die Direktverarbeitung betroffen ist, aus dessen halbierten Fassung mit dem Startwert 102. Diese Beschränkung ist eine bewusste methodische Entscheidung und in Abschnitt 7.1 begründet: Die Entwicklung der Prompts, des Aufgabenzuschnitts und der Auswertungslogik erfolgte an einem separaten Datensatz, dessen Werte damit nicht unbefangen sind. Der hier ausgewertete Datensatz war zum Zeitpunkt der Festlegung aller Einstellungen unbekannt und wurde anschließend ohne jede weitere Anpassung durchlaufen. Er trägt deshalb als einziger eine bestätigende Aussage [5, 8].

Sämtliche Genauigkeitswerte beziehen sich auf die in Abschnitt 4.5.1 definierte Genauigkeit in ihrer abgeglichenen Ausprägung: Stimmt das Zielschema, wird auch bei abweichender Datensatzanzahl der Anteil korrekt reproduzierter Werte gemessen, statt das Ergebnis pauschal als unbrauchbar zu verwerfen. Läufe, die überhaupt keine auswertbare Ausgabe erzeugen, abgebrochene Ausführungen ebenso wie Ergebnisse, in denen eine geforderte Spalte fehlt, gehen mit dem Wert null in die Mittelwerte ein, da sie im praktischen Einsatz kein verwertbares Ergebnis liefern. Ergänzend wird als Erfolgsquote der Anteil derjenigen Läufe ausgewiesen, die das Zielschema *und* die erwartete Datensatzanzahl einhalten. Beide Kennzahlen sind damit bewusst unterschiedlich streng: Eine Ausgabe, die das Zielschema erfüllt, aber von der erwarteten Zeilenzahl abweicht, zählt nicht zur Erfolgsquote, geht aber mit ihrem abgeglichenen Wert in die Genauigkeit ein. Die mittlere Genauigkeit kann die Erfolgsquote deshalb übersteigen. Der Abstand beider Größen zeigt an, wie viele Läufe strukturell danebenliegen, inhaltlich aber weitgehend richtig sind.

8.1. Ergebnisse je ETL-Aufgabe

Über alle 216 Läufe der Faktormatrix hinweg erreichen die vier Modelle eine mittlere Genauigkeit von 67,5 Prozent bei einer Erfolgsquote von 69,4 Prozent. Von den 216 Läufen führten 150 zu einer verwertbaren Ausgabe, 48 brachen mit einem Programmfehler ab und 18 lieferten eine lauffähige, aber strukturell abweichende Ausgabe. Der gesamte Aufwand belief sich auf 3,14 US-Dollar bei rund 103.000 Eingabe- und 277.000 Ausgabe-Token sowie einer mittleren Antwortzeit von 12,0 Sekunden je Aufruf. Diese Mittelwerte verdecken jedoch eine erhebliche Spreizung, die erst in der

Aufschlüsselung nach Aufgaben sichtbar wird und die zentrale Beobachtung dieser Arbeit bereits vorwegnimmt: Die Ergebnisqualität variiert zwischen den Aufgaben deutlich stärker als zwischen den Modellen.

Abbildung 8.1 zeigt diese Spreizung im Überblick. Aufgetragen ist je Aufgabe das beste erreichte Ergebnis der drei untersuchten Ansätze, sodass der Vergleich das jeweils Erreichbare und nicht den Durchschnitt einer beliebig herausgegriffenen Konfiguration abbildet. Drei Muster treten unmittelbar hervor: Die formal beschreibbaren Aufgaben werden von allen drei Ansätzen nahezu fehlerfrei gelöst, die beiden anspruchsvollen Entdoppelungsaufgaben trennen die Ansätze deutlich, und die zusammengesetzte Aufgabe aus Join, Bereinigung und Aggregation scheitert bei allen Ansätzen, einschließlich der regelbasierten Pipeline.

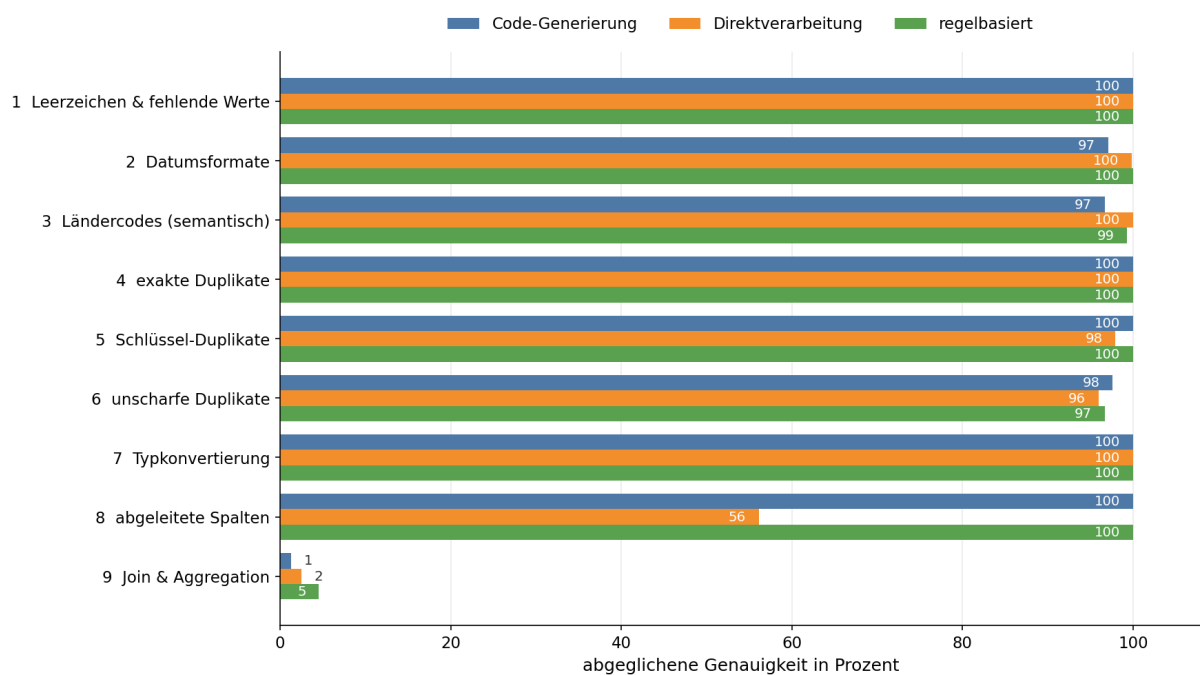


Abbildung 8.1.: Bestes je Aufgabe erreichtes Ergebnis der drei Ansätze. Für die Code-Generierung ist der beste Wert über Modelle, Strategien und Wiederholungen ausgewiesen, für die Direktverarbeitung der beste Wert über die Modelle. Deren Rückstand bei den abgeleiteten Spalten beruht auf einer am Ausgabelimit abgeschnittenen Antwort und nicht auf inhaltlichen Fehlern. Abschnitt 8.7 weist dies nach.

Tabelle 8.1 bezieht dieses Bild je Aufgabe. Ausgewiesen sind die über alle Modelle, Strategien und Wiederholungen gemittelte Genauigkeit, die zugehörige Erfolgsquote sowie zum Vergleich das Ergebnis der regelbasierten Pipeline. Die Reihenfolge folgt den drei Schwierigkeitsstufen aus Abschnitt 4.3, sodass sich der Einfluss der Aufgabenkomplexität unmittelbar ablesen lässt.

Tabelle 8.1.: Mittlere Genauigkeit und Erfolgsquote je Aufgabe über alle vier Modelle, drei Strategien und zwei Wiederholungen der Faktormatrix, gegenübergestellt der regelbasierten Pipeline. Die vorletzte Spalte weist zusätzlich das Mittel der drei kommerziellen Modelle aus.

Kategorie	Aufgabe	Genau.	Erfolg	Cloud	Regel
Bereinigung	Einfach – Leerzeichen und fehlende Werte	79,2 %	79,2 %	100,0 %	100,0 %
Bereinigung	Mittel – Datumsformate	85,9 %	91,7 %	96,7 %	100,0 %
Bereinigung	Schwer – ISO-Ländercodes	84,2 %	91,7 %	94,8 %	99,3 %
Duplikate	Einfach – exakte Duplikate	100,0 %	100,0 %	100,0 %	100,0 %
Duplikate	Mittel – Schlüssel-Duplikate	53,0 %	45,8 %	37,8 %	100,0 %
Duplikate	Schwer – unscharfe Duplikate	37,2 %	41,7 %	49,5 %	96,7 %
Transformation	Einfach – Typkonvertierung	85,0 %	91,7 %	95,4 %	100,0 %
Transformation	Mittel – abgeleitete Spalten	83,3 %	83,3 %	100,0 %	100,0 %
Transformation	Schwer – Join und Aggregation	0,1 %	0,0 %	0,1 %	4,5 %

Bemerkenswert ist zunächst, dass selbst die einfachsten Aufgaben im Mittel deutlich unter der Marke von 100 Prozent bleiben, obwohl jedes einzelne kommerzielle Modell sie mit gut gewähltem Prompt fehlerfrei löst. Der Vergleich der beiden mittleren Spalten der Tabelle zeigt, worauf diese Differenz zurückgeht: Bei sechs der neun Aufgaben liegt das Mittel der drei kommerziellen Modelle deutlich über dem Gesamtmittel, bei der einfachen Bereinigung etwa bei 100,0 gegenüber 79,2 Prozent. Der Unterschied entsteht durch das lokal betriebene Modell, dessen Einordnung Abschnitt 8.2 vornimmt. Die Mittelwerte der Spalte „Genau.“ sind daher nicht als Aussage über die erreichbare Qualität zu lesen, sondern als Aussage über die Verlässlichkeit einer beliebig herausgegriffenen Konfiguration: Sie messen, was zu erwarten ist, wenn Modell und Prompt nicht gezielt für die Aufgabe ausgewählt wurden.

Bei den beiden Entdoppelungsaufgaben kehrt sich dieses Verhältnis teilweise um: Die Schlüssel-Duplikate erreichen unter den kommerziellen Modellen mit 37,8 Prozent einen *niedrigeren* Wert als im Gesamtmittel. Auch dieser zunächst überraschende Befund löst sich in Abschnitt 8.2 auf und ist ein erster Hinweis darauf, dass die Rangfolge der Modelle nicht über alle Aufgaben hinweg stabil ist.

8.1.1. Datenbereinigung

Die Datenbereinigung ist die Kategorie mit der höchsten und zugleich stabilsten Ergebnisqualität. Das Entfernen überflüssiger Leerzeichen und die Kennzeichnung fehlender Werte lösen alle vier Modelle, einschließlich des lokalen, mit gut gewähltem

Prompt vollständig korrekt. Die Vereinheitlichung der vier Datumsformate erreicht bei den kommerziellen Modellen einheitlich 97,1 Prozent. Beide Aufgaben sind, wie in Abschnitt 5.3 beschrieben, durch eine endliche Menge syntaktischer Regeln vollständig beschreibbar, und genau in diesem Bereich verhält sich der erzeugte Code so zuverlässig wie eine manuell geschriebene Lösung [9, 12].

Gerade hier zeigt sich jedoch auch ein Risiko, das sich in keiner Erfolgsquote niederschlägt. Es wurde am Entwicklungsdatensatz beobachtet (vgl. Abschnitt 7.1) und floss in die Gestaltung der Prompts ein. Die folgende Darstellung ist deshalb qualitativer Natur und trägt keine der berichteten Kennzahlen. Skript 8.1 stellt zwei Lösungen derselben Aufgabe gegenüber, die beide fehlerfrei durchlaufen und eine schemakonforme Ausgabe erzeugen. Die erste Variante wählt die zu bereinigenden Spalten über deren Datentyp aus, ein in der Praxis verbreitetes und über Jahre korrektes Idiom. In der eingesetzten Pandas-Version werden Textspalten jedoch nicht mehr als generische Objektspalten, sondern als eigener Zeichenkettentyp geführt, sodass die Auswahl leer bleibt und die Bereinigung ohne Wirkung bleibt, ohne dass ein Fehler auftritt. Die zweite Variante benennt die betroffenen Spalten explizit und ist damit gegenüber dieser Änderung unempfindlich.

```
1 # still wirkungslos: die Spaltenauswahl liefert keine Treffer
2 for col in df.select_dtypes(include=['object']).columns:
3     df[col] = df[col].str.strip()
4
5 # wirksam: die betroffenen Spalten werden explizit benannt
6 for col in ["full_name", "email", "country", "registered_at"]:
7     df[col] = df[col].astype(str).str.strip()
```

Skript 8.1: Zwei lauffähige Lösungen derselben Bereinigungsaufgabe: Die obere Variante bleibt in der eingesetzten Pandas-Version wirkungslos, weil Textspalten nicht mehr als Objektspalten geführt werden. Die untere benennt die Spalten explizit.

Dieser Befund ist für die Beurteilung der Eignung von erheblicher Bedeutung: Der Fehler ist nicht inhaltlicher, sondern umgebungsbedingter Natur, er erzeugt weder eine Fehlermeldung noch eine Schemaverletzung und wäre ohne eine bekannte Soll-Lösung im laufenden Betrieb nicht ohne Weiteres zu bemerken. Er entsteht dort, wo das Modell auf ein in seinen Trainingsdaten dominantes, inzwischen jedoch überholtes Idiom zurückgreift, und tritt damit systematisch und nicht zufällig auf [5, 6]. Behoben wurde er im Rahmen der Prompt-Gestaltung nicht durch einen allgemeinen Hinweis auf Versionsunterschiede, sondern erst dadurch, dass dem Modell die Datentypen der Eingabespalten explizit mitgeteilt wurden.

Die semantische Vereinheitlichung der Länderangaben erreicht unter den kommerziellen Modellen 94,8 Prozent im Mittel und 96,7 Prozent im besten Lauf, bleibt damit aber hinter der regelbasierten Pipeline mit 99,3 Prozent zurück. Der Grund für diese Lücke lässt sich anhand der erzeugten Skripte exakt benennen: Alle drei Modelle lösen die Aufgabe, indem sie im Code eine Zuordnungstabelle aufbauen und die Rohwerte darüber nachschlagen. Damit ist die Lösung nicht durch das Weltwissen des Modells begrenzt, sondern durch das, was es beim Schreiben des Codes vorab aufzählt. Der Modus der Code-Generierung reduziert die semantische Aufgabe strukturell auf dieselbe Nachschlagelogik, die auch die regelbasierte Pipeline verwendet, und erreicht folgerichtig ein ähnliches, hier sogar geringfügig schlechteres Ergebnis. Der Direkt-Modus, in dem das Modell jeden Wert einzeln beurteilt, statt eine verallgemeinernde Vorschrift zu formulieren, erreicht bei derselben Aufgabe 97,1 bis 100 Prozent und übertrifft damit erstmals die regelbasierte Lösung [6, 23]. Abschnitt 8.7 greift diesen Modusunterschied systematisch auf.

8.1.2. Duplikaterkennung

Die Duplikaterkennung weist die stärkste Spreizung aller drei Kategorien auf. Exakte Duplikate werden von allen vier Modellen und in jeder Strategie fehlerfrei entfernt. Die Aufgabe ist mit einem einzigen Funktionsaufruf lösbar und trennt die Ansätze nicht. Die schlüsselbasierte Entdoppelung anhand der E-Mail-Adresse fällt dagegen auf 53,0 Prozent im Mittel, die unscharfen Duplikate auf 37,2 Prozent.

Bei näherer Betrachtung sind diese niedrigen Mittelwerte weniger auf ein Unvermögen der Modelle als auf die Zweiseitigkeit der Aufgabe zurückzuführen. Anders als bei der Bereinigung, bei der jede Zeile für sich betrachtet wird, verlangt die Entdoppelung eine Entscheidung über das Verhältnis von Datensätzen zueinander, bei der sowohl zu wenig als auch zu viel entfernt werden kann. Ein zu enger Ähnlichkeitsschwellenwert übersieht echte Duplikate, ein zu weiter führt unbeteiligte Datensätze zusammen. Am Entwicklungsdatensatz trat dieser zweite Fehlertyp in besonders klarer Form auf (vgl. Abschnitt 7.1): Werden akademische Grade und Anreden nicht als solche erkannt, sondern als Namensbestandteile behandelt, entstehen Vergleichsschlüssel, unter denen mehrere verschiedene Kunden zusammenfallen. Erst die explizite Anweisung, Titel nicht als Teil des Namens zu behandeln und Übereinstimmung sowohl im Namen als auch in der E-Mail-Adresse zu verlangen, löste dieses Problem auf.

Dass die Aufgabe grundsätzlich lösbar ist, belegen die Bestwerte: Bei den unscharfen

Duplikaten erreichen Claude 97,6 und Gemini 97,4 Prozent und liegen damit über der regelbasierten Pipeline mit 96,7 Prozent. Bei den Schlüssel-Duplikaten erreichen GPT und das lokale Modell 100 Prozent. Der Abstand zwischen Mittel und Bestwert ist in dieser Kategorie mit gut 60 Prozentpunkten der größte der Faktormatrix. Genau hier, und nur hier, entfaltet auch die Wahl der Prompting-Strategie eine messbare Wirkung, wie Abschnitt 8.3 zeigt.

8.1.3. Datentransformation

Typkonvertierung und abgeleitete Spalten fügen sich in das Bild der formal beschreibbaren Aufgaben ein: Beide werden von den kommerziellen Modellen mit gut gewähltem Prompt vollständig korrekt gelöst, die abgeleiteten Spalten sogar von allen drei Modellen in jeder Strategie und jeder Wiederholung.

Die zusammengesetzte Aufgabe aus Join, Bereinigung und Aggregation ist dagegen die einzige der neun Aufgaben, die von keinem Modell und in keiner Strategie gelöst wurde. Auch die regelbasierte Pipeline erreicht nur 4,5 Prozent. Ihre Analyse zeigt zwei wiederkehrende Fehlerbilder. Im ersten aggregiert das Ergebnis lediglich 478 der 2.000 Bestellungen, weil die Verknüpfung Bestellungen ohne passenden Kundendatensatz verwirft. Im zweiten werden zwar alle 2.000 Bestellungen erfasst, die Gruppierung erfolgt jedoch über 41 oder 42 verschiedene Rohschreibweisen der Länderangabe statt über die zehn ISO-Codes, sodass etwa DE, Deutschland, Germany, GER und Deutshcland als fünf getrennte Länder erscheinen. Da die bereinigte Länderangabe zugleich Gruppierungsschlüssel ist, wirkt sich jede unvollständige Bereinigung auf sämtliche aggregierten Werte aus, sodass kein einziger Kennzahlenwert korrekt bleibt.

Dieser Nullbefund verlangt jedoch eine Einordnung, die ihn wesentlich relativiert, und diese Einordnung ist selbst ein Ergebnis. Die Aufgabenstellung verlangt zwar, die Daten „so weit wie nötig“ zu bereinigen, benennt die erforderliche Vereinheitlichung der Länderangaben aber an keiner Stelle ausdrücklich. Sie ist lediglich implizit über das Zielschema angedeutet. Hinzu tritt ein Umstand der Soll-Lösung: In den Rohdaten fehlt bei 38 der 540 Kunden die Landangabe vollständig, und auf diese Kunden entfallen 144 der 2.000 Bestellungen. Die Soll-Lösung ordnet sie dennoch den zehn realen Ländern zu, weil ihr die Erzeugungsvorschrift des Datensatzes zugrunde liegt. Aus den Eingabedaten allein ist dieses Ergebnis folglich gar nicht erreichbar, und zwar für kein Verfahren, was auch erklärt, warum die regelbasierte Pipeline hier ebenfalls

scheitert.

Die Kennzahl misst an dieser Stelle somit weder die Fähigkeit zur Aggregation noch die zur Komposition mehrerer Schritte, sondern in erster Linie die Passung zwischen Aufgabentext und Soll-Lösung. Ein Nullwert, den alle Verfahren gleichermaßen erreichen, trennt nichts mehr und ist als Leistungsaussage wertlos. Abschnitt 8.6.2 löst diese Vermengung auf, indem er dieselbe Verarbeitung mit ausdrücklich benannten Teilschritten und einer aus den Eingabedaten tatsächlich erreichbaren Soll-Lösung erneut stellt. Die Modelle erreichen dort zwischen 54,5 und 75,8 Prozent. Der ursprüngliche Nullbefund ist damit weit überwiegend ein Artefakt der Aufgabengestaltung und nicht ein Befund über die Modelle, ein Ergebnis, das für die Gestaltung von Bewertungsverfahren erheblicher ist als für die Beurteilung der Modelle selbst [7, 8].

8.2. Vergleich der Modelle

Abbildung 8.2 fasst den Modellvergleich über die 216 Läufe zusammen. Unter den kommerziell betriebenen Modellen liegen die Ergebnisse eng beieinander: Das Modell von OpenAI erreicht mit einer Genauigkeit von 78,9 Prozent bei einer Erfolgsquote von 87,0 Prozent den höchsten Wert, gefolgt von Anthropic mit 74,5 und 72,2 Prozent sowie Google mit 71,3 und 72,2 Prozent. Die Abstände zwischen diesen drei Modellen sind mit rund acht Prozentpunkten deutlich geringer als die zuvor beobachteten Unterschiede zwischen den Aufgaben, die von 0 bis 100 Prozent reichen. Für die Frage der Eignung ist damit die Wahl der Aufgabe erheblich folgenreicher als die Wahl des Anbieters, ein Befund, der die in der Literatur verbreitete Fokussierung auf Modellranglisten relativiert [18, 19].

Dass diese Rangfolge nicht über alle Aufgaben trägt, zeigt der Blick auf die Einzelwerte. Bei den Schlüssel-Duplikaten erreicht GPT 100 Prozent, Claude 80,0 und Gemini 0,0 Prozent. Gemini erzeugt hier in beiden Wiederholungen und allen drei Strategien keine verwertbare Ausgabe. Bei den unscharfen Duplikaten kehrt sich das Verhältnis um: Dort liegen Claude mit 97,6 und Gemini mit 97,4 Prozent vor GPT mit 90,0 Prozent. Genau diese aufgabenweise Umkehrung ist die Ursache dafür, dass die kommerziellen Modelle bei den Schlüssel-Duplikaten in Tabelle 8.1 schlechter abschneiden als das Gesamtmittel. Eine über alle Aufgaben gemittelte Modellrangfolge verdeckt somit mehr, als sie zeigt.

Das untere Bild der Abbildung macht einen zweiten Unterschied sichtbar, der sich in

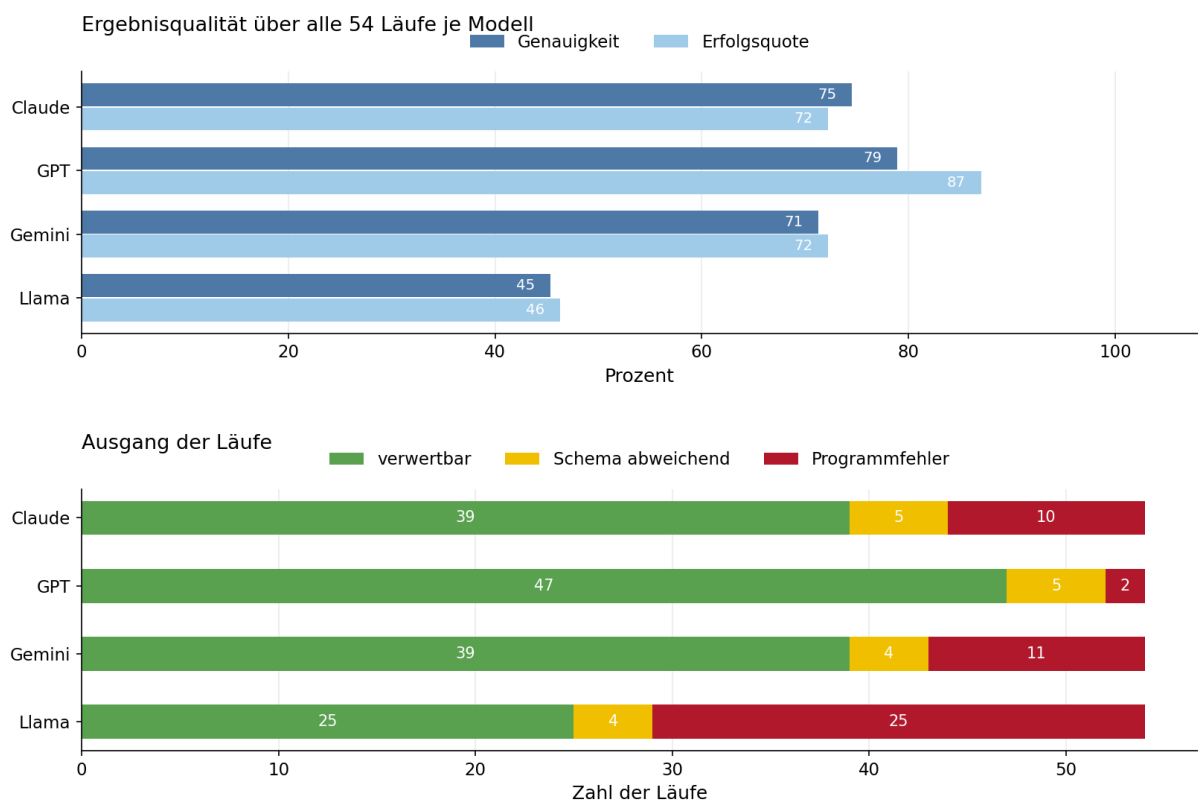


Abbildung 8.2.: Modellvergleich über die je 54 Läufe der Faktormatrix: oben Genauigkeit und Erfolgsquote, unten der Ausgang der Läufe nach verwertbarem Ergebnis, abweichendem Schema und Programmfehler.

der Genauigkeit nur teilweise niederschlägt. GPT brach lediglich 2 seiner 54 Läufe mit einem Programmfehler ab, Claude 10 und Gemini 11. Diese Trennung ist praktisch bedeutsam, weil sie einen qualitativen Unterschied markiert: Ein inhaltlich abweichendes Ergebnis lässt sich prüfen und korrigieren, ein nicht lauffähiges Skript hingegen erzwingt einen erneuten Aufruf und verschiebt das Problem lediglich [5, 6].

Deutlich abgesetzt ist das lokal betriebene Modell der Llama-Reihe mit einer Genauigkeit von 45,4 Prozent und einer Erfolgsquote von 46,3 Prozent. Von seinen 54 Läufen brachen 25 mit einem Programmfehler ab. Das lokale Modell scheitert damit weit überwiegend nicht an der inhaltlichen Aufgabe, sondern daran, überhaupt lauffähigen Code zu erzeugen. Dass es die Aufgaben durchaus versteht, belegen seine Bestwerte: Die einfache Bereinigung, die exakten Duplikate, die abgeleiteten Spalten und die Schlüssel-Duplikate löst es mit 100 Prozent und liegt bei letzteren damit über zwei der drei kommerziellen Modelle.

Diese Beobachtung erfordert eine Einordnung, die zugleich eine Entscheidung über den weiteren Aufbau dieses Kapitels begründet. Der gemessene Abstand ist keine Aussage über lokal betreibbare Modelle im Allgemeinen, sondern über dasjenige Modell, das sich auf der für diese Arbeit verfügbaren Hardware sinnvoll ausführen ließ. Die Größe eines lokal betriebenen Modells ist unmittelbar durch den verfügbaren Grafikspeicher begrenzt, und mit ihr die Fähigkeit, längere Anweisungen zu verarbeiten und syntaktisch fehlerfreien Code zu erzeugen. Die kommerziellen Modelle unterliegen dieser Beschränkung nicht, da die Ausführung beim Anbieter stattfindet. Ein Vergleich zwischen beiden misst somit nicht allein die Modellqualität, sondern zu einem erheblichen Teil die Ausstattung des ausführenden Rechners [5, 18].

Aus diesem Grund wird das lokale Modell in den folgenden Untersuchungsebenen, der verketteten Pipeline, dem Zuschnitt der Aufgabenstellung und dem Modusvergleich, nicht weiter geführt. Der Grund ist nicht sein schwächeres Abschneiden als solches, sondern dass seine Ausfälle die Mittelwerte in einer Weise verschieben, die sich nicht mehr der jeweils untersuchten Einflussgröße zurechnen lässt. Seine Ergebnisse aus der Faktormatrix bleiben in Abschnitt 8.1, im vorliegenden Abschnitt und im Strategievergleich vollständig ausgewiesen. Ausgeschlossen wird es nur dort, wo es eine andere Fragestellung überlagern würde.

Für die Praxis folgt daraus ausdrücklich keine Empfehlung gegen den lokalen Betrieb. Er ist die einzige der untersuchten Optionen, bei der keine Daten das eigene Netz

verlassen, und er verursacht keine nutzungsabhängigen Kosten, gegenüber 0,98 bis 1,10 US-Dollar je kommerziellem Modell für die Faktormatrix und 0,176 bis 0,216 US-Dollar je vollständigem Pipeline-Durchlauf. Dieser Vergleich erfasst allerdings nur eine Seite der Rechnung: Anschaffung und Betrieb der erforderlichen Hardware wurden hier nicht bilanziert, sodass sich aus den erhobenen Daten kein vollständiger Kostenvergleich, sondern allein die Aussage ableiten lässt, dass die laufenden Kosten mit steigendem Verarbeitungsvolumen zugunsten des lokalen Betriebs skalieren. Da die beobachtete Schwäche überwiegend in der Erzeugung lauffähigen Codes und nicht im inhaltlichen Verständnis der Aufgaben liegt, ist zu erwarten, dass ein auf leistungsfähigerer Hardware ausführbares, größeres Modell diesen Abstand deutlich verringert. Für Vorhaben mit entsprechender Ausstattung ist eine eigene Erprobung daher lohnend, und die in Abschnitt 9.2 genannten Bedingungen, kleinteiliger Aufgabenzuschnitt und mehrere zugestandene Versuche je Schritt, geben dafür den Rahmen vor [5, 18].

Die Kostenseite relativiert die Qualitätsunterschiede zusätzlich. Über die je 54 Läufe eines kommerziellen Modells liegen die Nutzungskosten zwischen 0,98 und 1,10 US-Dollar und damit in derselben Größenordnung. Die Antwortzeiten unterscheiden sich mit 10,8 bis 13,6 Sekunden je Aufruf ebenfalls nur moderat. Ein Modell allein aufgrund von Kosten oder Geschwindigkeit auszuwählen, ist auf dieser Grundlage nicht zu begründen. Die Auswahl folgt vielmehr der Zuverlässigkeit bei der konkret vorliegenden Aufgabenart.

8.3. Vergleich der Prompting-Strategien

Über alle vier Modelle und Aufgaben hinweg erzielt das Few-Shot-Prompting mit 70,6 Prozent die höchste Genauigkeit, gefolgt vom Chain-of-Thought-Prompting mit 69,0 und dem Zero-Shot-Prompting mit 63,0 Prozent. Beschränkt man die Auswertung auf die drei kommerziellen Modelle, bleibt die Reihenfolge erhalten, bei durchweg höherem Niveau: 77,9 gegenüber 75,1 und 71,8 Prozent. Auch beim lokalen Modell bleibt der Zero-Shot-Prompt mit 36,6 Prozent deutlich hinter den beiden aufwändigeren Strategien zurück, die mit 50,6 (Chain-of-Thought) und 48,9 Prozent (Few-Shot) nahezu gleichauf liegen. Der Befund geht damit nicht auf das Verhalten eines einzelnen Modells zurück, sondern zeigt sich über alle vier hinweg.

Der Aufwand wächst allerdings deutlich stärker als der Nutzen: Das Few-Shot-Prompting benötigt im Mittel 875 Eingabe-Token annähernd das Dreifache des Zero-Shot-Prompts mit 302 Token, um unter den kommerziellen Modellen einen Vorsprung von

6,1 Prozentpunkten zu erzielen. Für einen Dauerbetrieb, in dem die Aufgabenbeschreibung bei jedem Aufruf erneut übertragen wird, ist diese Rechnung bei den heutigen Preisen dennoch unkritisch, da der Prompt gegenüber der erzeugten Antwort kaum ins Gewicht fällt.

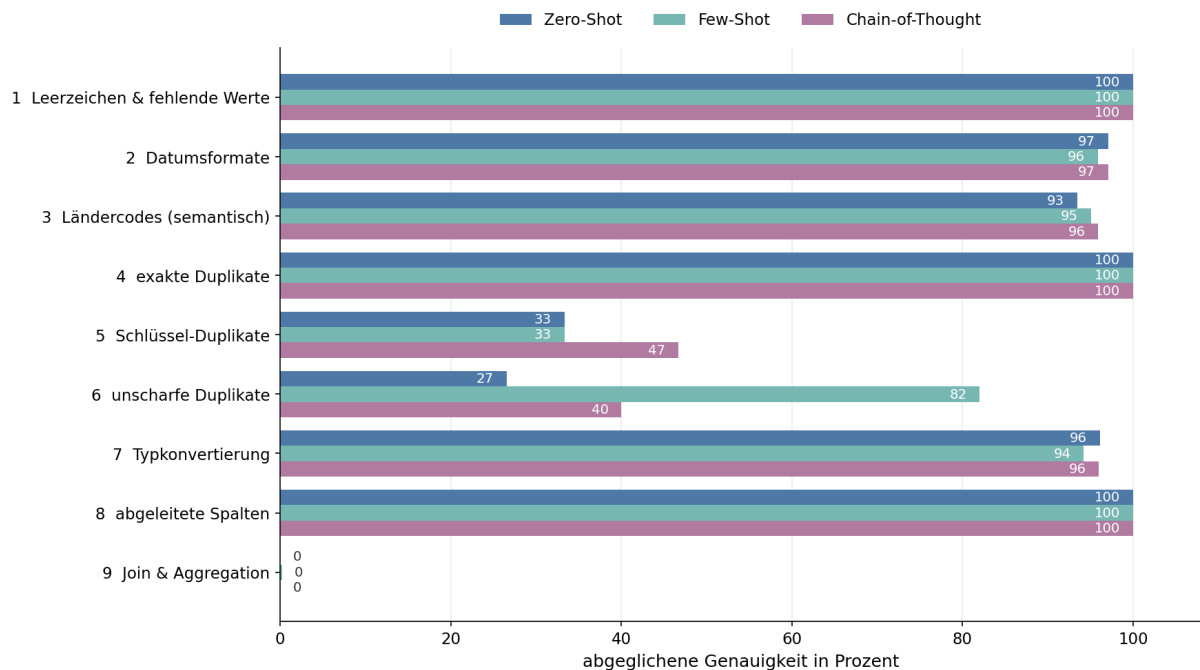


Abbildung 8.3.: Abgeglichene Genauigkeit der drei Prompting-Strategien je Aufgabe, gemittelt über die drei kommerziellen Modelle und beide Wiederholungen. Sieben der neun Aufgaben unterscheiden nicht zwischen den Strategien.

Aussagekräftig ist der Strategievergleich jedoch erst auf Aufgabenebene, und dort zeigt Abbildung 8.3 ein sehr klares Bild: Sieben der neun Aufgaben unterscheiden zwischen den Strategien praktisch nicht. Die drei Bereinigungsaufgaben, die exakten Duplikate, die Typkonvertierung und die abgeleiteten Spalten werden von jeder Strategie nahezu fehlerfrei gelöst, die zusammengesetzte Aufgabe von keiner. Der gesamte messbare Unterschied konzentriert sich auf die beiden anspruchsvollen Entdoppelungsaufgaben, und dort mit klarer Zuordnung: Bei den unscharfen Duplikaten führt das Few-Shot-Prompting mit 82,0 gegenüber 40,0 Prozent für Chain-of-Thought und 26,6 für Zero-Shot, bei den Schlüssel-Duplikaten hingegen das Chain-of-Thought-Prompting mit 46,7 gegenüber jeweils 33,3 Prozent. Beispiele wirken damit dort, wo eine Ähnlichkeitsentscheidung zu treffen ist, für die eine verbale Beschreibung schlecht taugt. Die Aufforderung zur schrittweisen Herleitung wirkt dort, wo eine mehrstufige Auswahlregel umzusetzen ist [17, 20].

Die über alle Aufgaben gemittelte Rangfolge ist damit kein eigenständiger Befund, son-

dern eine Folge dieser einen Aufgabe: Der Vorsprung des Few-Shot-Promptings von 6,1 Prozentpunkten unter den kommerziellen Modellen entsteht nahezu vollständig aus den 55 Prozentpunkten, die es bei den unscharfen Duplikaten gewinnt. Wer aus der Gesamtrangfolge die Regel ableitet, Beispiele seien generell hilfreich, überträgt eine Beobachtung aus einem Neuntel des Aufgabenraums auf das Ganze. Die praktisch verwertbare Aussage lautet stattdessen: Der Nutzen zusätzlicher Anleitung ist nicht allgemein, sondern an die Eigenart der Aufgabe gebunden. Wo eine Aufgabe in einem einzigen gedanklichen Schritt lösbar ist, stiftet sie keinen Mehrwert und verleitet zu unnötig komplexem und damit fehleranfälligerem Code. Wo eine Ermessensgrenze zu setzen oder eine mehrstufige Entscheidung zu treffen ist, wirkt sie [21, 22].

Wie stark diese Bindung an die Aufgabe tatsächlich ist, zeigt Abschnitt 8.6: Wird derselbe Verarbeitungsinhalt anders zugeschnitten, verschwindet der Strategieunterschied vollständig.

8.4. Ergebnisse der verketteten Pipeline

Die bisher dargestellten Ergebnisse betrachten jede Aufgabe für sich. Die in Abschnitt 4.1 eingeführte Untersuchungsebene der verketteten Ausführung prüft, wie sich dieses Bild verändert, wenn dieselben Aufgaben als durchgängige Strecke ausgeführt werden, in der jeder Schritt auf dem Ergebnis seines Vorgängers aufsetzt. Beide Durchläufe, der verkettete und der isolierte, wurden je Modell fünfmal wiederholt. Sämtliche Werte dieses Abschnitts sind Mittel über diese fünf Wiederholungen.

8.4.1. Qualitätsverlauf über die Kette

Das obere Bild in Abbildung 8.4 zeigt den Verlauf der neun Schritte für die drei Modelle und die regelbasierte Pipeline. Alle drei Modelle durchlaufen die Kette in jeder Wiederholung vollständig und erreichen in den Schritten 1 bis 8 durchweg Werte zwischen 96,0 und 100 Prozent. Über alle neun Schritte gemittelt erzielt Claude 96,1 Prozent, Gemini 94,9 und GPT 93,1, gegenüber 94,1 Prozent für die regelbasierte Vergleichspipeline. Der Aufwand einer vollständigen Strecke ist dabei gering: Er beträgt je Modell zwischen 0,176 und 0,216 US-Dollar sowie 102 bis 135 Sekunden, gegenüber 3,14 US-Dollar für die vollständige Faktormatrix. Die regelbasierte Pipeline benötigt für dieselbe Strecke 0,18 Sekunden und keine Nutzungskosten.

Der Endschrift bricht dagegen bei allen Teilnehmern ein: Claude erreicht 69,1 Prozent,

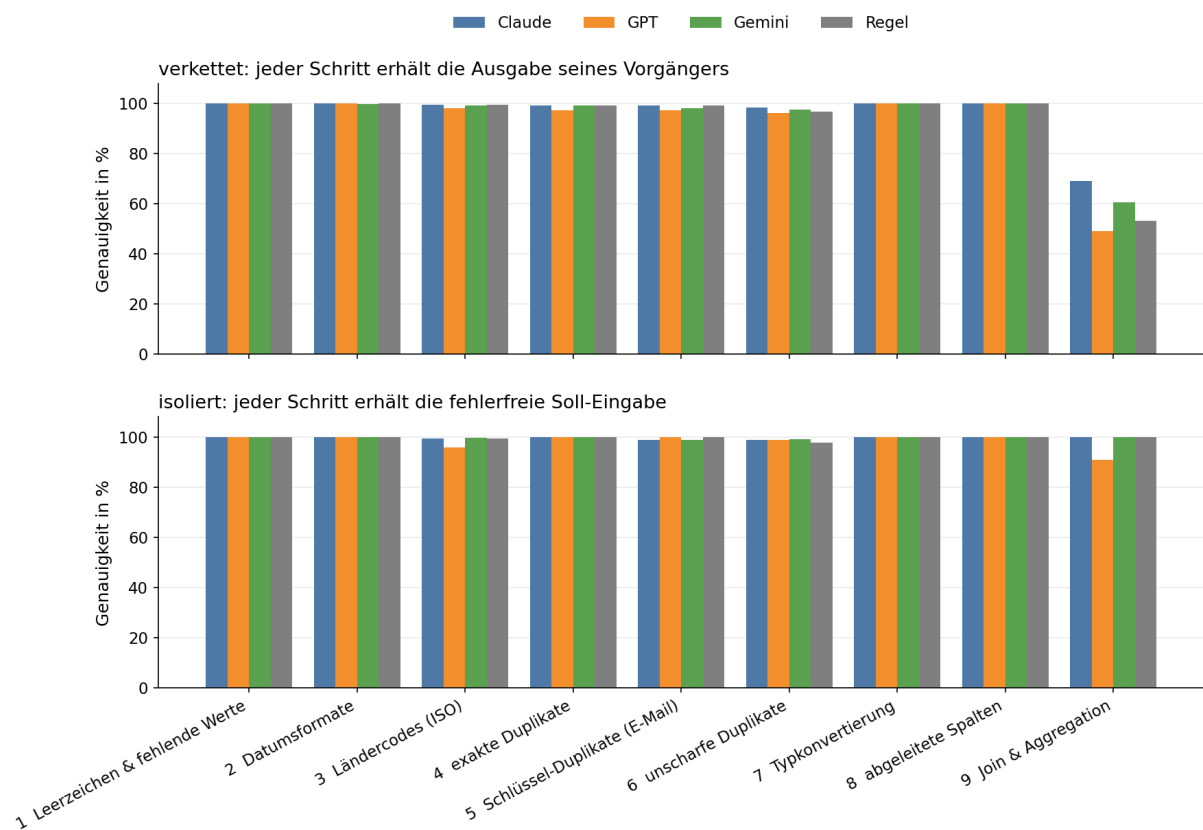


Abbildung 8.4.: Abgeglichene Genauigkeit je Schritt, gemittelt über fünf Wiederholungen. Oben die verkettete Ausführung, in der jeder Schritt die Ausgabe seines Vorgängers erhält. Unten die isolierte Ausführung mit fehlerfreier Soll-Eingabe je Schritt.

Gemini 60,6 und GPT 49,1, die regelbasierte Pipeline 53,0 Prozent. Bemerkenswert ist zunächst, dass diese Werte weit über dem Ergebnis derselben Aufgabe als Einzellauf liegen, wo sie bei 0,1 Prozent lag. Die Ursache ist dieselbe wie die in Abschnitt 8.1.3 beschriebene: In der Kette erhält der Endschrift bereits bereinigte Länderangaben und muss die semantische Vereinheitlichung nicht mehr selbst erbringen. Zugleich prüft ihn die Kettenreferenz gegen eine Soll-Lösung, die aus seinen tatsächlichen Eingabedaten erreichbar ist. Der verbleibende Verlust entsteht ausschließlich aus den Daten, die er verarbeitet: Es bleiben Bestellungen ohne zugehörigen Kundendatensatz zurück, weil in der vorgelagerten Entdoppelung jeweils der andere der beiden unscharfen Datensätze behalten wurde. Da die Aggregation über Land und Kategorie gruppiert, macht jede solche Bestellung zwei Gruppen in beiden Kennzahlen falsch, sodass wenige fehlplatzierte Datensätze einen überproportionalen Ausschlag erzeugen.

Der Vergleich beider Betrachtungsweisen zeigt bereits für sich genommen einen deutlichen Unterschied: Für dieselben neun Aufgaben liegt das Ergebnis in der Kette im Mittel weit über dem des unabhängigen Einzellaufs. In der Kette erhält jeder Schritt bereits vorbereitete Daten und eine eng umrissene Teilaufgabe, während er im Einzellauf zusätzlich alle vorgelagerten Mängel mitbehandeln muss. Die Zerlegung einer Strecke in kleine, klar abgegrenzte Schritte verbessert das Ergebnis somit unabhängig vom eingesetzten Modell, ein Befund, den Abschnitt 8.6 systematisch prüft und der unmittelbar auf die praktische Gestaltung KI-gestützter ETL-Strecken durchschlägt [1, 3].

8.4.2. Trennung von eigener Leistung und Fehlerfortpflanzung

Der bis hierhin gemessene Verlauf lässt offen, welcher Anteil eines schwachen Schrittergebnisses dem Schritt selbst und welcher den Vorgängerschritten zuzurechnen ist. Der isolierte Durchlauf, in dem jeder Schritt die Soll-Ausgabe seines Vorgängers erhält, beantwortet diese Frage. Das untere Bild in Abbildung 8.4 stellt ihn dem verketteten gegenüber.

Mit fehlerfreier Eingabe erreichen die Modelle über alle neun Schritte 99,7 (Claude), 99,7 (Gemini) und 98,4 Prozent (GPT). Den Endschrift lösen Claude und Gemini in jeder einzelnen Wiederholung vollständig korrekt, GPT erreicht konstant 90,9 Prozent. Der in der Kette gemessene Verlust im Endschrift von 30,9 (Claude), 39,4 (Gemini) und 41,8 Prozentpunkten (GPT) ist damit fast vollständig geerbt und nicht dem Endschrift anzulasten. Als eigene, nicht geerbte Schwäche verbleiben genau die drei semantisch

geprägten Schritte. Die Länder-Vereinheitlichung lösen die Modelle auch mit fehlerfreier Eingabe nur zu 99,3, 99,6 und 95,9 Prozent, und zwar aus dem in Abschnitt 8.1.1 belegten Grund der unvollständig aufgezählten Zuordnungstabelle. Die schlüsselbasierte und die unscharfe Entdoppelung bleiben mit 98,8 bis 99,0 Prozent knapp darunter. Alle übrigen sechs Schritte werden mit fehlerfreier Eingabe von jedem Modell vollständig korrekt gelöst. Die verbleibenden Verluste konzentrieren sich damit vollständig auf jene Schritte, die eine Ermessens- oder Bedeutungsentscheidung verlangen.

Den unmittelbarsten Vergleich liefert dabei die regelbasierte Pipeline, die für diese Arbeit ebenfalls in beiden Formen ausgeführt wurde. Ihre Werte sind allerdings asymmetrisch zu lesen: In den formal beschreibbaren Schritten verwendet sie dieselben Routinen, aus denen auch die Kettenreferenz komponiert ist, sodass ein Ergebnis von 100 Prozent dort definitionsgemäß eintritt und keine Leistung ausweist. Aussagekräftig sind die beiden Schritte mit begrenzten Heuristiken, die sie mit 99,3 und 97,6 Prozent löst, und vor allem der Endschrift: Auch die Regel löst ihn mit fehlerfreier Eingabe vollständig korrekt und fällt in der Kette auf 53,0 Prozent. Ihr Verlust von 47,0 Prozentpunkten übersteigt damit den jedes einzelnen Modells. Die Fehlerfortpflanzung ist folglich keine Eigenheit KI-gestützter Verarbeitung, sondern eine Eigenschaft der verketteten Architektur, die das deterministische Verfahren in gleichem, hier sogar stärkerem Maße trifft [2, 11].

Für die Praxis folgt daraus eine unmittelbar umsetzbare Konsequenz: In verketteten Strecken entscheidet die Qualität der frühen, semantisch geprägten Schritte über das Endergebnis, und ein Fehler wird umso teurer, je weiter vorn er entsteht. Eine Prüfung nach jedem Schritt ist deshalb wirksamer als eine Endkontrolle, weil sie einen Mangel dort sichtbar macht, wo er noch mit begrenztem Aufwand zu beheben ist, statt ihn erst in seiner verstärkten Form am Ende der Strecke zu entdecken [8, 11].

8.5. Reproduzierbarkeit der Ergebnisse

Die in Abschnitt 4.1 als querschnittliche Dimension verankerte Reproduzierbarkeit lässt sich auf zwei Ebenen bestimmen: über die zwei Wiederholungen jeder Konfiguration der Faktormatrix und über die fünf Wiederholungen der vollständigen Pipeline. Beide Ebenen liefern ein sehr unterschiedliches Bild, und gerade deren Gegenüberstellung ergibt den Einzelbefund mit der größten Tragweite für dieses Kapitel.

Auf Ebene der Einzelaufgabe wirken die Ergebnisse weitgehend stabil. Über alle Konfi-

gurationen der Faktormatrix beträgt die mittlere Standardabweichung 3,7 Prozentpunkte, und 77,8 Prozent der Konfigurationen liefern über beide Wiederholungen hinweg ein exakt identisches Ergebnis. Die niedrig gewählte Temperatur hat die Variabilität somit wirksam begrenzt, aber nicht beseitigt. Die Verteilung dieser Instabilität ist dabei nicht zufällig: Sie konzentriert sich auf jene Aufgaben, die auch inhaltlich am schwersten fallen. Die unscharfen Duplikate weisen mit 19,9 Prozentpunkten die mit Abstand höchste Streuung auf, gefolgt von den Schlüssel-Duplikaten mit 4,7. Die abgeleiteten Spalten und die exakten Duplikate sind dagegen vollständig stabil. Der Zusammenhang ist plausibel: Wo eine Aufgabe eine eindeutige, formal beschreibbare Lösung besitzt, konvergieren wiederholte Generierungen auf dieselbe Umsetzung. Wo hingegen eine Ermessensentscheidung wie die Wahl eines Ähnlichkeitsschwellenwerts zu treffen ist, schlägt sich die probabilistische Natur des Modells unmittelbar im Ergebnis nieder [5, 6].

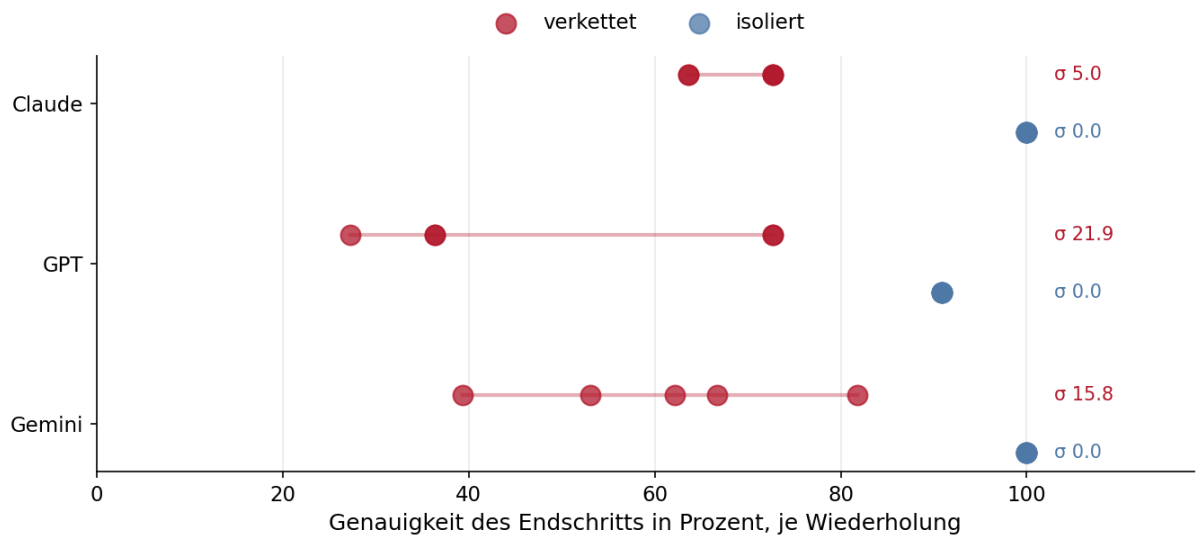


Abbildung 8.5.: Genauigkeit des Endschriffs in jeder der fünf Wiederholungen, verkettet und isoliert. Jeder Punkt ist ein vollständiger Durchlauf. Rechts ist die Standardabweichung in Prozentpunkten angegeben.

Auf Ebene der vollständigen Strecke ändert sich dieses Bild grundlegend, und Abbildung 8.5 zeigt worin. Der Gesamtwert der Strecke wirkt zunächst ebenfalls stabil: Die Standardabweichung über die fünf verketteten Wiederholungen beträgt 0,6 Prozentpunkte bei Claude, 2,0 bei Gemini und 3,2 bei GPT. Der Endschritt zeigt jedoch ein völlig anderes Verhalten. Dort schwankt das Ergebnis desselben Modells bei identischem Prompt und identischen Eingabedaten zwischen 27,3 und 72,7 Prozent bei GPT sowie zwischen 39,4 und 81,8 Prozent bei Gemini. Lediglich Claude bleibt mit 63,6 bis 72,7 Prozent in einem engeren Band. Eine einzelne Messung des Endergebnisses ist damit praktisch nicht aussagekräftig: Je nach Wiederholung ließe sich dasselbe Modell

als klar besser oder klar schlechter als die regelbasierte Pipeline darstellen.

Entscheidend ist der Vergleich mit dem isolierten Durchlauf, denn dort verschwindet diese Streuung vollständig. Alle drei Modelle liefern über sämtliche fünf Wiederholungen hinweg exakt denselben Endschrift-Wert, und die Standardabweichung des Gesamtwerts sinkt auf 0,08 bis 0,62 Prozentpunkte, bei Claude und Gemini um mehr als eine Größenordnung. Die beobachtete Instabilität entsteht folglich nicht aus der probabilistischen Natur der Textgenerierung selbst, sondern aus deren Verstärkung entlang der Kette: Eine kleine, für sich genommen belanglose Abweichung in der Wahl des Duplikat-Repräsentanten legt fest, welche Bestellungen im Endschrift einen Kundendatensatz finden, und schlägt dort um ein Vielfaches verstärkt durch. Die Reproduzierbarkeit einer KI-gestützten ETL-Strecke ist damit keine Eigenschaft des eingesetzten Modells, sondern eine Eigenschaft ihrer Architektur [5, 11].

Diese Kopplung hat zwei praktische Konsequenzen. Zum einen treten Instabilität und Fehleranfälligkeit gemeinsam auf: Gerade dort, wo ein Ergebnis besonders sorgfältig zu prüfen wäre, ist es zugleich am wenigsten verlässlich wiederholbar, sodass eine einmalige Abnahme eines Verarbeitungsschritts dessen künftiges Verhalten nicht garantiert. Zum anderen genügt es nicht, den erzeugten Code als festes Artefakt zu übernehmen, eine Konsequenz, die der Modus der Code-Generierung im Unterschied zur Direktverarbeitung überhaupt erst zulässt. Ebenso festzuschreiben sind die Zwischenergebnisse der frühen Schritte, da bereits deren Variation ausreicht, um das Endergebnis um über 45 Prozentpunkte zu verschieben, ohne dass sich an einer der beteiligten Vorschriften etwas geändert hätte.

8.6. Der Zuschnitt der Aufgabenstellung

Die bisherigen Abschnitte haben Aufgabe, Modell und Prompting-Strategie als Einflussgrößen behandelt. Dabei ist wiederholt eine vierte Größe sichtbar geworden: der Zuschnitt der Aufgabenstellung, also die Frage, wie viele Verarbeitungsentscheidungen in einem Arbeitsschritt zusammengefasst werden und wie ausdrücklich sie benannt sind. Als Untersuchungsebene ist sie in Tabelle 4.1 vorgesehen. Dieser Abschnitt wertet sie aus, in zwei Schritten und auf denselben Verarbeitungsinhalten. Der zweite dieser Schritte, die ausdrücklich gegliederte Fassung der zusammengesetzten Aufgabe in Abschnitt 8.6.2, wurde erst nach Sichtung der Ergebnisse aus Abschnitt 8.1.3 ergänzt und ist entsprechend als explorative Nachuntersuchung zu lesen, nicht als vorab geplanter Vergleich.

8.6.1. Gedrittelte Strecke

Die in Abschnitt 8.3 berichteten Strategieunterschiede unterliegen einer Einschränkung, die aus dem Aufbau der Faktormatrix folgt. Jede Aufgabe wird dort auf den unveränderten Rohdaten gestellt, sodass die anspruchsvollen Stufen zusätzlich alle vorgelagerten Mängel mitbehandeln müssen. Die Messung ist deshalb genau bei jenen Aufgaben unscharf, bei denen ein Strategieeffekt am ehesten zu erwarten wäre.

Um diese Blindstelle zu schließen, wurde eine Untersuchungsebene ergänzt, die zwischen der Einzelaufgabe und der vollständigen Neunkette liegt. Die neun Pipeline-Schritte werden zu drei Gruppen von je drei Schritten zusammengefasst: Bereinigung, Deduplizierung und Transformation. Jede Gruppe erhält die Soll-Ausgabe ihrer Vorgängergruppe als Eingabe. Die Anweisungen der drei zugehörigen Schritte werden dabei wörtlich aus der Pipeline übernommen und nummeriert in einem Prompt zusammengefasst, sodass die Gruppe inhaltlich dasselbe verlangt wie die drei Einzelschritte. Damit bleibt die Komposition mehrerer Verarbeitungsschritte als Anforderung erhalten, genau jene Eigenschaft, an der sich eine Strategie wie das Chain-of-Thought-Prompting bewähren sollte, ohne dass ein Fehler aus einer früheren Gruppe das Ergebnis überdeckt. Ausgeführt wurden 3 Strategien $\times$ 3 Modelle $\times$ 3 Gruppen $\times$ 2 Wiederholungen, also 54 Läufe.

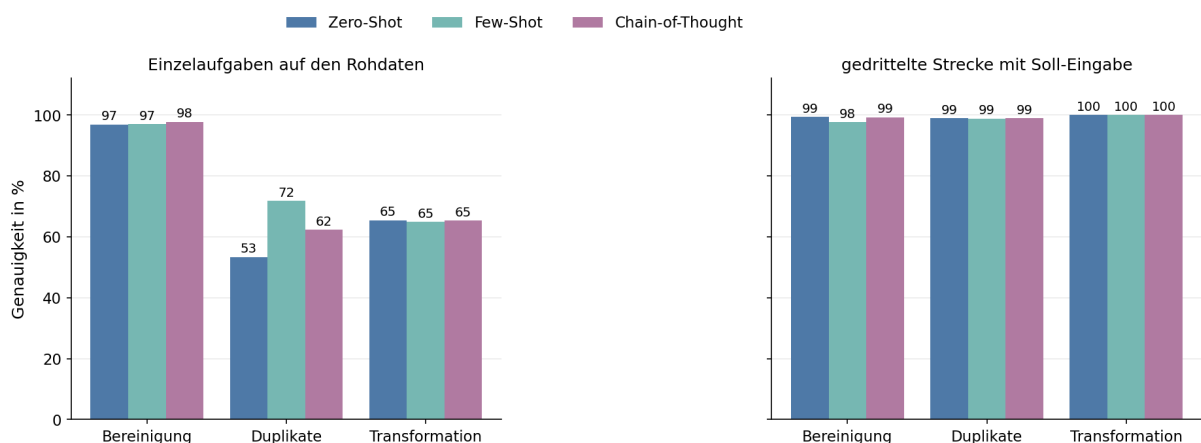


Abbildung 8.6.: Dieselben Verarbeitungsschritte als unabhängige Einzelaufgaben auf den Rohdaten (links) und als gedrittelte Strecke mit Soll-Eingabe je Gruppe (rechts), gemittelt über die drei kommerziellen Modelle. Der Strategieunterschied bei den Duplikaten verschwindet vollständig.

Sämtliche 54 Läufe lieferten ein verwertbares Ergebnis, und die Genauigkeit liegt durchgängig zwischen 95,3 und 100 Prozent. Über alle Gruppen und Wiederholungen gemittelt erreichen das Zero-Shot- und das Chain-of-Thought-Prompting jeweils 99,4 Pro-

zent und das Few-Shot-Prompting 98,8, eine Spanne von 0,6 gegenüber 6,1 Prozentpunkten, die dieselben drei Modelle in der Faktormatrix trennten. Auch zwischen den Modellen ist mit 99,0 bis 99,3 Prozent im Rahmen der erhobenen Wiederholungen kein Unterschied mehr erkennbar, und beide Wiederholungen liefern mit 99,3 und 99,1 Prozent nahezu denselben Gesamtwert. Besonders deutlich zeigt sich das an der Transformationsgruppe: Sie enthält als letzten Schritt genau jene Aggregation, die als Einzelaufgabe in allen Läufen scheiterte, und wird hier von jedem Modell, in jeder Strategie und in beiden Wiederholungen vollständig korrekt gelöst.

Abbildung 8.6 stellt beide Betrachtungsweisen unmittelbar gegenüber und macht damit die eigentliche Konsequenz sichtbar. Der auffälligste Strategieunterschied der Faktormatrix lag bei der Duplikaterkennung mit 53, 72 und 62 Prozent. Er verschwindet vollständig, sobald dieselben Schritte mit sauberer Eingabe und in einem klar abgegrenzten Zuschnitt gestellt werden. Was in der Matrix als Wirkung der Prompt-Gestaltung erscheint, ist demnach zu einem erheblichen Teil eine Wirkung des Aufgabenzuschnitts: Beispiele und schrittweise Herleitung helfen einem Modell, eine überladene Aufgabe zu bewältigen, sind aber entbehrlich, sobald die Aufgabe angemessen zugeschnitten ist. Für die Beantwortung von FF2 ist das ein Befund von eigenständigem Gewicht, denn er kehrt die übliche Reihenfolge der Gestaltungsentscheidungen um: Nicht der Prompt ist zuerst zu optimieren, sondern der Zuschnitt der Aufgabe, und ist dieser einmal richtig gewählt, verliert die Wahl der Prompting-Strategie ihre Bedeutung weitgehend [1, 21].

Der Aufwand dieser Ebene unterstreicht den Befund zusätzlich. Da jeder Gruppenlauf drei Verarbeitungsschritte umfasst, decken die 54 Läufe dieselben 162 Aufgabenausführungen ab wie die 162 Läufe der kommerziellen Modelle in der Faktormatrix, und kosteten dabei 2,50 gegenüber 3,14 US-Dollar. Vor allem aber lieferten sie ausnahmslos ein verwertbares Ergebnis, 49 davon im ersten Versuch, während in der Faktormatrix ein erheblicher Teil der Läufe ohne Ergebnis blieb. Der kleinteiligere Zuschnitt ist damit nicht nur genauer, sondern zugleich günstiger und zuverlässiger, eine Kombination, die in dieser Untersuchung sonst an keiner Stelle auftritt.

8.6.2. Explizite statt impliziter Anforderungen

Die gedrittelte Strecke verändert zwei Dinge gleichzeitig: Sie verkleinert den Zuschnitt *und* stellt fehlerfreie Eingabedaten bereit. Offen bleibt damit, welcher der beiden Eingriffe wirkt. Ein zweites Experiment trennt sie, indem es den Zuschnitt bewusst *nicht*

verkleinert: Gestellt wird erneut die vollständige zusammengesetzte Aufgabe aus Abschnitt 8.1.3 auf den unveränderten Rohdaten, in einem einzigen Arbeitsschritt, jedoch mit sechs ausdrücklich benannten und nummerierten Teilschritten anstelle der pauschalen Aufforderung, „so weit wie nötig“ zu bereinigen. Insbesondere wird die Vereinheitlichung der Länderangaben nun ausdrücklich verlangt, einschließlich der Vorgabe, nicht zuzuordnende Werte als UNKNOWN zu kennzeichnen. Die Soll-Lösung folgt diesen sechs Schritten wörtlich und ist damit, anders als jene der Faktormatrix, aus den Eingabedaten allein erreichbar. Die Aufgabe wurde in beiden Ausführungsmodi, unter allen drei Prompting-Strategien und in je zwei Wiederholungen gestellt, sodass der Modusvergleich nicht länger einen optimierten gegen einen festen Prompt stellt.

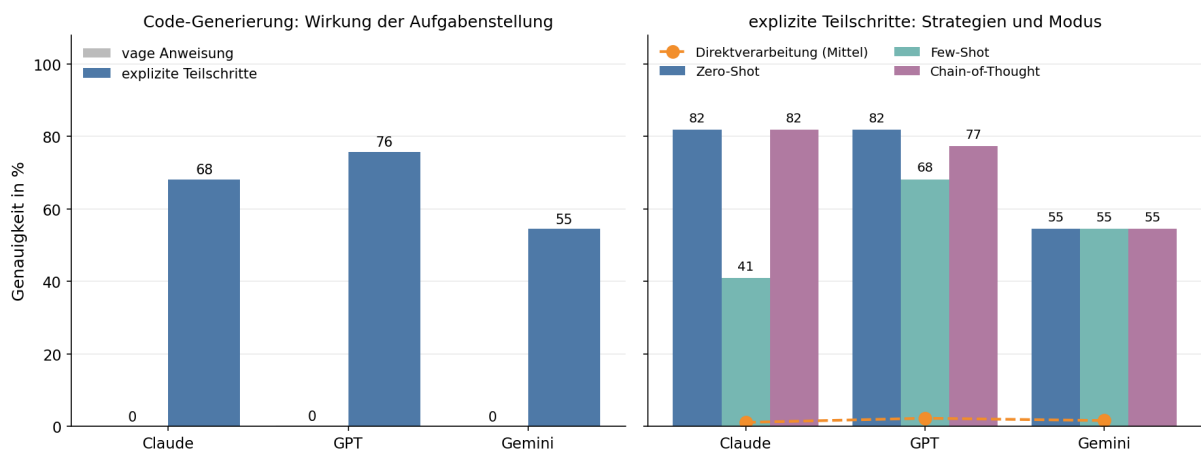


Abbildung 8.7.: Links dieselbe Verarbeitung mit vager und mit ausdrücklich benannter Anforderung im Code-Generierungs-Modus. Rechts das Ergebnis der expliziten Fassung nach Strategie sowie das Mittel der Direktverarbeitung.

Das Ergebnis ist der deutlichste Einzelbefund dieses Kapitels. Im Code-Generierungs-Modus steigt die Genauigkeit von 0,1 Prozent auf 75,8 Prozent bei GPT sowie 68,2 und 54,5 Prozent bei Claude und Gemini. Entscheidender als die Zahlen ist die Diagnose dahinter: In der ursprünglichen Fassung gruppierten die Modelle über 41 oder 42 Rohschreibweisen der Länderangabe, in der expliziten Fassung über die vorgesehenen elf Werte, in nahezu jedem Lauf, unabhängig von Modell und Strategie. Die Modelle können die Komposition mehrerer Verarbeitungsschritte also durchaus leisten. Sie hatten in der ursprünglichen Fassung schlicht eine Anforderung nicht erraten, die nie ausgesprochen wurde.

Was als Fehler verbleibt, ist der bereits aus Abschnitt 8.1.1 bekannte Mechanismus, nun jedoch ohne störende Überlagerung messbar. Der beste Lauf trifft 54 der 66 Ergebniszeilen auf den Cent genau und erfasst alle 2.000 Bestellungen. Die Gesamtsumme des Umsatzes stimmt exakt. Die zwölf abweichenden Zeilen gehen sämtlich

darauf zurück, dass die erzeugte Zuordnungstabelle das Kürzel `SUI` für die Schweiz nicht enthält, sodass 47 Bestellungen in die Gruppe `UNKNOWN` wandern. Der Fehler der Code-Generierung ist damit systematisch und summenerhaltend: Er verschiebt Datensätze zwischen Gruppen, ohne zu rechnen.

Die Direktverarbeitung liefert das exakte Gegenbild. Sie vereinheitlicht die Länderangaben in nahezu allen Läufen korrekt, liefert die richtige Zahl von Ergebniszeilen und trifft die Gesamtsumme der Bestellungen, und erreicht dennoch nur 1,2 bis 2,3 Prozent, in beiden Wiederholungen nahezu unverändert. Im besten Lauf ist keine einzige der 66 Umsatzzellen auf den Cent genau, die typische Abweichung beträgt rund 4.800 Euro bei Werten um 20.000 Euro, während die Randsummen näherungsweise stimmen. Das Modell erzeugt plausibel wirkende Zahlen, statt zu rechnen. Damit sind die beiden Fähigkeiten erstmals an derselben Aufgabe und unter demselben Prompt getrennt gemessen: Die semantische Zuordnung beherrscht die Direktverarbeitung besser, die Arithmetik über viele Datensätze beherrscht allein die Code-Generierung [6, 23].

Der Strategievergleich bestätigt hier das Bild aus Abschnitt 8.6.1: Zero-Shot und Chain-of-Thought liegen mit 72,7 und 71,2 Prozent gleichauf, das Few-Shot-Prompting kommt auf 54,5 Prozent. Dieser Abstand ist jedoch nicht der Strategie zuzurechnen, und der Vergleich der beiden Wiederholungen zeigt weshalb: Er entsteht vollständig aus einem einzigen Lauf, in dem der Few-Shot-Prompt bei Anthropic auch nach drei Versuchen keinen lauffähigen Code erzeugte, während dieselbe Konfiguration in der zweiten Wiederholung 81,8 Prozent erreichte. Klammert man diesen einen Ausfall aus, liegen alle drei Strategien zwischen 54,5 und 81,8 Prozent und damit innerhalb desselben Bandes.

Genau darin liegt der eigentliche Befund dieses Abschnitts zur Prompt-Gestaltung: Die Streuung zwischen zwei Wiederholungen derselben Konfiguration übersteigt hier den Unterschied zwischen den Strategien. Anthropic schwankte zwischen 54,5 und 81,8 Prozent, während Google in beiden Wiederholungen und allen drei Strategien konstant 54,5 Prozent lieferte. Wer aus einer einzelnen Messung eine Strategieempfehlung ableitet, misst unter diesen Bedingungen überwiegend Zufall, dieselbe Schlussfolgerung, die Abschnitt 8.5 für die verkettete Strecke gezogen hat, nun auf der Ebene einer Einzelaufgabe.

8.7. Vergleich der Ausführungsmodi

Die Direktverarbeitung wurde auf drei Ebenen erhoben: als Einzelaufgabe, als vollständige Kette und in der zusammengesetzten Aufgabe des vorangegangenen Abschnitts. Da die vollständigen Tabellen im Prompt mitgeführt werden müssen, kommt für die Kette die halbierte Fassung des Datensatzes mit dem Startwert 102 zum Einsatz. Die Werte sind entsprechend nicht zeilengleich mit denen der Code-Generierung, wohl aber strukturgleich.

Der erste und praktisch folgenreichste Befund betrifft nicht die inhaltliche Qualität, sondern das Ausgabefenster. Von den 27 Einzelaufgaben lieferten 7 eine unvollständige Tabelle: Bei den abgeleiteten Spalten gaben die Modelle 100, 718 und 1.123 der 2.000 erwarteten Zeilen zurück, bei den Schlüssel-Duplikaten 247, 503 und 511 von 520. Aufschlussreich ist, wie präzise sich dieser Abbruch in der Genauigkeit abbildet: Die abgeglichene Genauigkeit dieser sieben Läufe entspricht nahezu exakt dem gelieferten Zeilenanteil, 56,1 gegenüber 56,2 Prozent, 35,9 gegenüber 35,9 und 47,2 gegenüber 47,5 Prozent. Was geliefert wurde, war also inhaltlich korrekt. Verloren ging ausschließlich, was gar nicht mehr ausgegeben wurde.

Damit ist die bestimmende Grenze dieses Modus benannt, und sie ist keine der Ergebnisqualität: Sobald die Ergebnistabelle einige hundert Zeilen überschreitet, ist eine vollständige Antwort nicht mehr zuverlässig zu erhalten. Verschärfend wirkt, dass Modelle mit internem Denkprozess dessen Token dem Ausgabebudget zurechnen, sodass sich der Bedarf nicht aus der erwarteten Tabellengröße abschätzen lässt [18, 22].

Wo die Antwort vollständig ausfiel, ist die Qualität dagegen konkurrenzfähig. Die drei Bereinigungsaufgaben löst die Direktverarbeitung mit 96,0 bis 100 Prozent, die semantische Länder-Vereinheitlichung mit 97,1 bis 100 Prozent und damit als einziger Ansatz über der regelbasierten Pipeline. Bei den unscharfen Duplikaten erreicht sie 93,7 bis 96,0 Prozent und bei den Schlüssel-Duplikaten bis zu 97,9 Prozent, sofern die Antwort nicht abbrach. In der verketteten Ausführung erreicht sie in den Schritten 1 bis 8 zwischen 95,9 und 100 Prozent und liegt damit auf dem Niveau der Code-Generierung.

Der Endschritt bricht jedoch auf 2,7 bis 7,6 Prozent ein und damit weit unter jeden anderen Ansatz. Entscheidend ist, dass der isolierte Durchlauf hier nichts rettet: Mit fehlerfreier Eingabe erreicht der Endschritt lediglich 1,5 bis 4,5 Prozent. Anders als bei der Code-Generierung ist dieser Einbruch also gerade nicht geerbt, sondern eige-

nes Versagen, und die in Abschnitt 8.6.2 berichtete Zerlegung benennt seine Ursache genau: Das Modell muss die Aggregation über mehrere Dutzend Gruppen selbst rechnen, und diese arithmetische Zusammenfassung mehrerer hundert Einzelwerte gelingt ihm nicht.

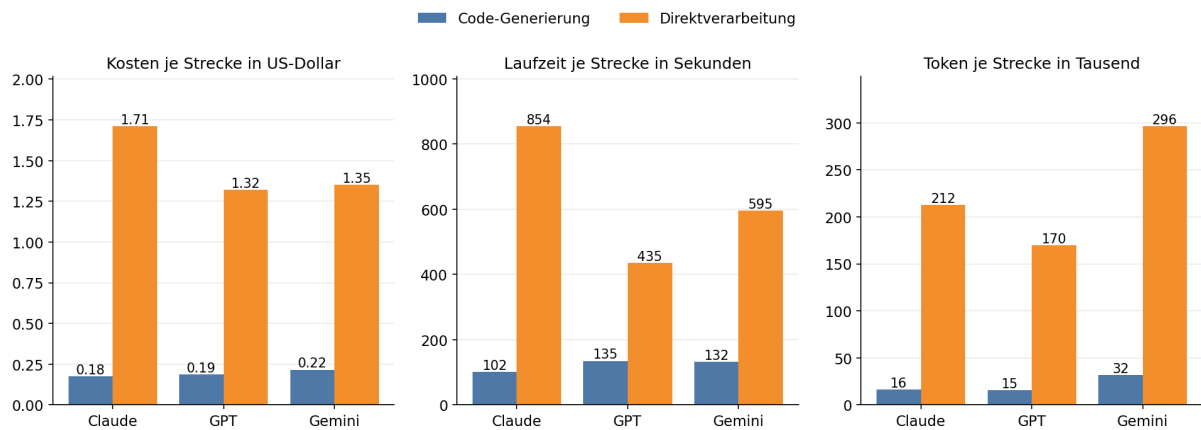


Abbildung 8.8.: Kosten, Laufzeit und Token-Verbrauch einer vollständigen Strecke aus neun Schritten je Ausführungsmodus, gemittelt über die Wiederholungen. Die Direktverarbeitung arbeitet dabei auf dem halbierten Datensatz.

Abbildung 8.8 beziffert schließlich den Aufwand. Eine vollständige Strecke im Code-Generierungs-Modus kostet zwischen 0,176 und 0,216 US-Dollar und benötigt zwischen 102 und 135 Sekunden. Die Direktverarbeitung liegt mit 1,32 bis 1,71 US-Dollar und 435 bis 854 Sekunden je nach Modell um das Sechs- bis Zehnfache beziehungsweise das Drei- bis Achtfache darüber, und das, obwohl sie nur den halbierten Datensatz verarbeitet. Auf die Zeile bezogen beträgt der Kostenunterschied damit rund das Fünfzehnfache. Der Grund liegt in der Struktur der Verarbeitung: Im Code-Generierungs-Modus durchläuft nur die Aufgabenbeschreibung das Modell, während die Daten lokal verarbeitet werden. Im Direkt-Modus wandert die vollständige Tabelle in jedem der neun Schritte einmal hinein und einmal wieder heraus. Entsprechend steigt der Token-Umsatz von rund 15.000 bis 32.000 auf 169.000 bis 296.000 je Strecke.

In der Zusammenschau ergibt sich ein klares Einsatzprofil. Als durchgängiger Verarbeitungsmodus für eine vollständige ETL-Strecke ist die Direktverarbeitung nicht zu begründen: Sie ist an das Ausgabefenster gebunden, arithmetisch unzuverlässig und um mehr als eine Größenordnung teurer je verarbeiteter Zeile. Sinnvoll einsetzbar ist sie allein als gezielt vorgeschalteter Einzelschritt für Teilaufgaben, die Einzelfallverständnis verlangen, auf überschaubaren Datenmengen arbeiten und keine Aggregation enthalten, eine Konstellation, die in dieser Untersuchung genau auf die semantische Vereinheitlichung zutrifft.

8.8. Vergleich KI-basiert vs. klassisch

Die regelbasierte Pipeline liegt bei sieben der neun Aufgaben vorn. Dieser Befund ist jedoch differenziert zu lesen. Für die sechs formal beschreibbaren Aufgaben ist er weitgehend definitorisch bedingt: Wie in Abschnitt 6.3 dargelegt, verwendet die regelbasierte Pipeline für diese Aufgaben dieselben deterministischen Routinen, die auch der Erzeugung der Soll-Lösung zugrunde liegen. Die explizit formulierte Regel *ist* dort die korrekte Lösung. Ein Ergebnis von 100 Prozent ist folglich kein Leistungsnachweis, sondern die zu erwartende Untergrenze für ein Verfahren, dessen Vorschrift mit der Aufgabendefinition zusammenfällt. Interessant ist an diesen Aufgaben nicht, dass die Regel gewinnt, sondern dass die Modelle mit gut gewähltem Prompt gleichauf liegen, ohne dass die Regel zuvor formuliert werden musste.

Aussagekräftig ist der Vergleich daher erst bei den drei Aufgaben, für die die regelbasierte Pipeline bewusst mit begrenzten Heuristiken ausgestattet wurde. Bei der Länder-Vereinheitlichung erreicht sie 99,3 Prozent: Ihre Zuordnungstabelle deckt ISO-Codes in zwei- und dreistelliger Form, IOC-Kürzel sowie deutsche und englische Ländernamen ab, lässt aber einzelne Schreibweisen offen. Genau hier liegt die Trennlinie zwischen beiden Verfahrensarten: Eine Zuordnungstabelle kann nur erfassen, was zuvor jemand aufgezählt hat, während ein Sprachmodell erschließt, dass „Holland“ umgangssprachlich für die Niederlande steht. Dass die Code-Generierung diesen Vorteil dennoch nicht ausspielt und mit 96,7 Prozent zurückbleibt, liegt daran, dass sie das Weltwissen des Modells in genau dieselbe aufzählende Form zwingt und dabei ebenfalls Lücken lässt, lediglich andere. Erst die Direktverarbeitung, in der jeder Wert einzeln beurteilt wird, übertrifft die regelbasierte Lösung mit bis zu 100 Prozent.

Bei den unscharfen Duplikaten kehrt sich das Verhältnis erstmals um: Die regelbasierte Pipeline erreicht 96,7 Prozent, Claude und Gemini erreichen im besten Lauf 97,6 und 97,4 Prozent. Der Abstand ist gering, aber er markiert den einzigen Punkt der Untersuchung, an dem die Code-Generierung die ausformulierte Regel schlägt, und zwar bei genau jener Aufgabe, deren Lösung sich am schlechtesten in einer Vorschrift fassen lässt. Zugleich macht diese Konstellation eine Grenze der Untersuchungsanlage sichtbar, die in Kapitel 10 aufzugreifen ist: Die eingebauten Abweichungen entstammen einer bekannten, endlichen Menge von Mustern, was regelbasierte Verfahren systematisch begünstigt. In realen Beständen, deren Abweichungsmuster nicht vorab bekannt sind, verschiebt sich dieses Verhältnis zugunsten der semantischen Verfahren [11, 12].

Der dritte aussagekräftige Fall ist die zusammengesetzte Aufgabe, bei der auch die regelbasierte Pipeline mit 4,5 Prozent nahezu vollständig scheitert, und zwar, wie in Abschnitt 8.1.3 dargelegt, weil die Soll-Lösung aus den Eingabedaten nicht erreichbar ist. Die Aufgabe trennt somit nicht zwischen den Ansätzen. In der verketteten Strecke hingegen lag die regelbasierte Pipeline mit 53,0 Prozent im Endschrift hinter Claude und Gemini, weil sie mehr Bestellungen ohne zugehörigen Kunden zurückließ, und verlor gegenüber ihrem eigenen isolierten Durchlauf mit 47,0 Prozentpunkten mehr als jedes Modell.

Zusammengefasst ergibt sich kein Verfahren, das über alle Aufgaben hinweg vorzuziehen wäre. Für formal beschreibbare Aufgaben ist die regelbasierte Umsetzung der KI-gestützten mindestens ebenbürtig und in Kosten, Geschwindigkeit und Determiniertheit deutlich überlegen, 0,18 Sekunden ohne Nutzungskosten gegenüber gut zwei Minuten und rund 0,19 US-Dollar für dieselbe Strecke. Ihr Vorteil endet jedoch genau dort, wo die Verarbeitungsvorschrift nicht mehr vollständig vorab formulierbar ist, und in diesem Bereich liegt der Beitrag der Modelle weniger im Ersetzen als im Ergänzen der Regel [12, 30].

8.9. Diskussion der Ergebnisse

8.9.1. Stärken, Schwächen und Risiken

Die deutlichste Stärke der untersuchten Modelle liegt nicht in der erreichten Genauigkeit, sondern im Aufwand, mit dem sie erreicht wird. Die hohe Ergebnisqualität der regelbasierten Vergleichspipeline ist das Resultat einer vorgelagerten Spezifikation, die sämtliche Verarbeitungsvorschriften einschließlich Zuordnungstabelle und Normalisierungsregel vorwegnimmt. Die Modelle erreichen bei den formal beschreibbaren Aufgaben vergleichbare Werte ohne diese Vorarbeit, in Sekunden und zu Kosten im Bereich weniger Cent. Hinzu tritt ein Vorteil bei den semantisch geprägten Aufgaben, allerdings in engerer Form als erwartet: Er tritt nur ein, wenn das Modell sein Wissen auf den einzelnen Wert anwenden kann, wie es die Direktverarbeitung zulässt. In generierten Code gefasst, muss dasselbe Wissen in eine aufzählende Tabelle oder einen Schwellenwert übersetzt werden und fällt damit auf die Ausdrucksmittel des regelbasierten Verfahrens zurück [6, 7].

Die praktisch größte Stärke ist jedoch die Zerlegbarkeit der Aufgabenstellung, wie sie Abbildung 8.6 zeigt: Bei kleinteilig gestellten Schritten steigt nicht nur das Ergebnis-

niveau, es verschwinden auch die Unterschiede zwischen Strategien und Modellen vollständig. Begrenzend wirkt demnach nicht die Schwierigkeit der einzelnen Operation, sondern die Zahl der Entscheidungen, die in einem Arbeitsschritt gleichzeitig zu treffen sind und die dabei unausgesprochen bleiben, beides Eigenschaften der Aufgabenformulierung und damit unmittelbar gestaltbar.

Dem stehen vier Risiken gegenüber. Das gravierendste ist das stille Versagen: Der in Skript 8.1 dokumentierte Fall zeigt einen Code, der fehlerfrei durchläuft, schema-konform ausgibt und dennoch nichts bewirkt. Weder Erfolgsquote noch Schema- oder Sichtprüfung hätten ihn aufgedeckt. Sichtbar wurde er allein im Abgleich mit einer Soll-Lösung, die im produktiven Einsatz gerade fehlt. Die Ursache ist ein in den Trainingsdaten dominantes, inzwischen überholtes Idiom und wirkt damit systematisch, sodass der Fehler reproduzierbar auftritt. Jede Produktivnutzung setzt folglich eine Ergebnisprüfung voraus, die nicht auf der Abwesenheit von Fehlermeldungen beruhen darf [6, 7].

Das zweite Risiko ist die Fehlerfortpflanzung, deren Ausmaß der isolierte Durchlauf beziffert: Bis zu 41,8 Prozentpunkte des im Endschrift gemessenen Verlusts sind geerbt. Entscheidend ist dabei nicht der Umfang, sondern die Art der verbliebenen Abweichung: Wenige typfremde Restwerte schaden mehr als viele gleichartige. Die Kennzahl eines einzelnen Schritts zeigt seine Verwendbarkeit im nachfolgenden daher nicht verlässlich an. Das dritte Risiko ist die begrenzte Reproduzierbarkeit, die sich ausgerechnet bei den schwierigsten Aufgaben am stärksten ausprägt und entlang der Kette auf über 45 Prozentpunkte verstärkt. Hinzu tritt die Abhängigkeit von serverseitig gepflegten Modellen, die sich durch die Protokollierung der Versionskennung dokumentieren, nicht aber beheben lässt [5, 18]. Viertens ist die Erwartung zu korrigieren, mehr Kontext im Prompt führe zu besseren Ergebnissen: Dieselbe Few-Shot-Strategie, die bei den unscharfen Duplikaten 55 Prozentpunkte gewinnt, bleibt in sieben von neun Aufgaben trotz dreifachen Token-Aufwands ohne Wirkung [21, 22].

8.9.2. Einordnung in den Stand der Forschung

Die Ergebnisse bestätigen die in Kapitel 3 referierte grundsätzliche Einschätzung, dass LLMs für Aufgaben der Datenbereinigung und -integration nutzbar sind, präzisieren sie jedoch in einem für die Praxis wesentlichen Punkt. Während die Literatur den Nutzen überwiegend konzeptionell oder anhand einzelner Anwendungsfälle beschreibt, zeigt der hier durchgeführte kontrollierte Vergleich, dass die Eignung nicht vom Modell,

sondern vom Zusammenspiel aus Aufgabentyp, Ausführungsmodus und Aufgabenzuschnitt abhängt [2, 6]. Die Spreizung zwischen den Aufgaben übersteigt jene zwischen den Modellen um ein Vielfaches, was der verbreiteten Orientierung an Modellranglisten eine begrenzte Aussagekraft für konkrete ETL-Vorhaben bescheinigt [18, 19].

Hypothese H1 gilt damit in ihrem Kern als bestätigt und in einem Teilaspekt als eingeschränkt bestätigt. Bestätigt ist die starke Abhängigkeit vom Aufgabentyp, die Ebenbürtigkeit bei formal beschreibbaren Aufgaben, der Verlust an Zuverlässigkeit mit zunehmender struktureller Komplexität sowie die Aussage, dass kein Ansatz über alle Aufgaben hinweg überlegen ist. Der erwartete Vorteil bei semantisch geprägten Aufgaben trat hingegen nur eingeschränkt ein: Bei den unscharfen Duplikaten übertrifft die Code-Generierung die Regel um knapp einen Prozentpunkt, bei der Länder-Vereinheitlichung bleibt sie hinter ihr zurück und wird erst von der Direktverarbeitung übertroffen. Der Vorteil ist damit real, aber an den Ausführungsmodus gebunden.

Genau diese Bindung ist Gegenstand von Hypothese H1a, und sie ist in beiden Teilen zu bestätigen. Der Ausführungsmodus entscheidet tatsächlich darüber, welche Fähigkeit eines Modells zur Wirkung kommt: Dieselbe zusammengesetzte Aufgabe, unter demselben Prompt gestellt, ergibt in der Code-Generierung bis zu 75,8 und in der Direktverarbeitung 1,2 bis 2,3 Prozent, wobei die Direktverarbeitung die semantische Zuordnung besser beherrscht und allein an der Arithmetik scheitert. Und der Zuschnitt wirkt stärker als die Modellwahl: Zwischen den Modellen liegen in der Faktormatrix acht Prozentpunkte, zwischen zwei Zuschnitten derselben Verarbeitungsinhalte dagegen mehr als vierzig. Beides findet in der Literatur bislang kaum Beachtung, weil dort weder der Ausführungsmodus noch der Aufgabenzuschnitt als eigene Gestaltungsdimension behandelt werden [1, 2].

Hypothese H2 ist in ihrer allgemeinen Form zu verwerfen, wenn auch aus einem anderen Grund als erwartet. Aufwändigere Prompting-Strategien verbessern die Ergebnisqualität im Mittel zwar messbar: Unter den kommerziellen Modellen liegt Few-Shot 6,1 Prozentpunkte vor Zero-Shot. Dieser Vorsprung entsteht jedoch nahezu vollständig aus einer einzigen der neun Aufgaben, während der dreifache Token-Aufwand in sieben Aufgaben ohne jede Wirkung bleibt. Ein über alle Aufgaben gemittelter Strategievergleich beschreibt damit keinen allgemeinen Zusammenhang, sondern verallgemeinert einen Einzelfall. Der erwartete abnehmende Grenznutzen trifft zu, in verschärfter Form: Er ist nicht nur abnehmend, sondern bei unpassendem Zuschnitt vernachlässigbar. Vor allem aber zeigt Abschnitt 8.6, dass der gesamte messbare Strategieeff-

fekt verschwindet, sobald die Aufgabe angemessen zugeschnitten ist. Die Prompting-Strategie ist damit der schwächere von zwei Hebeln, und der stärkere ist der Zuschnitt [17, 20].

Über die beiden Hypothesen hinaus liefert die verkettete Betrachtung einen Beitrag, der in der Literatur bislang wenig ausgeprägt ist. Die getrennte Messung von eigener Schrittleistung und geerbtem Fehler zeigt, dass ein erheblicher Teil der in Einzelbetrachtungen berichteten Schwächen gar nicht dem bewerteten Schritt zuzurechnen ist: Der Endschrift der Kette, der in der verketteten Ausführung zwischen 49,1 und 69,1 Prozent erreicht, ist mit fehlerfreier Eingabe von zwei der drei Modelle vollständig lösbar. Evaluierungen, die einzelne ETL-Operationen isoliert betrachten, überschätzen die Leistungsfähigkeit in Bezug auf reale Strecken daher ebenso, wie rein verkettete Betrachtungen sie unterschätzen. Erst die Gegenüberstellung beider Betrachtungsweisen erlaubt eine belastbare Zuordnung [2, 8].

Ein zweiter methodischer Beitrag ergibt sich aus der Reproduzierbarkeitsmessung auf Ebene ganzer Strecken. Der Endschrift derselben Konfiguration schwankte über fünf identische Wiederholungen um bis zu 45 Prozentpunkte, während er im isolierten Durchlauf in jeder Wiederholung exakt denselben Wert lieferte. Vergleichende Bewertungen von LLMs in ETL-Strecken, die auf einem einzelnen Durchlauf beruhen, wie es in der referierten Literatur überwiegend der Fall ist, messen unter diesen Bedingungen zu einem erheblichen Teil Zufall statt Leistungsfähigkeit [8, 18].

Ein dritter Beitrag betrifft die Gestaltung von Bewertungsaufgaben selbst. Die zusammengesetzte Aufgabe der Faktormatrix ergab für sämtliche Verfahren einschließlich des regelbasierten einen Wert nahe null und wirkte damit wie ein starker Negativbefund über die Kompositionsfähigkeit von LLMs. Die gezielte Wiederholung derselben Verarbeitung mit ausdrücklich benannten Teilschritten und einer aus den Eingabedaten erreichbaren Soll-Lösung hob dieses Ergebnis auf bis zu 75,8 Prozent. Ein Nullbefund, den alle Verfahren gleichermaßen erreichen, ist folglich zunächst als Hinweis auf die Aufgabengestaltung und erst danach als Aussage über die geprüften Verfahren zu lesen, eine Vorsichtsregel, die für die wachsende Zahl von Vergleichsstudien in diesem Feld unmittelbar praktisch ist [7, 8].

Auf dieser Grundlage lassen sich im folgenden Kapitel 9 konkrete Empfehlungen für die Auswahl zwischen klassischem, KI-gestütztem und hybridem Vorgehen je Aufgabentyp ableiten, wie sie FF1 einfordert, ebenso wie Empfehlungen zur Gestaltung der Prompts

im Sinne von FF2.

9. Handlungsempfehlungen für die Praxis

Die in Kapitel 8 dargestellten Befunde erlauben Empfehlungen, die über die Beantwortung der Forschungsfragen hinaus unmittelbar auf die Gestaltung KI-gestützter ETL-Strecken anwendbar sind. Sie werden entlang dreier Entscheidungen entwickelt, die in einem Vorhaben nacheinander zu treffen sind: ob ein LLM für eine gegebene Aufgabe überhaupt das geeignete Mittel ist (Abschnitt 9.1), wie Modell, Ausführungsmodus und Prompt zu wählen sind (Abschnitt 9.2) und wie sich beide Verfahrensarten in einer Gesamtarchitektur verbinden lassen (Abschnitt 9.3). Für alle gilt vorab dieselbe Einschränkung: Sie beruhen auf einem synthetischen Datensatz mit endlich vielen bekannten Abweichungsmustern und auf vier Modellen zu einem bestimmten Zeitpunkt. Übertragbar sind daher nicht die konkreten Zahlenwerte, sondern die Wirkungszusammenhänge, aus denen sie entstehen, sowie die Verfahren, mit denen sich die Werte im eigenen Kontext bestimmen lassen.

9.1. Eignung von LLMs für unterschiedliche ETL-Szenarien

Der über alle Untersuchungsebenen hinweg stabilste Befund dieser Arbeit ist, dass die Eignung eines LLM weit stärker von der Art der Aufgabe abhängt als von der Wahl des Modells. Zwischen dem besten und dem schwächsten Cloud-Modell liegen rund acht Prozentpunkte, zwischen der am besten und der am schlechtesten gelösten Aufgabe hingegen nahezu hundert. Die erste Frage in einem ETL-Vorhaben lautet daher nicht, welches Modell einzusetzen ist, sondern für welche Teilaufgaben sich der Einsatz überhaupt lohnt [5, 18].

Für diese Entscheidung hat sich in den Ergebnissen eine einzige Unterscheidung als tragfähig erwiesen: ob die Verarbeitungsvorschrift vollständig vorab formulierbar ist oder nicht. Aufgaben der ersten Art – das Entfernen von Leerzeichen, das Vereinheitlichen bekannter Datumsformate, das Entfernen exakter Duplikate, Typkonvertierungen, das Berechnen abgeleiteter Spalten – werden von einer regelbasierten Implementie-

nung definitionsgemäß korrekt gelöst, deterministisch, in Sekundenbruchteilen und ohne laufende Kosten. Ein LLM erreicht hier bestenfalls dasselbe Ergebnis und bringt zusätzlich Nichtdeterminismus, Kosten und Prüfaufwand mit sich. Sein Beitrag liegt bei diesen Aufgaben ausschließlich in der *Erstellung* der Regel, nicht in ihrer Ausführung, ein Unterschied, der in der Diskussion über KI-gestützte Datenverarbeitung häufig verwischt wird [12, 35].

Aufgaben der zweiten Art sind dadurch gekennzeichnet, dass die Vorschrift eine Aufzählung erfordern würde, die niemand vollständig angeben kann. Die Vereinheitlichung der Länderangaben ist der klarste Fall: Eine Zuordnungstabelle erfasst nur, was zuvor jemand eingetragen hat. Die regelbasierte Vergleichspipeline scheiterte an genau drei Schreibweisen, nämlich „Holland“, „England“ und dem Tippfehler `Deutschcland`. Genau hier liegt der originäre Beitrag des Modells, das erschließt, dass „Holland“ umgangssprachlich für die Niederlande steht, ohne dass diese Gleichsetzung je hinterlegt wurde. Dasselbe gilt für die unscharfe Duplikaterkennung, bei der die Entscheidung über zwei konkrete Datensätze einem Modell unmittelbar zugänglich ist, während ihre Verallgemeinerung zu einem Schwellenwert Information vernichtet.

Diese Zuordnung bleibt jedoch unvollständig, solange sie den Ausführungsmodus außer Acht lässt, denn der Vorteil bei semantischen Aufgaben tritt nur dann ein, wenn das Modell sein Wissen auf den einzelnen Wert anwenden kann. In generierten Code gefasst, fiel dieselbe Aufgabe auf das Niveau der regelbasierten Lösung zurück, weil das Modell gezwungen war, sein Wissen in eine ebenfalls unvollständige Aufzählung zu übersetzen. Die Eignungsfrage ist deshalb nicht mit „LLM oder Regel“ zu beantworten, sondern erst mit der Angabe, in welchem Modus das Modell arbeiten soll. Tabelle 9.1 fasst die daraus folgende Zuordnung zusammen.

Die letzte Zeile dieser Tabelle verdient besondere Beachtung, weil sie den folgenreichsten Gestaltungshebel benennt. Die zusammengesetzte Aufgabe aus Join, Bereinigung und Aggregation wurde als Einzelaufgabe von keinem Modell und in keiner Strategie gelöst. Dieselben Modelle lösten sie in zerlegter Form mit fehlerfreier Eingabe vollständig korrekt und in unzerlegter, aber ausdrücklich in Teilschritte gegliederter Form zu bis zu drei Vierteln. Nicht die Schwierigkeit der einzelnen Operation begrenzt die Leistungsfähigkeit, sondern die Zahl der Entscheidungen, die gleichzeitig zu treffen sind, und die Zahl derer, die unausgesprochen bleiben. Beide lassen sich durch die Formulierung der Aufgabe unmittelbar beeinflussen. Bevor ein Vorhaben ein Modell auswählt oder einen Prompt optimiert, sollte es daher seine Aufgabenstellung

Tabelle 9.1.: Empfohlenes Verfahren je Aufgabentyp auf Grundlage der Ergebnisse aus Kapitel 8.

Aufgabentyp	Merkmal	Empfehlung
Syntaktisch regelbar	Vorschrift vollständig aufzählbar	Regelbasiert ausführen, LLM allenfalls zur Erstellung des Codes
Semantisch, wertweise	Entscheidung je Einzelwert, Verallgemeinerung verlustbehaftet	LLM im Direkt-Modus, gezielt als einzelner Schritt
Rechnerisch aggregierend	viele Datensätze, exakte Arithmetik	Regelbasiert oder LLM im Code-Generierungs-Modus, Direkt-Modus ungeeignet
Zusammengesetzt	mehrere Entscheidungen in einem Schritt	In Einzelschritte zerlegen, dann je Schritt entscheiden

zerlegen. Das ist der Eingriff mit dem mit Abstand größten Ertrag [1, 3].

9.2. Empfehlungen zur Modell- und Prompt-Auswahl

Steht fest, dass ein LLM eingesetzt werden soll, sind drei Festlegungen zu treffen. Sie werden hier in der Reihenfolge ihrer Wirkung behandelt, die der verbreiteten Aufmerksamkeitsverteilung genau entgegengesetzt ist: Der Ausführungsmodus wirkt am stärksten, die Prompt-Gestaltung deutlich, die Modellwahl am wenigsten.

Ausführungsmodus: Die Wahl zwischen Code-Generierung und Direktverarbeitung ist keine technische Detailfrage, sondern bestimmt, welche Fähigkeit des Modells überhaupt zur Wirkung kommt. Für den Regelbetrieb ist der Code-Generierungs-Modus vorzuziehen, und zwar aus drei unabhängigen Gründen. Er ist um mehr als eine Größenordnung günstiger, 0,176 bis 0,216 US-Dollar gegenüber 1,32 bis 1,71 für dieselbe Strecke auf der halben Datenmenge. Er erzeugt ein prüfbares, versionierbares Artefakt, das sich einmal abnehmen und danach unverändert ausführen lässt, während die Direktverarbeitung bei jedem Lauf neu und potenziell anders entscheidet. Und er skaliert unabhängig vom Datenvolumen, während der Direkt-Modus durch das Ausgabefenster begrenzt ist und im Rahmen dieser Arbeit eine Halbierung des Datensatzes erzwang. Der Direkt-Modus ist demnach nicht als durchgängiger Verarbeitungsmodus zu empfehlen, sondern gezielt für einzelne, semantisch geprägte Schritte, dort allerdings mit messbarem Gewinn.

Prompt-Gestaltung: Hier ist die wichtigste Empfehlung eine über die Reihenfolge: Der Zuschnitt der Aufgabe ist vor dem Prompt zu optimieren, nicht danach. Die in Abschnitt 8.6.1 berichtete Gegenüberstellung zeigt, dass der auffälligste Strategieunterschied der Faktormatrix, 53 gegenüber 72 Prozent bei der Duplikaterkennung, vollständig verschwindet, sobald dieselben Schritte in angemessenem Zuschnitt und mit sauberer Eingabe gestellt werden: Dort liegen alle drei Strategien zwischen 98,8 und 99,4 Prozent. Beispiele und schrittweise Herleitung helfen einem Modell, eine überladene Aufgabe zu bewältigen. Sie werden entbehrlich, sobald die Aufgabe nicht mehr überladen ist. Wer zuerst den Prompt optimiert, arbeitet daher an einem Symptom.

Wo eine Aufgabe sich nicht weiter zerlegen lässt, gilt die Wirkung als aufgabengebunden und nicht als allgemein. Belegt sind genau zwei Zuordnungen: Beispiele wirken bei Entscheidungen über Ähnlichkeit, wie sie die unscharfe Duplikaterkennung verlangt, dort führte das Few-Shot-Prompting mit 82,0 gegenüber 26,6 Prozent für den Zero-Shot-Prompt. Die Aufforderung zur schrittweisen Herleitung wirkt bei mehrstufigen Auswahlregeln wie der schlüsselbasierten Entdoppelung. Bei den übrigen sieben Aufgaben war zwischen den Strategien kein Unterschied messbar, sodass dort der einfachste und billigste Prompt zu wählen ist.

Ausdrücklich zu warnen ist davor, eine an einer Aufgabe bewährte Strategie zu verallgemeinern. Dasselbe Few-Shot-Prompting, das bei den unscharfen Duplikaten 55 Prozentpunkte gewinnt, ist bei sieben der neun Aufgaben ohne jede Wirkung und verursacht dort lediglich den dreifachen Token-Aufwand. Wirksam ist zusätzlicher Kontext dagegen zuverlässig dann, wenn er einen konkret beobachteten Mangel adressiert: Die Angabe der Eingabe-Datentypen behob ein stilles Versagen der Spaltenauswahl, und die ausdrückliche Benennung der erforderlichen Teilschritte hob die zusammengesetzte Aufgabe von einem Wert nahe null auf bis zu 75,8 Prozent [21, 22].

Modellwahl: Die drei untersuchten Cloud-Modelle liegen so eng beieinander, dass sich eine allgemeine Empfehlung nicht begründen lässt. Auch die Kosten und Antwortzeiten unterscheiden sich nur moderat. Sinnvoll ist stattdessen eine Auswahl anhand der konkret vorliegenden Aufgabenart, wobei die Modelle in ihren Fehlermustern durchaus unterscheidbar sind. In der gedrittelten Strecke, also bei angemessenem Aufgabenzuschnitt, verschwindet auch dieser Unterschied: Dort liegen alle drei zwischen 99,0 und 99,3 Prozent.

Für das lokal betriebene Modell gilt eine eigene Empfehlung. Sein in Abschnitt 8.2 berichteter Rückstand ist maßgeblich durch die verfügbare Hardware bedingt und keine Aussage über lokal betreibbare Modelle im Allgemeinen. Er entstand zudem weit überwiegend daraus, überhaupt lauffähigen Code zu erzeugen, und nicht aus einem mangelnden Verständnis der Aufgaben. Für Vorhaben, in denen Daten das eigene Netz nicht verlassen dürfen oder in denen dauerhaft hohe Volumina verarbeitet werden, ist eine eigene Erprobung auf leistungsfähigerer Hardware daher lohnend. Der Wegfall nutzungsabhängiger Kosten wiegt bei anhaltendem Betrieb schwer, ist aber gegen Anschaffung und Betrieb der Hardware zu rechnen, die diese Arbeit nicht erhoben hat. Die Bedingungen dafür lassen sich aus den Ergebnissen benennen: ausschließlich im Code-Generierungs-Modus, mit kleinteilig zugeschnittenen Aufgaben und mit mehreren zugestandenen Versuchen je Schritt. Dass diese Bedingungen den Ausschlag geben, zeigt das hier eingesetzte, hardwarebegrenzte Modell selbst: Vier der neun Aufgaben löste es mit 100 Prozent und lag bei der schlüsselbasierten Entdoppelung damit über zwei der drei Cloud-Modelle, während es bei den übrigen weit überwiegend daran scheiterte, überhaupt lauffähigen Code zu erzeugen [5, 18].

Messung statt Übernahme: Keine dieser Empfehlungen ersetzt die Messung im eigenen Kontext, und der in Abschnitt 8.5 berichtete Bestätigungslauf zeigt, welche Anforderungen eine solche Messung erfüllen muss. Eine Konfiguration ist an Daten zu bewerten, gegen die sie nicht justiert wurde, und mehrfach auszuführen: Der Endschritt derselben Konfiguration schwankte über fünf identische Wiederholungen zwischen 27,3 und 72,7 Prozent. Wer auf dieser Grundlage eine einzelne Messung zur Entscheidung heranzieht, wählt zu einem erheblichen Teil zufällig.

9.3. Hybride Architekturen

Aus den vorstehenden Empfehlungen ergibt sich, dass weder ein rein regelbasiertes noch ein rein KI-gestütztes Vorgehen der jeweils anderen Variante über alle Aufgaben hinweg überlegen ist. Die naheliegende Konsequenz ist eine Architektur, die beide Verfahren nicht als Alternativen, sondern als Bausteine derselben Strecke behandelt und je Schritt dasjenige einsetzt, dessen Stärken zur Aufgabe passen [12, 30].

Der erste Baustein ist die bereits begründete Zerlegung. Sie ist hier nicht nur eine Frage der Übersichtlichkeit, sondern die Voraussetzung dafür, dass je Schritt überhaupt eine Verfahrenswahl möglich wird, und zugleich die einzige Stelle, an der sich Qualität

ohne zusätzliche Kosten gewinnen lässt.

Der zweite Baustein ist die Zuordnung nach Tabelle 9.1: regelbasierte Ausführung für die syntaktisch beschreibbaren Schritte, ein Modell im Direkt-Modus für die wenigen semantisch geprägten Schritte, und die abschließende Aggregation wieder regelbasiert oder über generierten Code. Für die untersuchte Strecke bedeutet dies konkret, dass sieben der neun Schritte regelbasiert auszuführen wären und lediglich die Länder-Vereinheitlichung sowie die unscharfe Duplikaterkennung ein Modell benötigen. Der Kostenanteil einer solchen Architektur liegt damit bei einem Bruchteil des durchgängigen LLM-Einsatzes, während sie dessen Vorteil genau dort behält, wo er entsteht.

Der dritte und wichtigste Baustein ist die Prüfung nach jedem Schritt, die unmittelbar aus dem Befund zur Fehlerfortpflanzung folgt. Deren Tücke liegt darin, dass die verursachenden Schritte für sich betrachtet unauffällig wirken: Sie erreichten durchweg über 96 Prozent, und dennoch genügten die wenigen falsch gewählten Duplikat-Repräsentanten, um das Endergebnis um mehr als ein Drittel zu verschieben. Entscheidend ist zudem, dass diese Prüfung nicht auf der Abwesenheit von Fehlermeldungen beruhen darf, wie der Fall des stillen Versagens aus Abschnitt 8.9.1 zeigt. Zu prüfen sind daher inhaltliche Eigenschaften des Ergebnisses, etwa dass die Länderspalte ausschließlich zweistellige ISO-Codes enthält, dass die Datensatzanzahl im erwarteten Bereich liegt oder dass keine Bestellung ohne zugehörigen Kunden zurückbleibt [7, 11].

Der vierte Baustein betrifft den Umgang mit dem Nichtdeterminismus und ergibt sich aus der Reproduzierbarkeitsmessung. Da die Streuung des Endergebnisses fast vollständig aus der Verstärkung entlang der Kette und nicht aus der Textgenerierung selbst entstand, im isolierten Durchlauf lieferten alle Wiederholungen exakt denselben Wert, ist sie durch Festschreibung beherrschbar. Sowohl der einmal geprüfte Code als auch die Zwischenergebnisse der frühen, semantisch geprägten Schritte sind als feste Artefakte zu übernehmen, statt sie bei jeder Ausführung neu erzeugen zu lassen. Ein Modell wird damit zum Werkzeug der Entwicklungszeit, dessen Ergebnis in den Betrieb übernommen wird, statt zu einer Komponente des Betriebs, die bei jedem Lauf neu entscheidet.

Diese Aufteilung beschreibt zugleich die realistische Rolle von LLMs in der Datenintegration, wie sie sich aus den Ergebnissen dieser Arbeit ergibt. Sie ersetzen die klassi-

sche Verarbeitung nicht, sondern verschieben die Grenze dessen, was ohne vorherige vollständige Spezifikation automatisierbar ist. Ihr Beitrag liegt in den wenigen Schritten, in denen eine Vorschrift nicht vollständig formulierbar ist, sowie in der Verkürzung der Zeit, die von der Aufgabenstellung bis zu einer lauffähigen ersten Umsetzung vergeht, und beides ist erheblich, ohne die Aufgabe der prüfenden Instanz zu ersetzen [2, 23].

10. Fazit und Ausblick

10.1. Zusammenfassung der Ergebnisse

Ziel dieser Arbeit war eine experimentelle Evaluierung, inwieweit sich aktuelle LLMs für ausgewählte ETL-Teilaufgaben eignen. Dazu wurde ein synthetischer Datensatz mit kontrolliert eingebauten Qualitätsproblemen erstellt, neun ETL-Aufgaben in drei Kategorien und drei Schwierigkeitsstufen definiert und eine regelbasierte Vergleichspipeline implementiert, deren Ergebnis als Maßstab dient. Die Modelle wurden auf mehreren Untersuchungsebenen gemessen: in einer vollständig gekreuzten Faktormatrix aus Modell, Prompting-Strategie und Aufgabe, in einer verketteten Pipeline aus neun aufeinander aufbauenden Schritten, in einem sonst identischen isolierten Durchlauf, der jedem Schritt fehlerfreie Eingabedaten bereitstellt und damit eigene Leistung von geerbtem Fehler trennt, sowie in zwei Ebenen, die den Zuschnitt der Aufgabenstellung selbst zur Untersuchungsgröße machen. Ergänzend wurden zwei Ausführungsmodi gegenübergestellt: die Erzeugung von Verarbeitungscode und die unmittelbare Verarbeitung der Daten im Prompt. Sämtliche berichteten Ergebnisse stammen dabei aus einem Datensatz, der zum Zeitpunkt der Festlegung aller Prompts und Auswertungsregeln unbekannt war und anschließend ohne weitere Anpassung mehrfach durchlaufen wurde. Die Entwicklung der Untersuchungsanlage erfolgte auf einem separaten, nicht ausgewerteten Datensatz.

Der über alle Ebenen hinweg stabilste Befund ist die Dominanz des Aufgabentyps: Zwischen dem besten und dem schwächsten kommerziellen Modell liegen rund acht Prozentpunkte, zwischen der am besten und der am schlechtesten gelösten Aufgabe nahezu hundert. Welche Teilaufgaben einem Modell übertragen werden, ist für ein ETL-Vorhaben damit erheblich folgenreicher als die Wahl des Modells. Als zweite Einflussgröße erwies sich der Ausführungsmodus, der bislang kaum als eigene Gestaltungsdimension behandelt wird: Der erwartete Vorteil bei semantisch geprägten Auf-

gaben trat allein in der direkten Verarbeitung ein, während dieselbe Aufgabe in generierten Code gefasst auf das Niveau der regelbasierten Lösung zurückfiel. Umgekehrt scheiterte die direkte Verarbeitung an der abschließenden Aggregation nahezu vollständig und kostete je Zeile rund das Fünzfach. Die beiden Modi sind folglich nicht besser oder schlechter, sondern für unterschiedliche Aufgabenarten geeignet.

Am stärksten wirkt jedoch der Zuschnitt der Aufgabenstellung, und zwar in beide Richtungen. Die zusammengesetzte Aufgabe aus Join, Bereinigung und Aggregation blieb als Einzelaufgabe von jedem Modell und jeder Strategie ungelöst, wurde als Glied einer Kette mit fehlerfreier Eingabe jedoch vollständig korrekt bewältigt und in unzerlegter, aber ausdrücklich in Teilschritte gegliederter Form immerhin zu 75,8 Prozent. Derselbe Eingriff hebt auch den scheinbaren Strategieeffekt auf: Über alle Aufgaben gemittelt liegt Few-Shot bei den kommerziellen Modellen 6,1 Prozentpunkte vor Zero-Shot, doch dieser Vorsprung entsteht nahezu vollständig aus einer einzigen der neun Aufgaben, und in angemessen zugeschnittenen Gruppen liegen alle drei Strategien zwischen 98,8 und 99,4 Prozent. Was in der Einzelaufgaben-Matrix als Wirkung der Prompt-Gestaltung erscheint, ist demnach zu erheblichem Teil eine Wirkung des Zuschnitts, was die übliche Reihenfolge der Gestaltungsentscheidungen umkehrt: erst zuschneiden, dann formulieren.

Zwei weitere Befunde betreffen die Bewertung solcher Strecken selbst. Die Trennung von eigener Schrittleistung und geerbtem Fehler zeigt, dass der Endschrift je nach Modell zwischen 30,9 und 41,8 Prozentpunkte verlor, die ihm nicht zuzurechnen waren. Die verursachenden Schritte erreichten dabei durchweg über 96 Prozent und wirkten unauffällig. Dass die regelbasierte Pipeline an derselben Stelle mit 47,0 Prozentpunkten sogar mehr verlor als jedes Modell, verweist die Ursache in die Architektur der Kette. Evaluierungen einzelner ETL-Operationen überschätzen die Leistungsfähigkeit in realen Strecken daher systematisch, während rein verkettete Betrachtungen sie unterschätzen. Ebenso schwankte der Endschrift derselben Konfiguration über fünf identische Wiederholungen zwischen 27,3 und 72,7 Prozent, lieferte im isolierten Durchlauf aber jedes Mal exakt denselben Wert: Die Instabilität entsteht nicht aus der probabilistischen Textgenerierung, sondern aus deren Verstärkung entlang der Kette, und ist damit eine Eigenschaft der Architektur und nicht des Modells.

10.2. Beantwortung der Forschungsfragen

FF1: Die erste Forschungsfrage richtete sich auf die Eignung aktueller LLMs für die Bearbeitung typischer ETL-Teilaufgaben im Vergleich zu einer klassischen, regelbasierten Pipeline, auf die Abhängigkeit dieser Eignung von Aufgabentyp und Ausführungsform sowie auf das Vorgehen, das die Ergebnisse je Aufgabentyp nahelegen.

Auf den ersten Teil der Frage lautet die Antwort, dass die Eignung nicht pauschal, sondern ausschließlich je Aufgabentyp und Ausführungsmodus zu beurteilen ist. Für die sechs formal beschreibbaren Aufgaben erreichen die kommerziellen Modelle mit gut gewähltem Prompt das Niveau der regelbasierten Pipeline, ohne dieses zu übertreffen. Ihr Beitrag liegt dort nicht im Ergebnis, sondern darin, dass die Verarbeitungsvorschrift nicht zuvor formuliert werden musste. Für die semantisch geprägten Aufgaben übertreffen sie die regelbasierte Lösung, allerdings nur im Direkt-Modus und nur um wenige Prozentpunkte. Für die zusammengesetzte Aufgabe versagen alle untersuchten Ansätze einschließlich der regelbasierten Pipeline, solange sie ungeteilt gestellt wird. Die in H1 formulierte Erwartung ist damit in ihrem Kern bestätigt – die starke Abhängigkeit vom Aufgabentyp, die Ebenbürtigkeit bei formalisierbaren Aufgaben, der Zuverlässigkeitsverlust bei zunehmender struktureller Komplexität und die Abwesenheit eines durchgängig überlegenen Ansatzes –, in einem Teilaspekt jedoch einzuschränken: Der erwartete Vorteil bei semantischen Aufgaben tritt nicht generell ein, sondern ist an den Ausführungsmodus gebunden und bleibt im primär untersuchten Modus der Code-Generierung aus.

Zum zweiten Teil der Frage, der Abhängigkeit von der Ausführungsform, lautet die Antwort, dass diese die Eignung ebenso stark bestimmt wie der Aufgabentyp selbst. Der Ausführungsmodus legt fest, welche Fähigkeit überhaupt zur Wirkung kommt: Die Direktverarbeitung beherrscht die semantische Einzelfallentscheidung, scheitert aber an Aggregationen und am Ausgabefenster. Die Code-Generierung rechnet zuverlässig, verliert dafür das Weltwissen an die Grenzen einer aufgezählten Tabelle. Der Zuschnitt wirkt noch stärker: Dieselben Verarbeitungsinhalte ergeben je nach Zuschnitt 0,1 oder 100 Prozent. Hypothese H1a ist damit in beiden Teilen bestätigt.

Auf den dritten Teil der Frage folgt die Empfehlung eines hybriden Vorgehens, das in Abschnitt 9.3 ausgeführt ist: regelbasierte Ausführung für die syntaktisch beschreibbaren Schritte, ein Modell im Direkt-Modus für die wenigen semantisch geprägten Schritte, regelbasierte oder codegenerierte Ausführung für die abschließende Aggregation,

und dies durchgehend in kleinteilig zugeschnittenen Schritten mit einer inhaltlichen Prüfung nach jedem einzelnen. Für die untersuchte Strecke bedeutet dies, dass sieben der neun Schritte regelbasiert auszuführen wären und lediglich zwei ein Modell benötigen. Ein durchgängiger LLM-Einsatz ist auf Grundlage dieser Ergebnisse nicht zu empfehlen, ein durchgängiger Verzicht ebenso wenig. Ausdrücklich festzuhalten ist dabei, dass diese hybride Variante aus den gemessenen Befunden abgeleitet, aber nicht selbst als Ganzes implementiert und gemessen wurde. Sie ist eine begründete Empfehlung, kein Ergebnis.

FF2: Die zweite Forschungsfrage richtete sich auf den Einfluss der Prompting-Strategie auf die Ergebnisqualität und auf die daraus abzuleitende Empfehlung.

Der Einfluss ist erheblich, aber ungleich verteilt und stark vom Aufgabenzuschnitt abhängig. Bei sieben der neun Aufgaben war zwischen den Strategien kein Unterschied messbar. Bei den beiden anspruchsvollen Entdoppelungsaufgaben entschied die Wahl über bis zu 55 Prozentpunkte. Wo die Prompt-Gestaltung überhaupt wirkt, ist ihr Gewicht demnach mit dem der Modellwahl vergleichbar, nur wirkt sie eben selten.

H2 ist in ihrer allgemeinen Form zu verwerfen, allerdings mit einer Begründung, die von der zunächst naheliegenden abweicht. Aufwändigere Strategien verbessern die Ergebnisqualität im Mittel durchaus messbar, doch dieser Mittelwert bildet keinen allgemeinen Zusammenhang ab, sondern einen Einzelfall: Er entsteht nahezu vollständig aus einer der neun Aufgaben. In sieben Aufgaben bleibt der dreifache Token-Aufwand ohne jede Wirkung. Zutreffend bleibt der erwartete abnehmende Grenznutzen in verschärfter Form: Er ist bei unpassendem Zuschnitt negativ. Bestätigt hat sich vor allem die erwartete Aufgabenabhängigkeit der Wirkung.

Die Empfehlung, die daraus folgt, betrifft weniger die Wahl der Strategie als die Reihenfolge der Entscheidungen. Wird dieselbe Verarbeitung in angemessen zugeschnittenen Gruppen statt als überladene Einzelaufgabe gestellt, liegen alle drei Strategien zwischen 98,8 und 99,4 Prozent, und die Frage nach der besten Strategie erübrigt sich. Zuerst ist daher die Aufgabe zu zerlegen. Bleibt danach eine Aufgabe übrig, die sich nicht weiter teilen lässt, gelten die beiden belegten Zuordnungen: Beispiele bei Ähnlichkeitsentscheidungen, schrittweise Herleitung bei mehrstufigen Auswahlregeln, im Übrigen der einfachste und billigste Prompt. Zusätzlicher Kontext ist ausschließlich als Reaktion auf einen konkret beobachteten Mangel zu ergänzen, so behob die

Angabe der Eingabe-Datentypen ein stilles Versagen der Spaltenauswahl, niemals vorsorglich.

Reproduzierbarkeit: Die querschnittlich erhobene Reproduzierbarkeitsdimension fügt beiden Antworten eine Einschränkung hinzu. Die niedrig gewählte Temperatur begrenzte die Variabilität wirksam, beseitigte sie aber nicht: In gut einem Fünftel der Konfigurationen führte derselbe Prompt bei demselben Modell zu messbar unterschiedlichen Ergebnissen, und die Instabilität konzentrierte sich auf genau jene Aufgaben, die auch inhaltlich am schwersten fielen. Gerade dort, wo ein Ergebnis besonders sorgfältig zu prüfen wäre, ist es zugleich am wenigsten verlässlich wiederholbar. Da die Verstärkung entlang der Kette sich als der dominierende Beitrag zu dieser Streuung erwies, ist sie durch Festschreibung des geprüften Codes und der frühen Zwischenergebnisse jedoch weitgehend beherrschbar.

10.3. Limitationen der Arbeit

Die vorstehenden Aussagen unterliegen mehreren Einschränkungen, die bei ihrer Übertragung zu berücksichtigen sind.

Die erste und gewichtigste betrifft den Datensatz. Er ist synthetisch erzeugt, und seine Qualitätsprobleme entstammen einer bekannten, endlichen Menge von Mustern. Diese Anlage war notwendig, um eine eindeutige Soll-Lösung zu besitzen und die Ansätze überhaupt vergleichbar messen zu können, begünstigt jedoch systematisch die regelbasierte Vergleichspipeline: Deren Zuordnungstabelle und Normalisierungsregel wurden gegen genau diese Muster entworfen und erreichen deshalb Werte, die in einem realen Bestand mit unbekannten Abweichungsmustern nicht zu erwarten wären. Der in dieser Arbeit gemessene Vorsprung der regelbasierten Lösung ist damit als Obergrenze zu lesen, und das Verhältnis verschiebt sich in realen Beständen zugunsten der semantischen Verfahren. Um welchen Betrag er sich verschiebt, lässt sich aus diesen Daten allerdings nicht bestimmen.

Die zweite Einschränkung betrifft die Größenordnung. Der Datensatz umfasst einige hundert Kunden und einige tausend Bestellungen, was für die inhaltliche Bewertung ausreicht, aber keine Aussage über das Verhalten bei produktionstypischen Datenmengen erlaubt. Für den Code-Generierungs-Modus ist dies unkritisch, da die Datenmenge dort nicht durch das Modell hindurchgeht. Für den Direkt-Modus hingegen ist

es die bestimmende Grenze, die im Rahmen dieser Arbeit bereits bei wenigen hundert Datensätzen eine Halbierung des Datensatzes erzwang.

Eng damit verbunden ist eine Einschränkung zum lokal betriebenen Modell. Sein Rückstand ist maßgeblich durch den verfügbaren Grafikspeicher bestimmt, der die Größe des ausführbaren Modells begrenzt. Gemessen wurde insofern nicht allein Modellqualität, sondern auch die Ausstattung des ausführenden Rechners. Aus diesem Grund wurde es aus der verketteten Pipeline, den Zuschnitt-Ebenen und dem Modusvergleich herausgenommen, eine Entscheidung, die die Vergleichbarkeit dieser Ebenen sichert, zugleich aber bedeutet, dass für lokal betreibbare Modelle nur die Ergebnisse der Faktormatrix vorliegen. Ob ein größeres, auf leistungsfähigerer Hardware ausführbares Modell den Abstand schließt, bleibt hier offen. Die Fehlerverteilung, weit überwiegend nicht lauffähiger Code statt inhaltlicher Fehler, legt es nahe, belegt es aber nicht.

Die dritte Einschränkung betrifft die Momentaufnahme. Untersucht wurden vier Modelle zu einem bestimmten Zeitpunkt in einem Feld, das sich in Monatsabständen verändert. Die exakte Versionskennung wurde bei jeder Ausführung protokolliert, sodass die Läufe dokumentiert bleiben. Da die kommerziellen Modelle jedoch serverseitig gepflegt werden, lässt sich ihr Verhalten weder einfrieren noch nach einer Modellaktualisierung wiederherstellen. Die konkreten Zahlenwerte sind daher nicht als dauerhaft gültig zu betrachten – die Wirkungszusammenhänge, aus denen sie entstehen, hingegen sehr wohl, da sie in ihren Ursachen benannt und über mehrere Untersuchungsebenen hinweg übereinstimmend beobachtet wurden.

Die vierte Einschränkung betrifft die Zahl der Wiederholungen. Für die Faktormatrix wurden zwei Wiederholungen je Konfiguration ausgeführt, für die verkettete Pipeline in beiden Formen je fünf und für die beiden Ebenen zum Aufgabenzuschnitt je zwei. Allein die direkten Einzelaufgaben liegen als einmalige Messung vor. Angesichts der beobachteten Streuung – allein der Endschritt schwankte über fünf Wiederholungen um bis zu 45 Prozentpunkte – ist dies eine schmale Grundlage für Aussagen über einzelne Konfigurationen. Die Befunde dieser Arbeit sind deshalb bewusst auf Ebene der Muster formuliert, die über Modelle und Aufgaben hinweg wiederkehren, und nicht auf Ebene einzelner Zahlenwerte.

Damit verbunden ist eine methodische Einschränkung, die ausdrücklich zu benennen ist: Die Auswertung verzichtet vollständig auf Inferenzstatistik. Bei zwei Wiederholungen je Konfiguration wären Signifikanztests, Konfidenzintervalle oder Effektstärken

nicht sinnvoll zu bestimmen, weshalb sämtliche Unterschiede als beobachtete Werte und nicht als statistisch abgesicherte Effekte zu lesen sind. Belastbar sind vor diesem Hintergrund allein Unterschiede, die die beobachtete Streuung deutlich übersteigen. Der Abstand zwischen den Ausführungsmodi bei der zusammengesetzten Aufgabe etwa liegt bei mehr als einer Größenordnung, der zwischen den Modellen mit acht Prozentpunkten dagegen in derselben Größenordnung wie die Streuung selbst und ist entsprechend zurückhaltend interpretiert worden.

Die fünfte Einschränkung betrifft den Zuschnitt der Untersuchung. Betrachtet wurde die Transformationsphase des ETL-Prozesses. Extraktion und Ladevorgang blieben ebenso außen vor wie Aspekte des produktiven Betriebs, etwa Fehlerbehandlung, Überwachung, inkrementelle Verarbeitung oder Datenschutzanforderungen beim Übertragen von Daten an externe Anbieter. Gerade der letzte Punkt ist für die praktische Anwendbarkeit erheblich und wurde hier nur insoweit berührt, als mit dem lokal betriebenen Modell eine Option ohne Datenabfluss mituntersucht wurde.

Schließlich ist die Rolle der prüfenden Instanz zu benennen. Die Untersuchung hat die Modelle weitgehend autonom arbeiten lassen und ihre Ergebnisse gegen eine bekannte Soll-Lösung bewertet. Der in der Praxis dominierende Einsatz als Assistenz einer Fachkraft, die den erzeugten Code liest, korrigiert und verantwortet, wurde damit nicht abgebildet. Die in dieser Arbeit gemessenen Werte beschreiben folglich, was ohne menschliche Nachbearbeitung entsteht, was zugleich die Untergrenze dessen markiert, was im assistierten Einsatz zu erwarten ist.

10.4. Ausblick auf weitere Forschung

Aus den Ergebnissen und ihren Grenzen ergeben sich mehrere Anschlussfragen.

Am unmittelbarsten anschlussfähig ist die Übertragung auf reale Datenbestände. Da der Vorsprung der regelbasierten Pipeline maßgeblich daraus resultiert, dass ihre Regeln gegen bekannte Abweichungsmuster entworfen wurden, wäre eine Wiederholung des Vergleichs auf einem gewachsenen Bestand mit unbekannten Mustern der aussagekräftigste nächste Schritt. Zu erwarten ist, dass sich das Verhältnis zugunsten der semantischen Verfahren verschiebt. Die Größe dieser Verschiebung ist die eigentlich interessante Messgröße und zugleich diejenige, die über die wirtschaftliche Tragfähigkeit eines LLM-Einsatzes entscheidet.

Ein zweiter Ansatzpunkt ist die in Abschnitt 9.3 skizzierte hybride Architektur, die aus den Ergebnissen abgeleitet, aber nicht mehr als Ganzes implementiert und gemessen wurde. Eine Umsetzung, die regelbasierte Schritte, gezielt eingesetzte Direkt-Modus-Schritte und eine Prüfung nach jedem Schritt tatsächlich verbindet, ließe sich unmittelbar gegen die hier gemessenen rein regelbasierten und rein KI-gestützten Varianten stellen und würde zeigen, ob die erwartete Kombination der Stärken auch in der Ausführung eintritt.

Der dritte Ansatzpunkt betrifft die Fehlerfortpflanzung, deren Messung diese Arbeit auf den Vergleich zweier Durchläufe stützt. Offen bleibt, welche Eigenschaften eines Zwischenergebnisses seine Verwendbarkeit in nachgelagerten Schritten bestimmen. Die Ergebnisse legen nahe, dass nicht der Umfang, sondern die Art der verbliebenen Abweichung entscheidet: Schritte mit über 96 Prozent Genauigkeit verschoben das Endergebnis um mehr als ein Drittel, weil ihre wenigen Abweichungen ausgerechnet die Verknüpfungsschlüssel betrafen, während gleichmäßig verteilte kleinere Ungenauigkeiten folgenlos blieben. Eine Kennzahl, die diese Unterscheidung abbildet, wäre für die Überwachung KI-gestützter Strecken erheblich wertvoller als die übliche Genauigkeit je Schritt.

Ein vierter Ansatzpunkt ergibt sich aus dem Befund zum Ausführungsmodus. Da die direkte Verarbeitung bei semantischen Einzelentscheidungen überlegen, bei Aggregationen über viele Datensätze jedoch unbrauchbar ist, liegt eine Kombination nahe, in der das Modell die semantische Entscheidung je Wert trifft und deren Ergebnis anschließend deterministisch weiterverarbeitet wird, etwa als generierte Zuordnungstabelle statt als generierte Aufzählungslogik. Ob eine solche Zwischenform den Genauigkeitsvorteil des Direkt-Modus bei dessen Kostenstruktur vermeidet, ist eine unmittelbar prüfbare Frage.

Schließlich verdient der in Abschnitt 10.3 angesprochene assistierte Einsatz eigene Aufmerksamkeit. Diese Arbeit hat gemessen, was Modelle ohne menschliche Nachbearbeitung leisten. Die praktisch relevantere Frage ist, wie viel Aufwand eine Fachkraft benötigt, um ein erzeugtes Ergebnis auf ein verantwortbares Niveau zu bringen, und ob dieser Aufwand geringer ausfällt als der einer eigenständigen Implementierung. Gerade der in dieser Arbeit dokumentierte Fall des stillen Versagens – eines Codes, der fehlerfrei durchläuft, schemakonform ausgibt und dennoch wirkungslos bleibt – legt nahe, dass diese Frage nicht allein über die Ergebnisqualität, sondern wesentlich über den Prüfaufwand zu beantworten ist.

Literatur

- [1] Bilal Khan u. a. „An Overview of ETL Techniques, Tools, Processes and Evaluations in Data Warehousing“. In: *Journal on Big Data* 6 (2024), S. 1–20. ISSN: 25790048. DOI: 10.32604/jbd.2023.046223. URL: <https://www.proquest.com/docview/3200128360/abstract/ABA55C04CFAC48B2PQ/1> (besucht am 08. 05. 2026).
- [2] Afef Walha, Faiza Ghazzi und Faiez Gargouri. „Data Integration from Traditional to Big Data: Main Features and Comparisons of ETL Approaches“. In: *The Journal of Supercomputing* 80.19 (1. Dez. 2024), S. 26687–26725. ISSN: 1573-0484. DOI: 10.1007/s11227-024-06413-1. URL: <https://doi.org/10.1007/s11227-024-06413-1> (besucht am 08. 05. 2026).
- [3] Kushvanth Chowdary Nagabhyru. *Bridging Traditional ETL Pipelines with AI Enhanced Data Workflows: Foundations of Intelligent Automation in Data Engineering*. 24. Nov. 2022. DOI: 10.2139/ssrn.5505199. Social Science Research Network: 5505199. URL: <https://papers.ssrn.com/abstract=5505199> (besucht am 08. 05. 2026). Vorveröffentlichung.
- [4] Chandran Ravi. „ETL (Extract, Transform & Load) Automation“. In: *International Journal of Emerging Trends in Computer Science and Information Technology* 6.1 (12. Feb. 2025), S. 52–55. ISSN: 3050-9246. DOI: 10.63282/3050-9246.IJETCSIT-V6I1P106. URL: <https://www.ijetcsit.org/index.php/ijetcsit/article/view/37> (besucht am 08. 05. 2026).
- [5] Wayne Xin Zhao u. a. *A Survey of Large Language Models*. 18. März 2026. DOI: 10.48550/arXiv.2303.18223. arXiv: 2303.18223 [cs]. URL: <http://arxiv.org/abs/2303.18223> (besucht am 08. 05. 2026). Vorveröffentlichung.
- [6] Hossein Hassani und Emmanuel Sirmal Silva. „The Role of ChatGPT in Data Science: How AI-Assisted Conversational Interfaces Are Revolutionizing the Field“. In: *Big Data and Cognitive Computing* 7.2 (Juni 2023), S. 62. ISSN: 2504-2289. DOI: 10.3390/bdcc7020062. URL: <https://www.mdpi.com/2504-2289/7/2/62> (besucht am 08. 05. 2026).
- [7] Alhassan Mumuni und Fuseini Mumuni. „Automated Data Processing and Feature Engineering for Deep Learning and Big Data Applications: A Survey“. In: *Journal of Information and Intelligence* 3.2 (1. März 2025), S. 113–153. ISSN: 2949-7159. DOI: 10.1016/j.jiixd.2024.01.002. URL: <https://www.sciencedirect.com/science/article/pii/S2949715924000027> (besucht am 08. 05. 2026).

-
- [8] Chaimae Boulahia u. a. „The Multi-Criteria Evaluation of Research Efforts Based on ETL Software: From Business Intelligence Approach to Big Data and Semantic Approaches“. In: *Evolutionary Intelligence* 17.4 (1. Aug. 2024), S. 2099–2124. ISSN: 1864-5917. DOI: 10.1007/s12065-023-00899-z. URL: <https://doi.org/10.1007/s12065-023-00899-z> (besucht am 08.05.2026).
- [9] Sandeep Rangineni u. a. „A Review on Enhancing Data Quality for Optimal Data Analytics Performance“. In: *International Journal of Computer Sciences and Engineering* 11 (31. Okt. 2023), S. 51–58. DOI: 10.26438/ijcse/v11i10.5158.
- [10] Sarbaree Mishra. „Automating the Data Integration and ETL Pipelines through Machine Learning to Handle Massive Datasets in the Enterprise“. In: *International Journal of Emerging Research in Engineering and Technology* 1.2 (30. Juni 2020), S. 69–78. DOI: 10.63282/3050-922X.IJERET-V1I2P109. URL: <https://ijeret.org/index.php/ijeret/article/view/231> (besucht am 08.05.2026).
- [11] Fakhitah Ridzuan und Wan Mohd Nazmee Wan Zainon. „A Review on Data Quality Dimensions for Big Data“. In: *Procedia Computer Science*. Seventh Information Systems International Conference (ISICO 2023) 234 (1. Jan. 2024), S. 341–348. ISSN: 1877-0509. DOI: 10.1016/j.procs.2024.03.008. URL: <https://www.sciencedirect.com/science/article/pii/S187705092400365X> (besucht am 08.05.2026).
- [12] Pierre-Olivier Côté u. a. „Data Cleaning and Machine Learning: A Systematic Literature Review“. In: *Automated Software Engineering* 31.2 (11. Juni 2024), S. 54. ISSN: 1573-7535. DOI: 10.1007/s10515-024-00453-w. URL: <https://doi.org/10.1007/s10515-024-00453-w> (besucht am 08.05.2026).
- [13] Raghavender Maddali. „Automating Data Quality Assurance Using Machine Learning in ETL Pipelines“. In: *IJLRP - International Journal of Leading Research Publication* 2.6 (o.J.). ISSN: 2582-8010. DOI: 10.5281/zenodo.15107533. URL: <https://www.ijlrp.com/research-paper.php?id=1455> (besucht am 08.05.2026).
- [14] Muhammad Fakhrul Safitra u. a. „Advancements in Artificial Intelligence and Data Science: Models, Applications, and Challenges“. In: *Procedia Computer Science*. Seventh Information Systems International Conference (ISICO 2023) 234 (1. Jan. 2024), S. 381–388. ISSN: 1877-0509. DOI: 10.1016/j.procs.2024.03.018. URL: <https://www.sciencedirect.com/science/article/pii/S1877050924003752> (besucht am 08.05.2026).

-
- [15] Jasmin Praful Bharadiya. „The Role of Machine Learning in Transforming Business Intelligence“. In: *International Journal of Computing and Artificial Intelligence* 4.1 (1. Jan. 2023), S. 16–24. ISSN: 27076571, 2707658X. DOI: 10.33545/27076571.2023.v4.i1a.60. URL: <https://www.computersciencejournals.com/ijcai/archives/2023.v4.i1.A.60> (besucht am 08.05.2026).
- [16] Zijng Liang u. a. „A Survey of Multimodal Large Language Models“. In: *Proceedings of the 3rd International Conference on Computer, Artificial Intelligence and Control Engineering*. CAICE '24. New York, NY, USA: Association for Computing Machinery, 6. Aug. 2024, S. 405–409. ISBN: 979-8-4007-1694-2. DOI: 10.1145/3672758.3672824. URL: <https://dl.acm.org/doi/10.1145/3672758.3672824> (besucht am 08.05.2026).
- [17] Pengfei Liu u. a. „Pre-Train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing“. In: *ACM Comput. Surv.* 55.9 (16. Jan. 2023), 195:1–195:35. ISSN: 0360-0300. DOI: 10.1145/3560815. URL: <https://dl.acm.org/doi/10.1145/3560815> (besucht am 08.05.2026).
- [18] *The 2026 AI Index Report | Stanford HAI*. o.J. URL: <https://hai.stanford.edu/ai-index/2026-ai-index-report> (besucht am 08.05.2026).
- [19] *Comparison of AI Models across Intelligence, Performance, and Price*. o.J. URL: <https://artificialanalysis.ai/models> (besucht am 08.05.2026).
- [20] Pranab Sahoo u. a. *A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications*. 5. Feb. 2024. arXiv: 2402.07927 [cs.AI]. URL: <https://arxiv.org/abs/2402.07927> (besucht am 17.08.2026).
- [21] Sander Schulhoff u. a. *The Prompt Report: A Systematic Survey of Prompt Engineering Techniques*. 26. Feb. 2025. DOI: 10.48550/arXiv.2406.06608. arXiv: 2406.06608 [cs]. URL: <http://arxiv.org/abs/2406.06608> (besucht am 08.05.2026). Vorveröffentlichung.
- [22] Kaiyan Chang u. a. *Efficient Prompting Methods for Large Language Models: A Survey*. 2. Dez. 2024. DOI: 10.48550/arXiv.2404.01077. arXiv: 2404.01077 [cs]. URL: <http://arxiv.org/abs/2404.01077> (besucht am 08.05.2026). Vorveröffentlichung.
- [23] Jahangir Khan. *Cognitive ETL: Using AI to Interpret Complex Data Relationships*. 10. Nov. 2025. DOI: 10.2139/ssrn.5728984. Social Science Research Network: 5728984. URL: <https://papers.ssrn.com/abstract=5728984> (besucht am 08.05.2026). Vorveröffentlichung.

-
- [24] Lin Long u. a. „On LLMs-Driven Synthetic Data Generation, Curation, and Evaluation: A Survey“. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Findings 2024. Hrsg. von Lun-Wei Ku, Andre Martins und Vivek Sri-kumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, S. 11065–11082. DOI: 10.18653/v1/2024.findings-acl.658. URL: <https://aclanthology.org/2024.findings-acl.658/> (besucht am 08.05.2026).
- [25] Hsin-Yu Chang u. a. *A Survey of Data Synthesis Approaches*. 4. Juli 2024. DOI: 10.48550/arXiv.2407.03672. arXiv: 2407.03672 [cs]. URL: <http://arxiv.org/abs/2407.03672> (besucht am 08.05.2026). Vorveröffentlichung.
- [26] Zhuoyan Li u. a. „Synthetic Data Generation with Large Language Models for Text Classification: Potential and Limitations“. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. EMNLP 2023. Hrsg. von Houda Bouamor, Juan Pino und Kalika Bali. Singapore: Association for Computational Linguistics, Dez. 2023, S. 10443–10461. DOI: 10.18653/v1/2023.emnlp-main.647. URL: <https://aclanthology.org/2023.emnlp-main.647/> (besucht am 08.05.2026).
- [27] Joao Fonseca und Fernando Bacao. „Tabular and Latent Space Synthetic Data Generation: A Literature Review“. In: *Journal of Big Data* 10.1 (10. Juli 2023), S. 115. ISSN: 2196-1115. DOI: 10.1186/s40537-023-00792-7. URL: <https://doi.org/10.1186/s40537-023-00792-7> (besucht am 08.05.2026).
- [28] Mandeep Goyal und Qusay H. Mahmoud. „A Systematic Review of Synthetic Data Generation Techniques Using Generative AI“. In: *Electronics* 13.17 (Jan. 2024), S. 3509. ISSN: 2079-9292. DOI: 10.3390/electronics13173509. URL: <https://www.mdpi.com/2079-9292/13/17/3509> (besucht am 08.05.2026).
- [29] Naveen Reddy Singi Reddy und Mahitha Adapa. „AI-Driven Data Integration: Transforming Enterprise Data Pipelines through Machine Learning“. In: *Journal of Computer Science and Technology Studies* 7.12 (26. Nov. 2025), S. 110–119. ISSN: 2709-104X. DOI: 10.32996/jcsts.2025.7.12.16. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/11533> (besucht am 08.05.2026).
- [30] Janardhan Reddy Kasireddy. „The Role of AI in Modern Data Engineering: Automating ETL and Beyond“. In: *ICT for Global Innovations and Solutions*. Hrsg. von Saurav Bhattacharya. Cham: Springer Nature Switzerland, 2026, S. 667–693. ISBN: 978-3-032-02853-2. DOI: 10.1007/978-3-032-02853-2_47.

-
- [31] Soumen Chakraborty. „Beyond ETL: How AI Agents Are Building Self-Healing Data Pipelines“. In: *Journal of Computer Science and Technology Studies* 7.3 (8. Mai 2025), S. 741–756. ISSN: 2709-104X. DOI: 10.32996/jcsts.2025.7.3.81. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/9427> (besucht am 08.05.2026).
- [32] Raghav Maddali. „Autonomous AI Agents for Real-Time Data Transformation and ETL Automation“. In: *INTERNATIONAL JOURNAL OF RESEARCH AND ANALYTICAL REVIEWS* 10 (23. Juli 2023), S. 12. DOI: 10.5281/zenodo.15096256.
- [33] Jahangir Khan. *Intelligent Anomaly Detection in ETL Workflows Using AI*. 9. Nov. 2025. DOI: 10.2139/ssrn.5729002. Social Science Research Network: 5729002. URL: <https://papers.ssrn.com/abstract=5729002> (besucht am 08.05.2026). Vorveröffentlichung.
- [34] Chiara Van Der Putten. „Transforming Data Flow: Generative AI in ETL Pipeline Automatization“. laurea. Politecnico di Torino, 11. Apr. 2024. 83 S. URL: <https://webthesis.biblio.polito.it/30855/> (besucht am 08.05.2026).
- [35] Shashank A. „AI-Enhanced ETL Processes: Leveraging Artificial Intelligence for Optimized Data Integration Systems“. In: *Journal Of Multidisciplinary* 5.8 (10. Aug. 2025), S. 219–225. ISSN: 2945-3445. URL: <https://sarcouncil.com/2025/8/ai-enhanced-etl-processes-leveraging-artificial-intelligence-for-optimized-data-integration-systems> (besucht am 08.05.2026).
- [36] Dhamotharan Seenivasan. *AI Driven Enhancement of ETL Workflows for Scalable and Efficient Cloud Data Engineering*. 28. Juni 2024. Social Science Research Network: 5153853. URL: <https://papers.ssrn.com/abstract=5153853> (besucht am 08.05.2026). Vorveröffentlichung.
- [37] Indu Joshi u. a. „Synthetic Data in Human Analysis: A Survey“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.7 (Juli 2024), S. 4957–4976. ISSN: 1939-3539. DOI: 10.1109/TPAMI.2024.3362821. URL: <https://ieeexplore.ieee.org/abstract/document/10423161> (besucht am 08.05.2026).

A. Anhang

Der Anhang dokumentiert die Bestandteile der Untersuchung, die für die Nachvollziehbarkeit der Ergebnisse erforderlich sind, den Lesefluss des Haupttextes jedoch unterbrochen hätten: die wörtlichen Prompt-Vorlagen (Abschnitt A.1), die Aufgabentexte der neun ETL-Aufgaben (Abschnitt A.2), den Aufgabentext der explizit gegliederten Variante (Abschnitt A.3) sowie den Verweis auf das Repositorium mit Code, Daten und sämtlichen Ergebnisdatensätzen (Abschnitt A.4).

A.1. Prompt-Vorlagen

Die Vorlagen liegen als versionierte YAML-Dateien vor und bestehen jeweils aus einer Systemanweisung und einer Nutzervorlage mit Platzhaltern, die zur Laufzeit durch den Aufgabentext, die Eingabeverzeichnis und den Ausgabepfad ersetzt werden. Wiedergegeben ist der Wortlaut ohne die Platzhalterauflösung. Die Vorlagen sind auf Englisch formuliert, da die Modelle in dieser Sprache durchgehend trainiert und die Anbieter-Dokumentationen darauf ausgelegt sind. Die Aufgabentexte selbst sind deutsch.

Code-Generierung

v1 – Zero-Shot: *System:* „You are an experienced data engineer. Respond only with executable Python code that uses pandas. No explanations, no markdown code block, no text outside the code.“

Nutzervorlage: „Write a self-contained Python script for the following ETL task: {task_description}. The input files are located in: {input_dir}. Write the result as a Parquet file to exactly: {output_path}. The script must run without any further input and must produce the output file at the specified path.“

v2 – Few-Shot: Systemanweisung wie v1. Die Nutzervorlage stellt der Aufgabe ein vollständig gelöstes Beispiel voran, eingeleitet mit: „Here is one fully solved ETL task as an example of the expected style. It is a different task from the one you must solve; use it only to understand the format and quality of the expected solution.“ Das Beispiel behandelt eine Mitarbeitertabelle mit Leerzeichen in Textspalten, fehlenden Abteilungen, als dekorierte Text abgelegten Gehältern und den Wahrheitswerten „ja“/„nein“. Es

umfasst Aufgabentext, Eingabetabelle und die vollständige Pandas-Lösung. Die Beispielaufgabe ist bewusst keine der neun evaluierten Aufgaben, damit sie keine Musterlösung vorwegnimmt.

v3 – Chain-of-Thought: *System:* „You are an experienced data engineer. Respond only with executable Python code (pandas). Record your reasoning as comments (with #) directly in the code. No text outside the code, no markdown.“

Nutzervorlage: „Solve the following ETL task. Proceed step by step: first break the task down into sub-steps (as comments), then implement each step in code. {task_description} ...“

Direktverarbeitung

Die drei Direkt-Vorlagen entsprechen den vorstehenden in Strategie und Beispiel, verlangen jedoch die Ergebnistabelle statt des Codes.

d1 – Zero-Shot: *System:* „You are a precise data-processing engine. You receive one or more input tables and a data transformation task. Apply the task directly to the data and return ONLY the resulting table as CSV (comma-separated, with a header row), wrapped in a single `csv` code block. Output every result row. No explanations, no prose, no row index column.“

Nutzervorlage: „Task: {task_description}. Input data (CSV): {data}. Return the COMPLETE resulting table as CSV inside one `csv` code block. Include ALL result rows, no index column, no commentary.“

d2 – Few-Shot: Wie d1, ergänzt um dasselbe Beispiel wie v2, jedoch mit der Ergebnistabelle statt des Lösungs codes als Vorbild.

d3 – Chain-of-Thought: Wie d1, ergänzt um die Aufforderung: „Work through it step by step: first identify the individual sub-steps the task consists of, then apply them to

the data one after another, so that each sub-step operates on the result of the previous one.“ Da die Antwort ausschließlich die Ergebnistabelle enthalten darf, bleibt die Zerlegung anders als bei v3 modellintern.

A.2. Aufgabentexte der neun ETL-Aufgaben

Wiedergegeben ist der Wortlaut, der als `task_description` in die Prompt-Vorlage eingesetzt wird. Für die schema-angereicherte Variante tritt jeweils eine Beschreibung des Zielschemas hinzu, die hier aus Platzgründen entfällt.

Datenbereinigung

Einfach (Leerzeichen und fehlende Werte): Lies die Datei `customers_raw.csv` ein. Entferne führende und nachfolgende Leerzeichen aus allen Textspalten. Ersetze fehlende Werte in der Spalte `country` durch „UNKNOWN“. Schreibe das Ergebnis als Parquet-Datei `cleaned_customers.parquet`.

Mittel (Datumsformate): Die Spalte `registered_at` in `customers_raw.csv` enthält Datumsangaben in unterschiedlichen Formaten (ISO-Datum, deutsches Format DD.MM.YYYY, US-Format Month DD YYYY und Unix-Timestamps). Normalisiere alle Werte auf das ISO-Format YYYY-MM-DD. Schreibe das Ergebnis als Parquet-Datei.

Schwer (semantische Vereinheitlichung): Die Spalte `country` in `customers_raw.csv` enthält Länder in verschiedenen Schreibweisen und Sprachen (z. B. „DE“, „Deutschland“, „Germany“, „Deutshcland“). Vereinheitliche alle Werte auf den zweistelligen ISO-3166-1-alpha-2-Code (z. B. „DE“). Werte, die sich nicht eindeutig zuordnen lassen, sollen den Wert „UNKNOWN“ erhalten. Schreibe das Ergebnis als Parquet-Datei.

Duplikaterkennung

Einfach (exakte Duplikate): Lies `customers_raw.csv` ein und entferne exakte Duplikate (Zeilen, in denen alle Spaltenwerte identisch sind). Behalte jeweils das erste Vorkommen. Schreibe das Ergebnis als Parquet-Datei.

Mittel (schlüsselbasiert): Entferne in `customers_raw.csv` Zeilen, in denen die `email`

mehrfach vorkommt. Bei mehreren Zeilen mit gleicher E-Mail soll die Zeile mit dem jüngsten `registered_at`-Wert behalten werden. Schreibe das Ergebnis als Parquet-Datei.

Schwer (unscharfe Duplikate): In `customers_raw.csv` befinden sich Fuzzy-Duplikate: gleiche Person, aber unterschiedliche Schreibweisen des Namens (Typfehler, unterschiedliche Groß-/Kleinschreibung, mit/ohne Mittelinitial) und unterschiedliche E-Mail-Schreibweisen (Punkte vor dem @, unterschiedliche Groß-/Kleinschreibung). Erkenne diese Duplikate und entferne sie. Behalte jeweils die Zeile mit der vollständigsten Information. Schreibe das Ergebnis als Parquet-Datei.

Datentransformation

Einfach (Typkonvertierung): Lies `products_raw.csv` ein. Konvertiere die Spalte `price_eur` von Text (z. B. „19,99“, „€19.99“, „19.99 EUR“) in einen numerischen Wert (Float, Dezimaltrennzeichen Punkt). Konvertiere die Spalte `in_stock` von Text („ja“/„nein“, „true“/„false“, „1“/„0“) in Boolean. Schreibe das Ergebnis als Parquet-Datei.

Mittel (abgeleitete Spalten): Lies `orders_raw.csv` ein. Berechne für jede Bestellzeile eine neue Spalte `total_eur` als `quantity * unit_price_eur`. Extrahiere aus `ordered_at` zusätzlich die Spalten `order_year` (Integer) und `order_month` (Integer, 1–12). Schreibe das Ergebnis als Parquet-Datei.

Schwer (Verknüpfung und Aggregation): Verknüpfe `orders_raw.csv`, `customers_raw.csv` und `products_raw.csv` über die jeweiligen ID-Spalten. Bereinige dabei die Daten so weit wie nötig (Typkonvertierungen, fehlende Werte sinnvoll behandeln). Aggregiere anschließend pro Land (`country_code`, ISO-2) und Produktkategorie (`category`) den Gesamtumsatz (`total_revenue_eur = sum(quantity * unit_price_eur)`) und die Anzahl Bestellungen (`order_count`). Sortiere absteigend nach Gesamtumsatz. Schreibe das Ergebnis als Parquet-Datei.

A.3. Aufgabentext der explizit gegliederten Variante

Dieser Text stellt dieselbe Verarbeitung wie die schwere Transformationsaufgabe, benennt die erforderlichen Teilschritte jedoch ausdrücklich. Abschnitt 8.6.2 wertet die damit erzielten Ergebnisse aus.

Erzeuge aus den drei Roh Tabellen `customers_raw.csv`, `products_raw.csv` und `orders_raw.csv` eine Umsatzauswertung. Führe dazu die folgenden Schritte in dieser Reihenfolge aus:

1. Entferne in `customers_raw.csv` führende und nachfolgende Leerzeichen aus allen Textspalten.
2. Vereinheitliche die Spalte `country` in `customers_raw.csv` auf den zweistelligen ISO-3166-1-alpha-2-Code und lege sie als `country_code` ab. Die Rohwerte liegen in unterschiedlichen Sprachen, als Abkürzungen und mit Tippfehlern vor (z. B. „Deutschland“, „Germany“, „GER“, „Deutchland“ → „DE“). Werte, die sich nicht eindeutig zuordnen lassen, sowie fehlende Werte erhalten „UNKNOWN“.
3. Entferne aus `customers_raw.csv` Datensätze mit mehrfach vorkommender `customer_id` und behalte jeweils das erste Vorkommen.
4. Konvertiere in `products_raw.csv` die Spalte `price_eur` (Text wie „19,99“, „19.99 EUR“) in einen Float und `in_stock` in einen Boolean.
5. Verknüpfe `orders_raw.csv` mit `customers_raw.csv` über `customer_id` und mit `products_raw.csv` über `product_id`. Alle Bestellzeilen aus `orders_raw.csv` bleiben dabei erhalten.
6. Aggregiere pro Land (`country_code`) und Produktkategorie (`category`) den Gesamtumsatz `total_revenue_eur = sum(quantity * unit_price_eur)`, auf zwei Nachkommastellen gerundet, sowie die Anzahl Bestellungen `order_count`. Die Spalten `quantity` und `unit_price_eur` stammen beide aus `orders_raw.csv`; der Preis aus `products_raw.csv` wird für die Umsatzberechnung nicht verwendet. Sortiere absteigend nach `total_revenue_eur`.

A.4. Repository

Sämtlicher Code, die Erzeugungsvorschrift des Datensatzes, die regelbasierte Vergleichspipeline, die Prompt-Vorlagen sowie alle Ergebnisdatensätze der 711 Modelaufrufe stehen unter <https://github.com/Luis198/Bachelorarbeit> zur Verfügung.

Die Struktur folgt den Kapiteln dieser Arbeit:

`project/src/dataset/` Erzeugung des synthetischen Datensatzes einschließlich der Aufgabendefinitionen (Kapitel 5).

`project/src/baseline/` Regelbasierte Vergleichspipeline (Abschnitt 4.6).

`project/src/llm/` **und** `project/prompts/` Anbindung der Modelle und die Prompt-Vorlagen (Abschnitt 6.4).

`project/src/experiments/` Die Ausführungsschichten der einzelnen Untersuchungsebenen (Kapitel 7).

`project/src/evaluation/` Umsetzung der Evaluationskriterien aus Abschnitt 4.5.

`project/results/` Die Ergebnisdatensätze je Lauf sowie die erzeugten Skripte und Ergebnistabellen.

Die berichteten Ergebnisse stammen sämtlich aus dem Datensatz mit dem Startwert 2 beziehungsweise dessen halbierten Fassung mit dem Startwert 102.